

SVAR documentation

Lorenzo Menna*

November 2025

Introduction

This PDF file contains the documentation for SVAR model MatLab functions. The methods presented closely follow Lutkephol (2005). The order of this documentation goes as follows. First section presents the baseline model, second section focuses on the reduced form VAR estimation, third and fourth sections present short-run and long-run identification respectively, fifth section presents the max-share approach to identification, sixth section presents sign restrictions methodology, seventh section focuses on Proxy SVARS also known as external instrument identification, eighth section presents the generalized impulse response methodology, ninth section presents the exogenous block addition and finally the tenth section presents extra functions like the granger test and the nolucas.

*Banco de México, lorenzo.menna@banxico.org.mx

Contents

1	reducedformVAR — Estimate reduced-form vector autoregression	3
1.1	Syntax	3
1.2	Description	3
1.3	Options	3
1.4	Stored results	3
1.5	Remarks	4
1.6	Examples	4
1.7	Methods and formulas	5
1.8	Reference	6
2	srSVAR — Structural VAR with short-run recursive restrictions	7
2.1	Syntax	7
2.2	Description	7
2.3	Options	7
2.4	Stored results	7
2.5	Remarks	8
2.6	Examples	8
2.7	Methods and formulas	9
3	bayes_srSVAR — Bayesian SVAR with short-run restrictions	10
3.1	Syntax	10
3.2	Description	10
3.3	Options	10
3.4	Stored results	10
3.5	Remarks	11
3.6	Examples	12
3.7	Methods and formulas	13
3.8	References	14
4	nonrecursiveSRSVAR — Structural VAR with non-recursive zero restrictions	15
4.1	Syntax	15
4.2	Description	15
4.3	Options	15
4.4	Stored results	15
4.5	Remarks	16
4.6	Examples	16
4.7	Methods and formulas	18
4.8	References	18
5	lrSVAR — Structural VAR with long-run restrictions	19
5.1	Syntax	19
5.2	Description	19
5.3	Options	19
5.4	Stored results	19
5.5	Remarks	20
5.6	Examples	21
5.7	Methods and formulas	22
5.8	References	23
6	bayes_lrSVAR — Bayesian SVAR with long-run restrictions	24
6.1	Syntax	24
6.2	Description	24
6.3	Options	24
6.4	Stored results	25
6.5	Remarks	25
6.6	Examples	26

6.7	Methods and formulas	27
6.8	References	28
7	nonrecursiveLRSVAR — Non-recursive SVAR with long-run restrictions	29
7.1	Syntax	29
7.2	Description	29
7.3	Options	29
7.4	Stored results	30
7.5	Remarks	30
7.6	Examples	31
7.7	Methods and formulas	33
7.8	References	34
8	gmmSRSVAR — GMM-estimated SVAR with short-run restrictions	35
8.1	Syntax	35
8.2	Description	35
8.3	Options	35
8.4	Stored results	36
8.5	Remarks	36
8.6	Examples	37
8.7	Methods and formulas	38
8.8	References	39
9	gmmLRSVAR — GMM-estimated SVAR with long-run restrictions	40
9.1	Syntax	40
9.2	Description	40
9.3	Options	40
9.4	Stored results	41
9.5	Remarks	41
9.6	Examples	42
9.7	Methods and formulas	44
9.8	References	45
10	maxshareSVAR — Max share identification	46
10.1	Syntax	46
10.2	Description	46
10.3	Options	46
10.4	Stored results	47
10.5	Remarks	47
10.6	Examples	48
10.7	Methods and formulas	50
10.8	References	51
11	bayes_maxshareSVAR — Bayesian max share identification	52
11.1	Syntax	52
11.2	Description	52
11.3	Options	52
11.4	Stored results	53
11.5	Remarks	53
11.6	Examples	55
11.7	Methods and formulas	56
11.8	References	58

12 signrestSVAR — Sign restrictions identification	59
12.1 Syntax	59
12.2 Description	59
12.3 Options	59
12.4 Stored results	60
12.5 Remarks	60
12.6 Examples	62
12.7 Methods and formulas	63
12.8 References	65
13 bloque_exo — Block-exogenous SVAR with different sample sizes	66
13.1 Syntax	66
13.2 Description	66
13.3 Options	66
13.4 Stored results	67
13.5 Remarks	67
13.6 Examples	69
13.7 Methods and formulas	71
13.8 References	73
14 ivSVAR_blockbootstrap — Proxy-SVAR with block bootstrap	74
14.1 Syntax	74
14.2 Description	74
14.3 Options	74
14.4 Stored results	75
14.5 Remarks	75
14.6 Examples	78
14.7 Methods and formulas	80
14.8 References	82
15 ivSVAR_wildbootstrap — Proxy-SVAR with wild bootstrap	83
15.1 Syntax	83
15.2 Description	83
15.3 Options	83
15.4 Stored results	83
15.5 Remarks	83
15.6 Examples	84
15.7 Methods and formulas	85
15.8 References	86
16 girfSVAR — Generalized impulse response functions	87
16.1 Syntax	87
16.2 Description	87
16.3 Options	87
16.4 Stored results	88
16.5 Remarks	88
16.6 Examples	90
16.7 Methods and formulas	92
16.8 References	94
17 nolucas — Counterfactual historical decomposition with channel restrictions	95
17.1 Syntax	95
17.2 Description	95
17.3 Options	95
17.4 Stored results	96
17.5 Remarks	96
17.6 Examples	98
17.7 Methods and formulas	101

17.8	References	102
18	granger — Pairwise Granger causality tests	103
18.1	Syntax	103
18.2	Description	103
18.3	Options	103
18.4	Stored results	104
18.5	Remarks	104
18.6	Examples	106
18.7	Methods and formulas	108
18.8	References	109

1 reducedformVAR — Estimate reduced-form vector autoregression

1.1 Syntax

```
EQ = reducedformVAR(data, nlags)
```

```
EQ = reducedformVAR(data, nlags, const, lr, forecast, dummy, exog)
```

1.2 Description

reducedformVAR estimates a reduced-form vector autoregression (VAR) model by ordinary least squares (OLS). The function supports restricted VARs where certain variables are constrained not to affect others, seasonal dummy variables, exogenous regressors, and out-of-sample forecasting with confidence intervals.

1.3 Options

Option	Description
<code>data</code>	$N \times K$ matrix of endogenous variables (N observations, K variables). Required.
<code>nlags</code>	Number of lags to include in the VAR. Required.
<code>const</code>	Include constant term if true ; exclude if false . Default is false .
<code>lr</code>	$K \times K$ matrix of zeros and ones specifying restrictions. Element $(i, j) = 0$ restricts variable j from affecting variable i ; $= 1$ allows the effect. Default is no restrictions.
<code>forecast</code>	Compute out-of-sample forecast if true . Forecast horizon is $\lfloor N/5 \rfloor$ periods. Confidence bands computed via Monte Carlo simulation (1000 draws). Default is false .
<code>dummy</code>	Include seasonal dummies. Specify 'month' for 11 monthly dummies or 'quarter' for 3 quarterly dummies. Default is no dummies.
<code>exog</code>	$N \times E$ matrix of exogenous variables entering without lags (e.g., event dummies). Assumed zero in forecast construction. Default is no exogenous variables.

1.4 Stored results

reducedformVAR stores the following in structure `EQ`:

Scalars

<code>EQ.AIC</code>	Akaike information criterion
<code>EQ.BIC</code>	Bayesian information criterion
<code>EQ.nlags</code>	number of lags used
<code>EQ.lastperiod</code>	last period in seasonal cycle (if seasonal dummies used)

Matrices

<code>EQ.coefficients</code>	$K \times (K \cdot \text{nlags})$ estimated VAR coefficients
<code>EQ.tstats</code>	$K \times (K \cdot \text{nlags})$ t-statistics for VAR coefficients
<code>EQ.constants</code>	$K \times 1$ estimated constants (if <code>const=true</code>)
<code>EQ.tstats_const</code>	$K \times 1$ t-statistics for constants (if <code>const=true</code>)
<code>EQ.exo</code>	$K \times E$ coefficients on exogenous variables (if provided)
<code>EQ.tstats_exo</code>	$K \times E$ t-statistics for exogenous variables (if provided)
<code>EQ.dummies</code>	$K \times \text{numdum}$ seasonal dummy coefficients (if specified)
<code>EQ.tstats_dumm</code>	$K \times \text{numdum}$ t-statistics for seasonal dummies (if specified)
<code>EQ.sigma</code>	$K \times K$ variance-covariance matrix of residuals
<code>EQ.sigma_adj</code>	$K \times K$ small-sample adjusted covariance matrix of residuals
<code>EQ.Sigma</code>	variance-covariance matrix of coefficient estimates
<code>EQ.Sigma_adj</code>	small-sample adjusted covariance of coefficient estimates
<code>EQ.resid</code>	$(N - \text{nlags}) \times K$ matrix of residuals
<code>EQ.data</code>	$N \times K$ original data matrix

Forecast results (if `forecast=true`)

<code>EQ.forecast.cent</code>	$K \times (N + \lfloor N/5 \rfloor - 1)$ point forecasts
<code>EQ.forecast.conf95</code>	$K \times (N + \lfloor N/5 \rfloor - 1)$ upper 95% confidence band
<code>EQ.forecast.conf05</code>	$K \times (N + \lfloor N/5 \rfloor - 1)$ lower 5% confidence band

1.5 Remarks

The reduced-form VAR model estimated is:

$$y_t = \nu + A_1 y_{t-1} + \cdots + A_p y_{t-p} + \Upsilon d_t + \Psi x_t + u_t$$

where y_t is the $K \times 1$ vector of endogenous variables, ν is a constant term, A_s are coefficient matrices for lags $s = 1, \dots, p$, d_t are seasonal dummies, x_t are exogenous variables, and u_t are innovations with variance-covariance matrix Σ_u .

Restrictions: When the `lr` option is used, the function imposes zero restrictions on specific coefficients across all lags. For example, if `lr(2,3) = 0`, then variable 3 does not affect variable 2 in any lag.

Forecasting: Confidence bands are constructed via Monte Carlo simulation. The function draws 1000 samples from the estimated residual distribution and computes forecast paths to obtain empirical 5th and 95th percentiles.

Estimation: The model is estimated equation-by-equation using OLS. When restrictions are imposed, a selection matrix R is used to enforce zero constraints. Standard errors are computed using the residual covariance matrix.

1.6 Examples

Example 1: Basic VAR with constant

Estimate a VAR(2) model with constant term:

```
EQ = reducedformVAR(data, 2, true);
disp(EQ.coefficients); % Display VAR coefficients
disp(EQ.constants);   % Display constant terms
```

Example 2: Restricted VAR

Estimate a VAR where variable 3 does not affect variables 1 and 2:

```
lr = [1 1 0; % Variable 1 affected by variables 1,2 only
      1 1 0; % Variable 2 affected by variables 1,2 only
      1 1 1]; % Variable 3 affected by all variables
```

```
EQ = reducedformVAR(data, 2, true, lr);
disp(EQ.coefficients(:, 3:3:6)); % Coefficients on var 3
```

Example 3: VAR with forecasting

Estimate a VAR and generate out-of-sample forecasts:

```
EQ = reducedformVAR(data, 2, true, [], true);

% Plot forecast for variable 1
figure;
plot(EQ.forecast.cent(1,:), 'k-', 'LineWidth', 1.5);
hold on;
plot(EQ.forecast.conf95(1,:), 'k--');
plot(EQ.forecast.conf05(1,:), 'k--');
title('Variable 1: Forecast with 90% confidence bands');
```

Example 4: VAR with seasonal dummies

Estimate a VAR with quarterly seasonal dummies:

```
EQ = reducedformVAR(data, 4, true, [], false, 'quarter');
disp(EQ.dummies); % Seasonal dummy coefficients
disp(EQ.tstats_dumm); % T-statistics for seasonal dummies
```

Example 5: VAR with exogenous variables

Estimate a VAR with a crisis dummy variable:

```
crisis_dummy = zeros(size(data,1), 1);
crisis_dummy(80:90) = 1; % Crisis period

EQ = reducedformVAR(data, 2, true, [], false, [], crisis_dummy);
disp(EQ.exo); % Crisis dummy coefficients
disp(EQ.tstats_exo); % T-statistics
```

Example 6: Multiple features combined

Combine restrictions and forecasting:

```
lr = [1 1 0; 1 1 0; 1 1 1];
EQ = reducedformVAR(data, 2, true, lr, true);
```

For complete, executable examples with simulated data, see `example_reducedformVAR.m` in the package repository.

1.7 Methods and formulas

The VAR(p) model is estimated via OLS applied to each equation separately. For the system:

$$Y = BZ + U$$

where Y is $K \times T$ containing observations on endogenous variables, Z is $(K \cdot p + 1 + \text{numdum} + \text{numexo}) \times T$ containing lagged endogenous variables, constants, dummies, and exogenous variables, and U contains residuals, the estimator is:

$$\hat{B} = YZ'(ZZ')^{-1}$$

When restrictions are imposed via matrix R , the restricted estimator is:

$$\hat{\gamma} = [R'(ZZ' \otimes \Sigma_u^{-1})R]^{-1}R'(Z \otimes \Sigma_u^{-1})y$$

where $y = \text{vec}(Y)$ and $\hat{\beta} = R\hat{\gamma}$.

The residual covariance matrix is:

$$\hat{\Sigma}_u = \frac{1}{T}(Y - \hat{B}Z)(Y - \hat{B}Z)'$$

with small-sample adjustment $\hat{\Sigma}_{u,\text{adj}} = \frac{T}{T-K \cdot p-1} \hat{\Sigma}_u$.

Information criteria:

$$\begin{aligned} \text{AIC} &= \log |\hat{\Sigma}_u| + \frac{2K^2p}{T} \\ \text{BIC} &= \log |\hat{\Sigma}_u| + \frac{(\log T)K^2p}{T} \end{aligned}$$

1.8 Reference

Lütkepohl, H. 2005. *New Introduction to Multiple Time Series Analysis*. Berlin: Springer-Verlag.

2 srSVAR — Structural VAR with short-run recursive restrictions

2.1 Syntax

```
EQ = srSVAR(data, nlags)
```

```
EQ = srSVAR(data, nlags, var_names, irf_lenght, monte_carlo, const, ...  
            lr, dum, exog, shock, confianza)
```

2.2 Description

srSVAR estimates a structural vector autoregression (SVAR) model identified via short-run restrictions using Cholesky decomposition. The function computes impulse response functions (IRFs), forecast error variance decomposition (FEVD), and structural shocks. Confidence intervals for IRFs are obtained via Monte Carlo simulation.

The identification assumes a recursive structure where shocks are ordered such that the first variable responds contemporaneously only to its own shock, the second variable responds to the first two shocks, and so on.

2.3 Options

Option	Description
<code>data</code>	$N \times K$ matrix of endogenous variables (N observations, K variables). Required.
<code>nlags</code>	Number of lags to include in the VAR. Required.
<code>var_names</code>	$1 \times K$ cell vector with variable names. Default is <code>{'var1', 'var2', ...}</code> .
<code>irf_lenght</code>	Length of impulse response functions. Default is 40.
<code>monte_carlo</code>	Number of Monte Carlo repetitions for computing IRF confidence intervals. Default is 1000.
<code>const</code>	Include constant term if <code>true</code> ; exclude if <code>false</code> . Default is <code>false</code> .
<code>lr</code>	$K \times K$ matrix of zeros and ones specifying restrictions on the reduced-form VAR. Element $(i, j) = 0$ restricts variable j from affecting variable i ; $= 1$ allows the effect. Default is no restrictions.
<code>dum</code>	Include seasonal dummies. Specify <code>'month'</code> for 11 monthly dummies or <code>'quarter'</code> for 3 quarterly dummies. Default is no dummies.
<code>exog</code>	$N \times E$ matrix of exogenous variables entering without lags (e.g., event dummies). Treated as deterministic in confidence band construction. Default is no exogenous variables.
<code>shock</code>	$K \times 1$ vector specifying shock sizes for each variable. Default is one-standard-deviation shocks (diagonal of Cholesky matrix).
<code>confianza</code>	Lower confidence band percentage. For example, <code>confianza = 5</code> produces 90% confidence intervals (5th and 95th percentiles). Default is 5.

2.4 Stored results

srSVAR stores the following in structure `EQ`:

Matrices

<code>EQ.vari_varj</code>	$K \times K$ collection of $3 \times \text{irf_length}$ matrices. For each variable i and shock j : first row = 95% confidence band, second row = IRF, third row = 5% confidence band.
<code>EQ.P</code>	$K \times K$ Cholesky matrix (lower triangular) of the residual variance-covariance matrix. Maps structural shocks to reduced-form residuals.
<code>EQ.vardecshock_vari</code>	K matrices of size $K \times \text{irf_length}$. Each row j shows the percentage of variance of variable j explained by shock i at each horizon.
<code>EQ.stdirf</code>	$K \times \text{irf_length} \times K$ array of standard deviations of IRF coefficients across Monte Carlo simulations.
<code>EQ.struc</code>	$K \times (N - \text{nlags})$ matrix of estimated structural shocks.
<code>EQ.irf_sim</code>	$K \times \text{irf_length} \times \text{monte_carlo} \times K$ array containing all simulated IRFs from Monte Carlo draws.

2.5 Remarks

Identification: The structural VAR is identified using Cholesky decomposition of the residual variance-covariance matrix. This imposes a recursive (triangular) structure on contemporaneous relationships:

$$u_t = P\varepsilon_t, \quad \varepsilon_t \sim N(0, I_K)$$

where u_t are reduced-form residuals, P is the lower-triangular Cholesky factor, and ε_t are orthogonal structural shocks.

Ordering: The ordering of variables matters. Variable 1 is affected only by its own shock contemporaneously; variable 2 is affected by shocks 1 and 2; variable K is affected by all K shocks contemporaneously. Choose the ordering carefully based on economic theory.

Confidence intervals: The function generates confidence bands via Monte Carlo simulation. For each draw: (1) simulate a VAR with the same structure using estimated parameters and randomly drawn shocks, (2) re-estimate the model, (3) compute IRFs. The empirical distribution across draws provides confidence intervals.

Plots: The function automatically generates K figures, one for each structural shock. Each figure shows the responses of all K variables to that shock, with confidence bands.

2.6 Examples

Example 1: Basic SVAR with Cholesky identification

Estimate a 3-variable SVAR with 2 lags and constant:

```
var_names = {'Output', 'Inflation', 'InterestRate'};
EQ = srSVAR(data, 2, var_names, 40, 1000, true);
```

```
% Access IRF: Output response to Output shock
disp(EQ.Output_Output); % 3x40 matrix
```

```
% Access Cholesky matrix
disp(EQ.P);
```

```
% Access variance decomposition
disp(EQ.vardecshock_Output); % Kx40 matrix
```

Example 2: SVAR with custom shock sizes

Use 2-standard-deviation shocks instead of 1:

```
shock_size = 2 * diag(EQ.P); % 2 std deviations
EQ2 = srSVAR(data, 2, var_names, 40, 1000, true, [], [], [], ...
    shock_size);
```

Example 3: SVAR with restricted reduced-form VAR

Restrict interest rate from affecting output and inflation in the VAR:

```
lr = [1 1 0;    % Output affected by output, inflation only
      1 1 0;    % Inflation affected by output, inflation only
      1 1 1]; % Interest rate affected by all

EQ3 = srSVAR(data, 2, var_names, 40, 1000, true, lr);
```

Example 4: SVAR with different confidence levels

Use 68% confidence intervals (16th and 84th percentiles):

```
EQ4 = srSVAR(data, 2, var_names, 40, 1000, true, [], [], [], ...
              [], 16);
```

Example 5: SVAR with seasonal dummies

Include quarterly seasonal dummies:

```
EQ5 = srSVAR(data, 2, var_names, 40, 1000, true, [], 'quarter');
```

Example 6: Analyzing structural shocks

Extract and analyze estimated structural shocks:

```
% Structural shocks are orthogonal by construction
shocks = EQ.struc;
corr_matrix = corr(shocks'); % Should be approximately identity

% Plot structural shocks over time
figure;
plot(shocks(1,:));
title('Output structural shock');
```

For complete, executable examples with simulated data, see `example_srSVAR.m` in the package repository.

2.7 Methods and formulas

The structural VAR model is:

$$y_t = \nu + A_1 y_{t-1} + \dots + A_p y_{t-p} + \Upsilon d_t + \Psi x_t + P \varepsilon_t$$

where P is the Cholesky factor (lower triangular) of the residual covariance matrix $\Sigma_u = PP'$, and $\varepsilon_t \sim N(0, I_K)$ are orthogonal structural shocks.

Impulse response functions: The IRF at horizon h is computed as:

$$\text{IRF}(h) = J \Phi^h P$$

where Φ is the companion matrix of the VAR and $J = [I_K : 0 : \dots : 0]$ is a $K \times (Kp)$ selection matrix.

Forecast error variance decomposition: The contribution of shock j to the forecast error variance of variable i at horizon h is:

$$\text{FEVD}_{i,j}(h) = \frac{\sum_{s=0}^{h-1} (e_i' J \Phi^s P e_j)^2}{\sum_{k=1}^K \sum_{s=0}^{h-1} (e_i' J \Phi^s P e_k)^2} \times 100$$

where e_i is the i -th unit vector.

Structural shocks: Recovered as:

$$\hat{\varepsilon}_t = P^{-1} \hat{u}_t$$

where $\hat{u}_t$ are the reduced-form residuals.

Confidence intervals: The function simulates M (default 1000) artificial datasets from the estimated VAR, re-estimates the model for each, and computes IRFs. The $(1 - \alpha)$ confidence interval is constructed from the empirical $\alpha/2$ and $1 - \alpha/2$ quantiles of the simulated IRF distribution.

3 bayes_srSVAR — Bayesian SVAR with short-run restrictions

3.1 Syntax

```
EQ = bayes_srSVAR(data, nlags)
```

```
EQ = bayes_srSVAR(data, nlags, var_names, irf_lenght, gibbs, const, ...  
                  lr, exog, confianza, parbayes, burnin, verbose)
```

3.2 Description

bayes_srSVAR estimates a Bayesian structural vector autoregression (SVAR) model identified via short-run restrictions using Cholesky decomposition. The function uses a Minnesota prior (Independent Normal-Wishart) on VAR coefficients and an uninformative prior on the residual variance-covariance matrix. Posterior distributions are obtained via Gibbs sampling, providing full uncertainty quantification for impulse response functions and variance decompositions.

The Bayesian approach offers several advantages: (1) incorporates prior information about parameter values, (2) provides complete posterior distributions rather than point estimates, (3) handles parameter uncertainty naturally, and (4) allows for "soft" restrictions on coefficients.

3.3 Options

Option	Description
data	$N \times K$ matrix of endogenous variables (N observations, K variables). Required.
nlags	Number of lags to include in the VAR. Required.
var_names	$1 \times K$ cell vector with variable names. Default is {'var1', 'var2', ...}.
irf_lenght	Length of impulse response functions. Default is 40.
gibbs	Number of Gibbs sampler iterations (after burn-in). Default is 2000.
const	Include constant term if true ; exclude if false . Default is false .
lr	$K \times K$ matrix of zeros and ones specifying restrictions on the reduced-form VAR. Element $(i, j) = 0$ imposes a soft restriction (prior variance = 0.001) that variable j does not affect variable i ; $= 1$ allows the effect. Default is no restrictions.
exog	$N \times E$ matrix of exogenous variables entering without lags (e.g., event dummies). Default is no exogenous variables.
confianza	Lower credible interval percentage. For example, confianza = 16 produces 68% credible intervals (16th and 84th percentiles). Default is 16.
parbayes	1×6 vector of prior hyperparameters: (1) prior mean on first own-lag coefficient (default = 1), (2) prior variance on first own-lag (default = 0.2), (3) ratio of prior variance on cross-lags to own-lags before SD adjustment (default = 1), (4) lag-decay parameter (default = 2), (5) prior variance on constant (default = 10^2), (6) prior variance on exogenous variables (default = 10^2).
burnin	Number of burn-in iterations to discard before collecting posterior draws. Default is 500.
verbose	Print iteration numbers if true ; suppress if false . Default is false .

3.4 Stored results

bayes_srSVAR stores the following in structure EQ:

Posterior means

EQ.P	$K \times K$ posterior mean of Cholesky matrix (identification matrix)
EQ.coefficient_mean	$K \times (K \cdot \text{nlags})$ posterior mean of VAR coefficients
EQ.constant_mean	$K \times 1$ posterior mean of constants (if <code>const=true</code>)
EQ.exo_mean	$K \times E$ posterior mean of exogenous variable coefficients (if <code>exog</code> provided)

IRFs and variance decomposition

EQ.vari_varj	$K \times K$ collection of $3 \times \text{irf_length}$ matrices. For each variable i and shock j : first row = 84th percentile (or as specified), second row = posterior median IRF, third row = 16th percentile (or as specified).
EQ.vardecshock_vari	K matrices of size $K \times \text{irf_length}$. Each row j shows the posterior mean percentage of variance of variable j explained by shock i at each horizon.

Full posterior distributions

EQ.coeff_sim	$K \times (\text{total_coeffs}) \times \text{gibbs}$ array of all coefficient draws from posterior
EQ.irf_sim	$K \times \text{irf_length} \times K \times \text{gibbs}$ array of all IRF draws from posterior

3.5 Remarks

Minnesota prior: The function implements the standard Minnesota (Litterman) prior for VAR coefficients. The prior assumes that each variable follows a random walk (prior mean of 1 on first own lag), with tighter priors on higher-order lags and cross-lags. Specifically, the prior variance on lag ℓ coefficient for variable j in equation i is:

- Own lag: $\text{Var}(\beta_{i,j,\ell}) = \lambda_2 / \ell^{\lambda_4}$
- Cross lag: $\text{Var}(\beta_{i,j,\ell}) = (\lambda_2 \lambda_3 \sigma_i) / (\ell^{\lambda_4} \sigma_j)$

where λ_2 is overall tightness, λ_3 scales cross-lags relative to own-lags, λ_4 controls lag decay, and σ_i are residual standard deviations from univariate autoregressions.

Gibbs sampling: The posterior distribution is not available in closed form, so the function uses Gibbs sampling to draw from the joint posterior of VAR coefficients and the residual covariance matrix. The algorithm alternates between:

1. Drawing VAR coefficients conditional on the covariance matrix (multivariate normal)
2. Drawing the covariance matrix conditional on coefficients (inverse Wishart)

For each draw, the function computes IRFs and variance decompositions, building up empirical posterior distributions.

Burn-in: The first `burnin` iterations (default 500) are discarded to allow the Markov chain to converge to the stationary distribution. Convergence can be checked by examining trace plots of key parameters.

Credible vs confidence intervals: Unlike frequentist confidence intervals, Bayesian credible intervals have a direct probability interpretation: there is a $(1 - \alpha)$ probability that the true parameter lies in the $\alpha/2$ to $1 - \alpha/2$ credible interval, given the data and prior. The default 68% credible interval (16th to 84th percentiles) corresponds roughly to ± 1 standard deviation.

Soft restrictions: When `lr` is specified with zeros, the function implements "soft" restrictions by setting the prior variance on those coefficients to 0.001 (instead of zero). This maintains conjugacy of the prior while strongly shrinking restricted coefficients toward zero. Hard zero restrictions would break the conjugate structure.

Identification: Structural shocks are identified using Cholesky decomposition, exactly as in `srSVAR`. The ordering of variables determines the contemporaneous causal structure. The Bayesian approach provides posterior distributions for the Cholesky matrix elements, capturing identification uncertainty.

Plots: The function automatically generates K figures showing posterior median IRFs with credible bands for each structural shock.

3.6 Examples

Example 1: Basic Bayesian SVAR

Estimate a Bayesian SVAR with default Minnesota prior:

```
var_names = {'Output', 'Inflation', 'InterestRate'};
EQ = bayes_srSVAR(data, 2, var_names, 40, 2000, true);
```

```
% Posterior mean of Cholesky matrix
disp(EQ.P);
```

```
% Posterior median IRF with 68% credible interval
disp(EQ.Output_Output); % 3x40 matrix
```

Example 2: Custom Minnesota prior

Use a tighter prior (smaller variance on first own lag):

```
% [prior mean, prior var, cross ratio, decay, const var, exog var]
parbayes = [1, 0.1, 1, 2, 100, 100]; % Tighter: var = 0.1 vs 0.2
EQ = bayes_srSVAR(data, 2, var_names, 40, 2000, true, [], [], ...
    16, parbayes);
```

Example 3: Custom credible intervals and burn-in

Use 95% credible intervals with longer burn-in:

```
% 95% CI: confianza = 2.5, burnin = 1000
EQ = bayes_srSVAR(data, 2, var_names, 40, 2000, true, [], [], ...
    2.5, [], 1000);
```

Example 4: Verbose mode for monitoring

Print iteration numbers to monitor progress:

```
EQ = bayes_srSVAR(data, 2, var_names, 40, 2000, true, [], [], ...
    16, [], 500, true);
```

Example 5: Soft restrictions

Restrict interest rate from affecting output and inflation:

```
lr = [1 1 0; % Output: affected by output, inflation only
      1 1 0; % Inflation: affected by output, inflation only
      1 1 1]; % Interest rate: affected by all
```

```
EQ = bayes_srSVAR(data, 2, var_names, 40, 2000, true, lr);
% Coefficients on interest rate will be shrunk toward zero
```

Example 6: Analyzing posterior distributions

Extract and analyze posterior draws:

```
% Posterior of first own-lag coefficient for Output
coef_draws = squeeze(EQ.coeff_sim(1, 2, :)); % Assumes const
```

```
% Summary statistics
mean_coef = mean(coef_draws);
median_coef = median(coef_draws);
ci_95 = quantile(coef_draws, [0.025, 0.975]);
```

```
% Plot posterior distribution
histogram(coef_draws, 50);
title('Posterior: Output first own-lag coefficient');
```

```
% IRF posterior at specific horizon
irf_h5 = squeeze(EQ.irf_sim(1, 5, 1, :)); % Var 1, h=5, shock 1
histogram(irf_h5, 50);
```

For complete, executable examples with posterior analysis and comparisons, see `example_bayes_srSVAR.m` in the package repository.

3.7 Methods and formulas

The Bayesian SVAR model is:

$$y_t = \nu + A_1 y_{t-1} + \dots + A_p y_{t-p} + \Psi x_t + P \varepsilon_t$$

where P is the Cholesky factor (lower triangular) of $\Sigma_u = PP'$, and $\varepsilon_t \sim N(0, I_K)$.

Prior distributions:

Let $\beta = \text{vec}([A_1, \dots, A_p, \nu, \Psi])$ contain all VAR coefficients. The Minnesota prior is:

$$\begin{aligned} \beta &\sim N(\mu_0, \Omega_0) \\ \Sigma_u^{-1} &\sim \text{Wishart}(\nu_0, S_0^{-1}) \end{aligned}$$

where μ_0 has 1 on first own lags and 0 elsewhere, Ω_0 is diagonal with variances specified by Minnesota formulas, and $\nu_0 = 0$, $S_0 = 0$ (uninformative prior on Σ_u).

Posterior distributions:

The posterior conditionals used in Gibbs sampling are:

$$\begin{aligned} \beta \mid \Sigma_u, Y &\sim N(\mu_1, \Omega_1) \\ \Sigma_u \mid \beta, Y &\sim \text{Inverse-Wishart}(\nu_1, S_1) \end{aligned}$$

where

$$\begin{aligned} \Omega_1 &= [\Omega_0^{-1} + (ZZ')^{-1} \otimes \Sigma_u^{-1}]^{-1} \\ \mu_1 &= \Omega_1 [\Omega_0^{-1} \mu_0 + ((ZZ')^{-1} \otimes \Sigma_u^{-1}) \text{vec}(Y)] \\ \nu_1 &= T + \nu_0 \\ S_1 &= S_0 + (Y - BZ)(Y - BZ)' \end{aligned}$$

where Z contains lagged variables and deterministic terms, Y contains dependent variables, and $B = \text{reshape}(\beta, K, \cdot)$.

Impulse response functions: For each Gibbs draw $(B^{(i)}, \Sigma_u^{(i)})$:

1. Compute $P^{(i)} = \text{chol}(\Sigma_u^{(i)})'$ (lower triangular)
2. Compute IRF as $\text{IRF}^{(i)}(h) = J \Phi^h P^{(i)}$

where Φ is the companion matrix from $B^{(i)}$. The collection $\{\text{IRF}^{(i)}(h)\}_{i=1}^{\text{gibbs}}$ forms the empirical posterior distribution.

Point estimates: The function reports posterior medians for IRFs (more robust than means) and posterior means for variance decompositions.

3.8 References

Koop, G., and D. Korobilis. 2010. "Bayesian Multivariate Time Series Methods for Empirical Macroeconomics." *Foundations and Trends in Econometrics* 3(4): 267–358.

4 nonrecursiveSRSVAR — Structural VAR with non-recursive zero restrictions

4.1 Syntax

```
EQ = nonrecursiveSRSVAR(data, nlags, idmat)
```

```
EQ = nonrecursiveSRSVAR(data, nlags, idmat, var_names, irf_lenght, ...  
                        monte_carlo, const, lr, dum, exog, shock, confianza)
```

4.2 Description

nonrecursiveSRSVAR estimates a structural vector autoregression (SVAR) model identified via non-recursive short-run zero restrictions. Unlike recursive (Cholesky) identification which imposes a strict triangular ordering, this function allows arbitrary zero restriction patterns on contemporaneous effects. The identification matrix must specify exactly $K(K + 1)/2$ non-zero elements for exact identification. The function uses the algorithm of Rubio-Ramirez, Waggoner, and Zha (2010) to find the structural impact matrix satisfying the imposed restrictions.

Non-recursive identification is particularly useful when economic theory suggests contemporaneous relationships that do not fit a recursive structure.

4.3 Options

Option	Description
<code>data</code>	$N \times K$ matrix of endogenous variables (N observations, K variables). Required.
<code>nlags</code>	Number of lags to include in the VAR. Required.
<code>idmat</code>	$K \times K$ identification matrix of zeros and ones. Element $(i, j) = 0$ means structural shock j does <i>not</i> affect variable i contemporaneously; $= 1$ means it does. Must have exactly $K(K + 1)/2$ non-zero elements for exact identification. Required.
<code>var_names</code>	$1 \times K$ cell vector with variable names. Default is <code>{'var1', 'var2', ...}</code> .
<code>irf_lenght</code>	Length of impulse response functions. Default is 40.
<code>monte_carlo</code>	Number of Monte Carlo repetitions for computing IRF confidence intervals. Default is 1000.
<code>const</code>	Include constant term if <code>true</code> ; exclude if <code>false</code> . Default is <code>false</code> .
<code>lr</code>	$K \times K$ matrix of zeros and ones specifying restrictions on the reduced-form VAR. Element $(i, j) = 0$ restricts variable j from affecting variable i ; $= 1$ allows the effect. Default is no restrictions.
<code>dum</code>	Include seasonal dummies. Specify <code>'month'</code> for 11 monthly dummies or <code>'quarter'</code> for 3 quarterly dummies. Default is no dummies.
<code>exog</code>	$N \times E$ matrix of exogenous variables entering without lags (e.g., event dummies). Treated as deterministic in confidence band construction. Default is no exogenous variables.
<code>shock</code>	$K \times 1$ vector specifying shock sizes for each variable. Default is one-standard-deviation shocks (diagonal of impact matrix).
<code>confianza</code>	Lower confidence band percentage. For example, <code>confianza = 5</code> produces 90% confidence intervals (5th and 95th percentiles). Default is 5.

4.4 Stored results

nonrecursiveSRSVAR stores the following in structure `EQ`:

Matrices

EQ.vari_varj	$K \times K$ collection of $3 \times \text{irf_length}$ matrices. For each variable i and shock j : first row = 95% confidence band, second row = IRF, third row = 5% confidence band.
EQ.P	$K \times K$ structural impact matrix satisfying the zero restrictions specified in <code>idmat</code> . Maps structural shocks to reduced-form residuals: $u_t = P\varepsilon_t$.
EQ.vardecshock_vari	K matrices of size $K \times \text{irf_length}$. Each row j shows the percentage of variance of variable j explained by shock i at each horizon.
EQ.stdirf	$K \times \text{irf_length} \times K$ array of standard deviations of IRF coefficients across Monte Carlo simulations.
EQ.struc	$K \times (N - \text{nlags})$ matrix of estimated structural shocks.

4.5 Remarks

Non-recursive vs recursive identification: Recursive (Cholesky) identification is a special case of non-recursive identification where `idmat` is lower triangular. Non-recursive identification allows more flexible patterns based on economic theory.

Exact identification: The model is exactly identified when the number of free parameters in P equals the number of unique elements in the residual covariance matrix Σ_u . Since Σ_u is symmetric, it has $K(K+1)/2$ unique elements. Therefore, `idmat` must have exactly $K(K+1)/2$ non-zero elements. The function checks this condition and returns an error if not satisfied.

Rubio-Ramirez, Waggoner, and Zha algorithm: The function implements the QR decomposition algorithm from Rubio-Ramirez, Waggoner, and Zha (2010). The algorithm:

1. Computes the square root of the residual covariance matrix: $L_0 = \Sigma_u^{1/2}$
2. Sorts columns by number of zero restrictions (most constrained first)
3. For each column, uses QR decomposition to find an orthonormal vector satisfying the zero restrictions
4. Constructs $P = L_0 Q$ where Q is the matrix of orthonormal vectors
5. Ensures positive diagonal elements by sign normalization

Confidence intervals: Generated via Monte Carlo simulation identical to `srSVAR`. For each draw, the non-recursive identification is re-applied to obtain structural IRFs consistent with the zero restrictions.

Structural shocks: Recovered as $\hat{\varepsilon}_t = P^{-1}\hat{u}_t$ where $\hat{u}_t$ are reduced-form residuals. The shocks should be approximately orthogonal by construction.

Sign normalization: The algorithm normalizes signs so that diagonal elements of P are positive. This ensures consistent interpretation of shocks across different datasets.

Plots: The function automatically generates K figures, one for each structural shock, showing the responses of all K variables with confidence bands.

4.6 Examples

Example 1: Basic non-recursive SVAR

Estimate a 3-variable SVAR with non-recursive pattern:

```
var_names = {'Output', 'Inflation', 'InterestRate'};

% Identification: shock 2 (monetary policy) affects all variables
% but shock 1 does not affect interest rate, shock 3 only affects
% interest rate
idmat = [1  1  0;    % Output: shocks 1 and 2
```

```

    1  1  0;    % Inflation: shocks 1 and 2
    0  1  1];  % Interest rate: shocks 2 and 3

% Check: sum(idmat(:)) = 6 = K(K+1)/2, exactly identified

EQ = nonrecursiveSRSVAR(data, 2, idmat, var_names, 40, 1000, true);

% Access impact matrix
disp(EQ.P); % Satisfies zero restrictions in idmat

```

Example 2: Different identification pattern

Alternative pattern with shock 3 affecting output directly:

```

% Shock 3 (financial) affects output and interest rate
idmat2 = [1  0  1;    % Output: shocks 1 and 3
          1  1  0;    % Inflation: shocks 1 and 2
          1  0  1];  % Interest rate: shocks 1 and 3

EQ2 = nonrecursiveSRSVAR(data, 2, idmat2, var_names, 40, 1000, true);

```

Example 3: Comparison with recursive identification

Recursive (Cholesky) is special case with lower triangular idmat:

```

% Recursive identification
idmat_rec = [1  0  0;
             1  1  0;
             1  1  1];

EQ_nonrec = nonrecursiveSRSVAR(data, 2, idmat_rec, var_names, ...
                               40, 1000, true);
EQ_chol = srSVAR(data, 2, var_names, 40, 1000, true);

% Impact matrices should be identical (up to numerical precision)
disp(abs(EQ_nonrec.P - EQ_chol.P)); % Should be near zero

```

Example 4: With custom shock sizes

Use 2-standard-deviation shocks:

```

shock_size = 2 * diag(EQ.P);
EQ3 = nonrecursiveSRSVAR(data, 2, idmat, var_names, 40, 1000, ...
                        true, [], [], [], shock_size);

```

Example 5: With restricted reduced-form VAR

Combine non-recursive identification with lag restrictions:

```

% Interest rate does not affect output/inflation in lags
lr = [1  1  0;
      1  1  0;
      1  1  1];

EQ4 = nonrecursiveSRSVAR(data, 2, idmat, var_names, 40, 1000, ...
                        true, lr);

```

Example 6: Checking structural shocks

Verify shocks are orthogonal:

```

shocks = EQ.struc;
corr_matrix = corr(shocks'); % Should be approximately identity

% Summary statistics
fprintf('Mean: [%f, %f, %f]\n', mean(shocks, 2));
fprintf('Std: [%f, %f, %f]\n', std(shocks, 0, 2));

```

For complete, executable examples with multiple identification schemes, see `example_nonrecursiveSRSVAR.m` in the package repository.

4.7 Methods and formulas

The structural VAR model is:

$$y_t = \nu + A_1 y_{t-1} + \cdots + A_p y_{t-p} + \Upsilon d_t + \Psi x_t + P \varepsilon_t$$

where P is the structural impact matrix satisfying zero restrictions, and $\varepsilon_t \sim N(0, I_K)$ are orthogonal structural shocks.

Zero restrictions: Let R be the set of pairs (i, j) such that shock j does not affect variable i contemporaneously. The identification requires:

$$P_{ij} = 0 \quad \text{for all } (i, j) \in R$$

and $|R| = K^2 - K(K+1)/2$ for exact identification.

Rubio-Ramirez, Waggoner, Zha (2010) algorithm:

Step 1: Compute $L_0 = \Sigma_u^{1/2}$ (square root of residual covariance matrix)

Step 2: For column $j = 1, \dots, K$:

- Let Z_j be a matrix of dimension $(n_j \times K)$ where n_j is the number of zero restrictions on shock j , with rows equal to unit vectors e_i for all i such that $(i, j) \in R$
- Compute $\tilde{Q}_j = [Z_j L_0; Q_{1:j-1}^T]^T$ where $Q_{1:j-1}$ contains previously computed columns
- Apply QR decomposition: $\tilde{Q}_j^T = QR$
- Set q_j (column j of Q) equal to last column of Q from QR decomposition

Step 3: Construct $P = L_0 Q$

Step 4: Normalize signs so $P_{jj} > 0$ for all j

Properties of P :

- $PP' = \Sigma_u$ (reproduces residual covariance matrix)
- $P_{ij} = 0$ for all $(i, j) \in R$ (satisfies zero restrictions)
- Columns of P are orthogonal (structural shocks are uncorrelated)

Impulse response functions: Computed as:

$$\text{IRF}(h) = J \Phi^h P$$

where Φ is the companion matrix and $J = [I_K : 0 : \cdots : 0]$ is the selection matrix.

Forecast error variance decomposition: The contribution of shock j to the forecast error variance of variable i at horizon h is:

$$\text{FEVD}_{i,j}(h) = \frac{\sum_{s=0}^{h-1} (e_i' J \Phi^s P e_j)^2}{\sum_{k=1}^K \sum_{s=0}^{h-1} (e_i' J \Phi^s P e_k)^2} \times 100$$

4.8 References

Lütkepohl, H. 2005. *New Introduction to Multiple Time Series Analysis*. Berlin: Springer-Verlag.

Rubio-Ramirez, J. F., D. F. Waggoner, and T. Zha. 2010. "Structural Vector Autoregressions: Theory of Identification and Algorithms for Inference." *Review of Economic Studies* 77(2): 665–696.

5 lrSVAR — Structural VAR with long-run restrictions

5.1 Syntax

```
EQ = lrSVAR(data, nlags, var_names, shock_names)
```

```
EQ = lrSVAR(data, nlags, var_names, shock_names, irf_lenght, monte_carlo, ...
            const, lr, dum, exog, shock, confianza)
```

5.2 Description

lrSVAR estimates a structural vector autoregression (SVAR) model identified via long-run restrictions à la Blanchard and Quah (1989). The identification imposes that certain structural shocks have no long-run effects on certain variables. Specifically, the long-run multiplier matrix $\Theta(1) = [I - A_1 - \dots - A_p]^{-1}P$ is constrained to be lower triangular, where P is the structural impact matrix.

This identification is particularly useful when economic theory suggests that some shocks (e.g., demand shocks) should have only temporary effects on certain variables (e.g., output), while other shocks (e.g., supply shocks) can have permanent effects. The function allows separate naming of variables and structural shocks, enabling clear economic interpretation.

5.3 Options

Option	Description
<code>data</code>	$N \times K$ matrix of endogenous variables (N observations, K variables). Required.
<code>nlags</code>	Number of lags to include in the VAR. Required.
<code>var_names</code>	$1 \times K$ cell vector with variable names. Default is <code>{'var1', 'var2', ...}</code> .
<code>shock_names</code>	$1 \times K$ cell vector with structural shock names. Default is <code>{'var1', 'var2', ...}</code> . This allows economic interpretation of shocks (e.g., <code>{'Supply', 'Demand', 'Labor'}</code>) separate from variable names.
<code>irf_lenght</code>	Length of impulse response functions. Default is 40.
<code>monte_carlo</code>	Number of Monte Carlo repetitions for computing IRF confidence intervals. Default is 1000.
<code>const</code>	Include constant term if <code>true</code> ; exclude if <code>false</code> . Default is <code>false</code> .
<code>lr</code>	$K \times K$ matrix of zeros and ones specifying restrictions on the reduced-form VAR. Element $(i, j) = 0$ restricts variable j from affecting variable i ; $= 1$ allows the effect. Default is no restrictions.
<code>dum</code>	Include seasonal dummies. Specify <code>'month'</code> for 11 monthly dummies or <code>'quarter'</code> for 3 quarterly dummies. Default is no dummies.
<code>exog</code>	$N \times E$ matrix of exogenous variables entering without lags (e.g., event dummies). Treated as deterministic in confidence band construction. Default is no exogenous variables.
<code>shock</code>	$K \times 1$ vector specifying shock sizes for each variable. Default is one-standard-deviation shocks (diagonal of impact matrix).
<code>confianza</code>	Lower confidence band percentage. For example, <code>confianza = 5</code> produces 90% confidence intervals (5th and 95th percentiles). Default is 5.

5.4 Stored results

lrSVAR stores the following in structure `EQ`:

Matrices

<code>EQ.vari_shockj</code>	$K \times K$ collection of $3 \times \text{irf_length}$ matrices. For each variable i and shock j : first row = 95% confidence band, second row = IRF, third row = 5% confidence band. Field names use <code>var_names</code> for variables and <code>shock_names</code> for shocks (e.g., <code>EQ.Output_Supply</code>).
<code>EQ.P</code>	$K \times K$ structural impact matrix. Satisfies $PP' = \Sigma_u$ and makes the long-run multiplier matrix $\Theta(1) = [I - \sum A_i]^{-1}P$ lower triangular.
<code>EQ.vardecshock_shockj</code>	K matrices of size $K \times \text{irf_length}$. Each row shows the percentage of variance of that variable explained by shock j at each horizon. Field names use <code>shock_names</code> (e.g., <code>EQ.vardecshock_Supply</code>).
<code>EQ.stdirf</code>	$K \times \text{irf_length} \times K$ array of standard deviations of IRF coefficients across Monte Carlo simulations.
<code>EQ.struc</code>	$K \times (N - \text{nlags})$ matrix of estimated structural shocks.

5.5 Remarks

Long-run restrictions: The Blanchard-Quah identification imposes that the long-run multiplier matrix $\Theta(1)$ is lower triangular. This means:

- Shock 2 has no long-run effect on variable 1
- Shock 3 has no long-run effect on variables 1 and 2
- And so on for systems with $K > 3$

These restrictions are motivated by economic theory, such as the idea that demand shocks have only temporary effects on output while supply shocks can have permanent effects.

Identification formula: Let $B = I - A_1 - \dots - A_p$ be the sum of lag coefficients subtracted from the identity matrix. The long-run multiplier is $\Theta(1) = B^{-1}P$. The function computes:

$$\Omega = B^{-1}\Sigma_u(B^{-1})' = B^{-1}(PP')(B^{-1})'$$

Then obtains P via:

$$P = B \cdot \text{chol}(\Omega, \text{'lower'})$$

This ensures $\Theta(1) = B^{-1}P$ is lower triangular (Cholesky decomposition of Ω).

Variable and shock names: The function accepts separate names for variables and shocks. This is essential for long-run identification where shocks have economic interpretations (supply, demand, monetary policy, etc.) that may differ from variable names. For example:

```
var_names = {'Output', 'Inflation', 'Unemployment'};
shock_names = {'Supply', 'Demand', 'Labor'};
```

Results are then accessed as `EQ.Output_Supply`, `EQ.Inflation_Demand`, etc., making economic interpretation clear.

Ordering: The ordering of variables and shocks matters. The first shock (typically a supply or technology shock) is the only shock that can have long-run effects on all variables. The last shock can only have long-run effects on the last variable. Choose ordering based on economic theory.

Stationarity requirement: Long-run restrictions require that the VAR is stationary in levels. If variables are $I(1)$, consider estimating in first differences.

Confidence intervals: Generated via Monte Carlo simulation. For each draw, the long-run identification is re-applied to obtain structural IRFs consistent with the restrictions.

Plots: The function automatically generates K figures, one for each structural shock, showing the responses of all K variables with confidence bands. Plot titles use both variable and shock names.

5.6 Examples

Example 1: Basic long-run SVAR (Blanchard-Quah)

Estimate a 3-variable SVAR with long-run restrictions:

```
var_names = {'Output', 'Inflation', 'Unemployment'};
shock_names = {'Supply', 'Demand', 'Labor'};

EQ = lrSVAR(data, 2, var_names, shock_names, 40, 1000, true);

% Access IRF: Output response to Supply shock
disp(EQ.Output_Supply); % 3x40 matrix

% Access variance decomposition: contribution of Demand shock
disp(EQ.vardecshock_Demand); % Kx40 matrix

% Access impact matrix
disp(EQ.P);
```

Example 2: Verifying long-run restrictions

Check that long-run multiplier is lower triangular:

```
% Get reduced-form VAR
Q = reducedformVAR(data, 2, true);

% Compute sum of lag coefficients
A_sum = Q.coefficients(:,1:3) + Q.coefficients(:,4:6);
B = eye(3) - A_sum;

% Compute long-run multiplier
LR_multiplier = inv(B) * EQ.P;

% Upper triangle should be near zero
disp(LR_multiplier);
```

Example 3: Interpreting long-run effects

Cumulative IRFs approximate long-run effects:

```
% Sum of IRF over 40 periods
lr_effect_supply = sum(EQ.Output_Supply(2,:));
lr_effect_demand = sum(EQ.Output_Demand(2,:));

fprintf('Supply shock LR effect on Output: %.4f\n', lr_effect_supply);
fprintf('Demand shock LR effect on Output: %.4f (should be ~0)\n', ...
        lr_effect_demand);
```

Example 4: Custom shock sizes

Use 2-standard-deviation shocks:

```
shock_size = 2 * diag(EQ.P);
EQ2 = lrSVAR(data, 2, var_names, shock_names, 40, 1000, true, ...
             [], [], [], shock_size);
```

Example 5: With restricted reduced-form VAR

Combine long-run identification with lag restrictions:


```
% Unemployment does not affect output/inflation in lags
lr = [1 1 0;
      1 1 0;
      1 1 1];

EQ3 = lrSVAR(data, 2, var_names, shock_names, 40, 1000, true, lr);
```

Example 6: Comparison with short-run identification

Compare long-run vs short-run (Cholesky) identification:

```
EQ_lr = lrSVAR(data, 2, var_names, shock_names, 40, 1000, true);
EQ_sr = srSVAR(data, 2, var_names, 40, 1000, true);

% Different impact matrices
disp('Long-run P:'); disp(EQ_lr.P);
disp('Short-run P:'); disp(EQ_sr.P);

% Different IRFs
figure;
subplot(1,2,1);
plot(EQ_lr.Output_Supply(2,:));
title('Long-run ID: Output response to Supply shock');

subplot(1,2,2);
plot(EQ_sr.Output_Output(2,:));
title('Short-run ID: Output response to first shock');
```

For complete, executable examples with economic interpretation and visualizations, see `example_lrSVAR.m` in the package repository.

5.7 Methods and formulas

The structural VAR model is:

$$y_t = \nu + A_1 y_{t-1} + \dots + A_p y_{t-p} + \Upsilon d_t + \Psi x_t + P \varepsilon_t$$

where P is the structural impact matrix and $\varepsilon_t \sim N(0, I_K)$ are orthogonal structural shocks.

Long-run multiplier: The long-run effect of a one-unit structural shock ε on y is:

$$\Theta(1) = [I - A_1 - \dots - A_p]^{-1} P$$

Blanchard-Quah identification: The restriction is $\Theta(1)$ lower triangular. To achieve this:

- Step 1: Estimate reduced-form VAR, obtain Σ_u and A_i
- Step 2: Compute $B = I - \sum_{i=1}^p A_i$
- Step 3: Compute $\Omega = B^{-1} \Sigma_u (B^{-1})'$
- Step 4: Compute Cholesky decomposition $\Omega = LL'$ (lower triangular)
- Step 5: Set $P = BL$

Verification:

$$\begin{aligned} \Theta(1) &= B^{-1} P = B^{-1} (BL) = L \quad (\text{lower triangular}) \\ PP' &= (BL)(BL)' = B\Omega B' = \Sigma_u \quad (\text{correct covariance}) \end{aligned}$$

Impulse response functions: For each shock, compute:

$$\text{IRF}(h) = J\Phi^h P$$

where Φ is the companion matrix and $J = [I_K : 0 : \cdots : 0]$ is the selection matrix.

Long-run effects: The long-run effect of shock j on variable i is:

$$\Theta_{ij}(1) = e_i' B^{-1} P e_j$$

where e_i is the i -th unit vector. By construction, $\Theta_{ij}(1) = 0$ for $i < j$.

Forecast error variance decomposition: The contribution of shock j to the forecast error variance of variable i at horizon h is:

$$\text{FEVD}_{i,j}(h) = \frac{\sum_{s=0}^{h-1} (e_i' J \Phi^s P e_j)^2}{\sum_{k=1}^K \sum_{s=0}^{h-1} (e_i' J \Phi^s P e_k)^2} \times 100$$

As $h \rightarrow \infty$, FEVD reflects the long-run restrictions.

5.8 References

Blanchard, O. J., and D. Quah. 1989. "The Dynamic Effects of Aggregate Demand and Supply Disturbances." *American Economic Review* 79(4): 655–673.

Lütkepohl, H. 2005. *New Introduction to Multiple Time Series Analysis*. Berlin: Springer-Verlag.

6 bayes_lrSVAR — Bayesian SVAR with long-run restrictions

6.1 Syntax

```
EQ = bayes_lrSVAR(data, nlags, var_names, shock_names)
```

```
EQ = bayes_lrSVAR(data, nlags, var_names, shock_names, irf_lenght, gibbs, ...  
                  const, lr, exog, confianza, parbayes, burnin, verbose)
```

6.2 Description

bayes_lrSVAR estimates a Bayesian structural vector autoregression (SVAR) model identified via long-run restrictions à la Blanchard and Quah (1989), using a Minnesota (Litterman) prior on VAR coefficients and an uninformative prior on the error covariance matrix. The identification imposes that the long-run multiplier matrix $\Theta(1) = [I - A_1 - \dots - A_p]^{-1}P$ is lower triangular, constraining certain structural shocks to have no long-run effects on certain variables.

The function uses a Gibbs sampler to draw from the posterior distribution, applying the long-run identification at each iteration. This yields full posterior distributions for impulse response functions, variance decompositions, and all VAR parameters. The Minnesota prior provides shrinkage toward a random walk with drift, which is particularly useful for macroeconomic data. The function allows separate naming of variables and structural shocks for clear economic interpretation.

6.3 Options

Option	Description
<code>data</code>	$N \times K$ matrix of endogenous variables (N observations, K variables). Required.
<code>nlags</code>	Number of lags to include in the VAR. Required.
<code>var_names</code>	$1 \times K$ cell vector with variable names. Default is <code>{'var1', 'var2', ...}</code> .
<code>shock_names</code>	$1 \times K$ cell vector with structural shock names. Default is <code>{'var1', 'var2', ...}</code> . Allows economic interpretation separate from variable names (e.g., <code>{'Supply', 'Demand', 'Labor'}</code>).
<code>irf_lenght</code>	Length of impulse response functions. Default is 40.
<code>gibbs</code>	Number of Gibbs sampler iterations (after burn-in). Default is 2000.
<code>const</code>	Include constant term if <code>true</code> ; exclude if <code>false</code> . Default is <code>false</code> .
<code>lr</code>	$K \times K$ matrix of zeros and ones specifying soft restrictions on the reduced-form VAR. Element $(i, j) = 0$ imposes a soft restriction (prior variance = 0.001) that variable j does not affect variable i ; $= 1$ allows the effect with normal prior variance. Default is no restrictions.
<code>exog</code>	$N \times E$ matrix of exogenous variables entering without lags (e.g., event dummies). Default is no exogenous variables.
<code>confianza</code>	Lower credible interval percentage. For example, <code>confianza = 16</code> produces 68% credible intervals (16th and 84th percentiles). Default is 16.
<code>parbayes</code>	1×6 vector of Minnesota prior hyperparameters: [prior mean on first own lag (default 1), prior variance on first own lag (default 0.2), ratio of cross-lag to own-lag prior variance (default 1), lag decay parameter (default 2), prior variance on constant (default 100), prior variance on exogenous variables (default 100)].
<code>burnin</code>	Number of initial Gibbs draws to discard. Default is 500.
<code>verbose</code>	Print iteration numbers if <code>true</code> ; suppress if <code>false</code> . Default is <code>false</code> .

6.4 Stored results

`bayes_lrSVAR` stores the following in structure `EQ`:

Matrices

<code>EQ.vari_shockj</code>	$K \times K$ collection of $3 \times \text{irf_length}$ matrices. For each variable i and shock j : first row = 84th percentile, second row = median, third row = 16th percentile. Field names use <code>var_names</code> for variables and <code>shock_names</code> for shocks.
<code>EQ.P</code>	$K \times K$ posterior mean of the structural impact matrix. Satisfies long-run identification on average across draws.
<code>EQ.vardecshock_shockj</code>	K matrices of size $K \times \text{irf_length}$. Each row shows the posterior mean percentage of variance of that variable explained by shock j at each horizon.
<code>EQ.constant_mean</code>	$K \times 1$ posterior mean of the constant (if <code>const=true</code>).
<code>EQ.exo_mean</code>	$K \times E$ posterior mean of exogenous variable coefficients (if <code>exog</code> provided).
<code>EQ.coefficient_mean</code>	$K \times (K \cdot \text{nlags})$ posterior mean of VAR lag coefficients.
<code>EQ.coeff_sim</code>	Full coefficient draws array for posterior analysis.
<code>EQ.irf_sim</code>	$K \times \text{irf_length} \times K \times \text{gibbs}$ array of all IRF draws from posterior distribution.

6.5 Remarks

Bayesian long-run identification: The function applies Blanchard-Quah identification at each Gibbs iteration. For each draw of VAR coefficients and error covariance matrix, the long-run multiplier $\Theta(1)$ is constrained to be lower triangular. This ensures the restriction holds exactly within each posterior draw, providing valid uncertainty quantification for long-run effects.

Minnesota (Litterman) prior: The prior distribution is:

- Own first lags: $N(\text{alf1}, \text{alf2})$ where $\text{alf1}=1$ (random walk) and $\text{alf2}=0.2$
- Own higher lags: Variance decays as $\text{alf2}/j^{\text{decay}}$ where j is lag order
- Cross-variable lags: Variance adjusted by ratio of residual standard deviations
- Constant and exogenous: Large variance (uninformative)
- Error covariance: Uninformative (Wishart with $\text{df}=0$)

Credible vs confidence intervals: Bayesian credible intervals have a direct probability interpretation: "There is a 68% posterior probability the true IRF lies in this interval." This differs from frequentist confidence intervals which have a sampling interpretation. Credible intervals incorporate prior information and provide full posterior distributions.

Gibbs sampler: The algorithm alternates between:

1. Drawing VAR coefficients from $\beta|\Sigma, Y \sim N(\cdot, \cdot)$ (multivariate normal)
2. Drawing error covariance from $\Sigma|\beta, Y \sim IW(\cdot, \cdot)$ (inverse Wishart)
3. Applying long-run identification to obtain structural impact matrix
4. Computing IRFs and variance decompositions for this draw

Soft restrictions: When `lr` matrix contains zeros, the function implements soft restrictions by setting prior variance to 0.001 (very small but not exactly zero). This maintains conjugacy of the prior while strongly discouraging these effects. Hard zero restrictions would require more complex posterior sampling.

Burn-in: Initial Gibbs draws may not represent the stationary posterior distribution. The burn-in period discards these early draws. Monitor trace plots to verify convergence. Increase burn-in if traces show trending behavior or haven't stabilized.

Variable and shock names: Essential for long-run identification where shocks have economic interpretations (supply, demand, policy) distinct from variable names. Results accessed as `EQ.Output_Supply`, `EQ.Inflation_Demand`, etc.

Posterior analysis: Use `EQ.irf_sim` to analyze full IRF distributions. Construct fan charts, compute custom quantiles, test hypotheses, or examine tail behavior beyond standard credible intervals.

6.6 Examples

Example 1: Basic Bayesian long-run SVAR

Estimate with default Minnesota prior:

```
var_names = {'Output', 'Inflation', 'Unemployment'};
shock_names = {'Supply', 'Demand', 'Labor'};

EQ = bayes_lrSVAR(data, 2, var_names, shock_names, 40, 2000, true);

% Access posterior median IRF
disp(EQ.Output_Supply(2,:)); % Median response

% Access posterior mean coefficients
disp(EQ.coefficient_mean);
```

Example 2: Verifying long-run restrictions

Check restrictions hold in posterior draws (via cumulative IRFs):

```
% For a random draw, cumulative IRF approximates long-run effect
g = 1000; % Some Gibbs iteration

% Demand shock -> Output (should be near zero)
cum_demand_output = sum(squeeze(EQ.irf_sim(1, :, 2, g)));

% Labor shock -> Inflation (should be near zero)
cum_labor_inflation = sum(squeeze(EQ.irf_sim(2, :, 3, g)));

fprintf('Cumulative effects: %.6f, %.6f\n', cum_demand_output, ...
        cum_labor_inflation);
% Values will be very small (e.g., 0.001), confirming restriction
```

Example 3: Custom Minnesota prior (tighter)

Reduce prior variance for more shrinkage:

```
% [prior mean, prior var, cross ratio, decay, const var, exog var]
parbayes = [1, 0.1, 1, 2, 100, 100]; % Tighter: 0.1 vs default 0.2

EQ2 = bayes_lrSVAR(data, 2, var_names, shock_names, 40, 2000, true, ...
                   [], [], 16, parbayes);

% Tighter prior pulls estimates closer to 1.0 for own first lags
```

Example 4: Different credible interval levels

Use 95% credible intervals:

```
% confianza = 2.5 gives 95% CI (2.5th to 97.5th percentile)
EQ3 = bayes_lrSVAR(data, 2, var_names, shock_names, 40, 2000, true, ...
    [], [], 2.5);
```

Example 5: Posterior distribution analysis

Analyze full posterior of IRF at specific horizon:

```
% Extract Output response to Supply shock at horizon 5
irf_h5 = squeeze(EQ.irf_sim(1, 5, 1, :));

% Posterior statistics
fprintf('Mean: %.4f, Median: %.4f, Std: %.4f\n', ...
    mean(irf_h5), median(irf_h5), std(irf_h5));

% Custom quantiles
q = quantile(irf_h5, [0.05, 0.16, 0.50, 0.84, 0.95]);

% Plot histogram
histogram(irf_h5, 50);
```

Example 6: With soft restrictions

Restrict unemployment from affecting other variables in lags:

```
lr = [1 1 0;
      1 1 0;
      1 1 1];

EQ4 = bayes_lrSVAR(data, 2, var_names, shock_names, 40, 2000, true, lr);

% Coefficients on unemployment lags will be near zero
disp(EQ4.coefficient_mean(:, [3, 6]));
```

For complete, executable examples with posterior analysis and visualizations, see `example_bayes_lrSVAR.m` in the package repository.

6.7 Methods and formulas

The structural VAR model is:

$$y_t = \nu + A_1 y_{t-1} + \dots + A_p y_{t-p} + \Upsilon d_t + \Psi x_t + P \varepsilon_t$$

where P is the structural impact matrix and $\varepsilon_t \sim N(0, I_K)$ are orthogonal structural shocks.

Minnesota prior on coefficients:

Let β denote all VAR coefficients (excluding constants and exogenous). The prior is $\beta \sim N(\mu_0, V_0)$ where: Diagonal elements of V_0 for coefficient (i, j) at lag ℓ :

$$V_0(i, j, \ell) = \begin{cases} \text{alf2}/\ell^{\text{decay}} & \text{if } i = j \text{ (own lag)} \\ \text{alf2} \cdot \text{alf3} \cdot \frac{\sigma_i}{\sigma_j \cdot \ell^{\text{decay}}} & \text{if } i \neq j \text{ (cross lag)} \end{cases}$$

where σ_i are residual standard deviations from univariate $\text{AR}(p)$ regressions. Prior mean μ_0 : `alf1` (default 1) for own first lags, 0 elsewhere.

Prior on error covariance:

$$\Sigma^{-1} \sim W(\Psi_0, \nu_0)$$

with $\nu_0 = 0$ (uninformative) and $\Psi_0 = 0$ (improper prior that becomes proper after conditioning on data).

Gibbs sampler algorithm:

For $g = 1, \dots, G + B$ (where B is burn-in):

Step 1: Draw coefficients

$$\beta^{(g)} | \Sigma^{(g-1)}, Y \sim N(\mu_g, V_g)$$

where:

$$V_g = [V_0^{-1} + (Z'Z \otimes \Sigma^{-1})]^{-1}$$

$$\mu_g = V_g[V_0^{-1}\mu_0 + (Z'Z \otimes \Sigma^{-1})\text{vec}(\hat{\beta}_{OLS})]$$

Step 2: Draw error covariance

$$\Sigma^{(g)} | \beta^{(g)}, Y \sim IW(\Psi_g, \nu_g)$$

where $\nu_g = T$ and $\Psi_g = (Y - \beta^{(g)}Z)(Y - \beta^{(g)}Z)'$.

Step 3: Apply long-run identification

$$\text{somma} = A_1^{(g)} + \dots + A_p^{(g)}$$

$$B^{(g)} = I - \text{somma}$$

$$\Omega^{(g)} = [B^{(g)}]^{-1} \Sigma^{(g)} [B^{(g)}]^{-1'}$$

$$P^{(g)} = B^{(g)} \cdot \text{chol}(\Omega^{(g)}, \text{'lower'})$$

This ensures $\Theta(1)^{(g)} = [B^{(g)}]^{-1} P^{(g)} = \text{chol}(\Omega^{(g)})$ is lower triangular.

Step 4: Compute IRFs for draw g :

$$\text{IRF}^{(g)}(h) = J\Phi^{h(g)}P^{(g)}$$

Posterior summaries: After discarding burn-in draws:

- Point estimates: Posterior median (more robust than mean)
- Credible intervals: Quantiles of posterior draws
- Variance decomposition: Posterior mean

Soft restrictions: When $lr(i, j) = 0$, set $V_0(i, j, \cdot) = 0.001$ instead of the Minnesota formula. This creates a very tight prior around zero while maintaining conjugacy.

6.8 References

Blanchard, O. J., and D. Quah. 1989. "The Dynamic Effects of Aggregate Demand and Supply Disturbances." *American Economic Review* 79(4): 655–673.

Koop, G., and D. Korobilis. 2010. "Bayesian Multivariate Time Series Methods for Empirical Macroeconomics." *Foundations and Trends in Econometrics* 3(4): 267–358.

Lütkepohl, H. 2005. *New Introduction to Multiple Time Series Analysis*. Berlin: Springer-Verlag.

Litterman, R. B. 1986. "Forecasting with Bayesian Vector Autoregressions—Five Years of Experience." *Journal of Business & Economic Statistics* 4(1): 25–38.

7 nonrecursiveLRSVAR — Non-recursive SVAR with long-run restrictions

7.1 Syntax

```
EQ = nonrecursiveLRSVAR(data, nlags, idmat, var_names, shock_names)
```

```
EQ = nonrecursiveLRSVAR(data, nlags, idmat, var_names, shock_names, ...
                        irf_lenght, monte_carlo, const, lr, dum, exog, ...
                        shock, confianza)
```

7.2 Description

nonrecursiveLRSVAR estimates a structural vector autoregression (SVAR) model identified via long-run zero restrictions that do not require a recursive (triangular) ordering. Unlike the Blanchard-Quah decomposition (**lrSVAR**) which imposes a lower triangular structure on the long-run multiplier matrix, this function uses the Rubio-Ramirez, Waggoner, and Zha (2010) algorithm to accommodate non-recursive patterns such as symmetric restrictions, block structures, or any pattern satisfying the exact identification condition.

This approach is particularly useful when economic theory suggests long-run restriction patterns that cannot be achieved through a recursive ordering. For example, in a system with real and nominal variables, one might want to impose that all real shocks have no long-run effect on nominal variables and vice versa, creating a block structure rather than a triangular one.

7.3 Options

Option	Description
<code>data</code>	$N \times K$ matrix of endogenous variables (N observations, K variables). Required.
<code>nlags</code>	Number of lags to include in the VAR. Required.
<code>idmat</code>	$K \times K$ identification matrix of zeros and ones. Element $(i, j) = 1$ means structural shock j <i>does</i> have a long-run effect on variable i ; $= 0$ means it does not. Must have exactly $K(K + 1)/2$ ones for exact identification (note: ones, not zeros). Required.
<code>var_names</code>	$1 \times K$ cell vector with variable names. Default is <code>{'var1', 'var2', ...}</code> .
<code>shock_names</code>	$1 \times K$ cell vector with structural shock names. Default is <code>{'var1', 'var2', ...}</code> . Essential for interpreting non-recursive patterns (e.g., <code>{'Supply', 'Demand', 'Monetary'}</code>).
<code>irf_lenght</code>	Length of impulse response functions. Default is 40.
<code>monte_carlo</code>	Number of Monte Carlo repetitions for computing IRF confidence intervals. Default is 1000.
<code>const</code>	Include constant term if <code>true</code> ; exclude if <code>false</code> . Default is <code>false</code> .
<code>lr</code>	$K \times K$ matrix of zeros and ones specifying restrictions on the reduced-form VAR. Element $(i, j) = 0$ restricts variable j from affecting variable i ; $= 1$ allows the effect. Default is no restrictions.
<code>dum</code>	Include seasonal dummies. Specify <code>'month'</code> for 11 monthly dummies or <code>'quarter'</code> for 3 quarterly dummies. Default is no dummies.
<code>exog</code>	$N \times E$ matrix of exogenous variables entering without lags (e.g., event dummies). Treated as deterministic in confidence band construction. Default is no exogenous variables.
<code>shock</code>	$K \times 1$ vector specifying shock sizes for each variable. Default is one-standard-deviation shocks (diagonal of estimated impact matrix).

<code>confianza</code>	Lower confidence band percentage. For example, <code>confianza = 5</code> produces 90% confidence intervals (5th and 95th percentiles). Default is 5.
------------------------	---

7.4 Stored results

`nonrecursiveLRSVAR` stores the following in structure `EQ`:

Matrices

<code>EQ.vari_shockj</code>	$K \times K$ collection of $3 \times \text{irf_length}$ matrices. For each variable i and shock j : first row = 95% confidence band, second row = IRF, third row = 5% confidence band. Field names use <code>var_names</code> for variables and <code>shock_names</code> for shocks.
<code>EQ.P</code>	$K \times K$ estimated structural impact matrix. Satisfies $PP' = \Sigma_u$ and ensures the long-run multiplier matrix $\Theta(1) = [I - \sum A_i]^{-1}P$ has the non-recursive zero pattern specified in <code>idmat</code> .
<code>EQ.vardecshock_shockj</code>	K matrices of size $K \times \text{irf_length}$. Each row shows the percentage of variance of that variable explained by shock j at each horizon. Field names use <code>shock_names</code> .
<code>EQ.stdirf</code>	$K \times \text{irf_length} \times K$ array of standard deviations of IRF coefficients across Monte Carlo simulations.
<code>EQ.struc</code>	$K \times (N - \text{lags})$ matrix of estimated structural shocks.

7.5 Remarks

Non-recursive long-run restrictions: The key innovation over `lrSVAR` is the ability to impose long-run restrictions that do not follow a recursive (lower triangular) pattern. Examples include:

- **Symmetric patterns:** Shock 1 affects variables 1 and 2; Shock 2 affects variables 1 and 3; Shock 3 affects variables 2 and 3
- **Block structures:** Real shocks affect only real variables; nominal shocks affect only nominal variables
- **Economic theory-driven:** Any pattern where $K(K+1)/2$ long-run restrictions are economically motivated

Exact identification requirement - CRITICAL: The `idmat` matrix must have **exactly** $K(K+1)/2$ ones. This is different from other SVAR functions:

- In `srSVAR`, `gmmSRSVAR`: need at least $K(K-1)/2$ **zeros**
- In `lrSVAR`, `gmmLRSVAR`: need at least $K(K-1)/2$ **zeros**
- In `nonrecursiveLRSVAR`: need exactly $K(K+1)/2$ **ones**

The function will return an error if $\sum_{i,j} \text{idmat}(i,j) \neq K(K+1)/2$.

Rubio-Ramirez-Waggoner-Zha algorithm: The estimation uses a QR decomposition-based algorithm:

1. Compute the long-run covariance of reduced-form residuals
2. Obtain Cholesky decomposition of this long-run covariance
3. Sort columns by number of zero restrictions (most restricted first)
4. For each column, use QR decomposition to find orthonormal vectors satisfying zero restrictions
5. Construct the long-run multiplier matrix from these orthonormal vectors
6. Recover the structural impact matrix via back-transformation

When to use non-recursive vs recursive:

- Use `lrSVAR` (Blanchard-Quah) when restrictions naturally form a triangular pattern
- Use `nonrecursiveLRSVAR` when:
 - Restrictions are symmetric (e.g., shock 1 $\nrightarrow$ var 2, shock 2 $\nrightarrow$ var 1)
 - Economic theory suggests block structure
 - Multiple orderings seem equally plausible (non-recursive removes ordering arbitrariness)

Stationarity requirement: Like all long-run identification methods, this requires either stationary variables or proper treatment of cointegration. Long-run effects are defined as $\Theta(1) = [I - \sum A_i]^{-1}P$, which requires $[I - \sum A_i]$ to be invertible.

Verifying restrictions: Cumulative IRFs approximate long-run effects. For stationary VARs, all IRFs decay to zero, but:

- If shock j has long-run effect on variable i : $\sum_{h=0}^{\infty} \text{IRF}_{i,j}(h) \neq 0$
- If shock j has no long-run effect on variable i : $\sum_{h=0}^{\infty} \text{IRF}_{i,j}(h) \approx 0$

The cumulative sum distinguishes between temporary fluctuations that cancel out versus persistent directional effects.

Sign normalization: The algorithm normalizes signs so that diagonal elements of the long-run multiplier matrix are positive. This ensures uniqueness of the decomposition.

Confidence intervals: Generated via Monte Carlo simulation, applying the full Rubio-Ramirez-Waggoner-Zha algorithm to each bootstrap sample.

Plots: The function automatically generates K figures, one for each structural shock, showing the responses of all K variables with confidence bands.

7.6 Examples

Example 1: Symmetric non-recursive pattern

Impose symmetric restrictions impossible with Blanchard-Quah:

```
var_names = {'Output', 'Inflation', 'Interest'};
shock_names = {'Productivity', 'Demand', 'Monetary'};

% Symmetric pattern: each shock affects exactly 2 variables in LR
% Productivity -> Output, Inflation (not Interest)
% Demand -> Inflation, Interest (not Output)
% Monetary -> Output, Interest (not Inflation)
idmat = [1 0 1;      % 2 ones
         1 1 0;      % 2 ones
         0 1 1];     % 2 ones
% Total: 6 = K(K+1)/2

EQ = nonrecursiveLRSVAR(data, 2, idmat, var_names, shock_names, ...
                        40, 1000, true);

% Verify restrictions via cumulative IRFs
cum_prod_interest = sum(EQ.Interest_Productivity(2,:));
cum_demand_output = sum(EQ.Output_Demand(2,:));
cum_monet_inflation = sum(EQ.Inflation_Monetary(2,:));

fprintf('Productivity -> Interest (LR): %.4f (should be ~0)\n', ...
        cum_prod_interest);
fprintf('Demand -> Output (LR):          %.4f (should be ~0)\n', ...
        cum_demand_output);
fprintf('Monetary -> Inflation (LR):      %.4f (should be ~0)\n', ...
        cum_monet_inflation);
```

Example 2: Block structure

Real shocks have no long-run effect on nominal variables:

```
var_names = {'Output', 'Employment', 'Inflation'};
shock_names = {'Productivity', 'Labor', 'Monetary'};

% Block pattern: real/nominal separation
idmat = [1 1 1;      % Output: all shocks
         1 1 1;      % Employment: all shocks
         0 0 1];      % Inflation: only Monetary
% Real shocks (Productivity, Labor) have no LR effect on Inflation

EQ = nonrecursiveLRSVAR(data, 2, idmat, var_names, shock_names, ...
                        40, 1000, true);

% Verify block structure
cum_prod_infl = sum(EQ.Inflation_Productivity(2,:));
cum_labor_infl = sum(EQ.Inflation_Labor(2,:));

fprintf('Productivity -> Inflation (LR): %.4f\n', cum_prod_infl);
fprintf('Labor -> Inflation (LR):      %.4f\n', cum_labor_infl);
% Both should be near zero
```

Example 3: Comparison with recursive identification

Compare non-recursive with standard Blanchard-Quah:

```
% Non-recursive pattern
idmat_nonrec = [1 0 1;
                1 1 0;
                0 1 1];

EQ_nonrec = nonrecursiveLRSVAR(data, 2, idmat_nonrec, ...
                               var_names, shock_names);

% Recursive (Blanchard-Quah)
EQ_rec = lrSVAR(data, 2, var_names, shock_names);

disp('Non-recursive allows symmetric patterns:');
disp(idmat_nonrec);

disp('Recursive requires lower triangular:');
disp([1 0 0; 1 1 0; 1 1 1]);
```

Example 4: Visualizing cumulative IRFs

Distinguish temporary vs permanent effects:

```
% Plot standard IRFs
figure;
subplot(1,2,1);
plot(1:40, EQ.Output_Productivity(2,:), 'k-', 'LineWidth', 2);
title('IRF (all decay to zero in stationary VAR)');

% Plot cumulative IRFs (long-run effects)
subplot(1,2,2);
plot(1:40, cumsum(EQ.Output_Productivity(2,:)), 'k-', 'LineWidth', 2);
hold on;
```

```

plot(1:40, cumsum(EQ.Output_Demand(2,:)), 'k--', 'LineWidth', 2);
title('Cumulative IRF (long-run multipliers)');
legend('Productivity (LR \neq 0)', 'Demand (LR = 0)');

% Cumulative for Demand should approach zero
% Cumulative for Productivity should remain non-zero

```

Example 5: Checking exact identification

Verify the idmat has exactly the right number of ones:

```

K = 3;
idmat = [1  0  1;
         1  1  0;
         0  1  1];

required = K*(K+1)/2;
actual = sum(idmat(:));

fprintf('Ones required: %d\n', required); % 6
fprintf('Ones in idmat: %d\n', actual);   % 6

if actual ~= required
    error('Not exactly identified!');
end

```

For complete, executable examples with non-recursive pattern analysis and cumulative IRF visualizations, see `example_nonrecursiveLRSVAR.m` in the package repository.

7.7 Methods and formulas

The structural VAR model is:

$$y_t = \nu + A_1 y_{t-1} + \dots + A_p y_{t-p} + \Upsilon d_t + \Psi x_t + P \varepsilon_t$$

where P is the structural impact matrix and $\varepsilon_t \sim N(0, I_K)$ are orthogonal structural shocks.

Long-run multiplier: The long-run effect of structural shocks on the variables is:

$$\Theta(1) = [I - A_1 - \dots - A_p]^{-1} P$$

Non-recursive restriction: The `idmat` matrix specifies which elements of $\Theta(1)$ are zero:

$$\Theta_{ij}(1) = 0 \quad \text{if and only if} \quad \text{idmat}(i, j) = 0$$

For exact identification, exactly $K(K-1)/2$ elements must be zero, equivalently exactly $K(K+1)/2$ elements must be non-zero (ones in `idmat`).

Rubio-Ramirez-Waggoner-Zha algorithm:

Step 1: Estimate reduced-form VAR, obtain $\hat{\Sigma}_u$ and $\sum_{i=1}^p \hat{A}_i$.

Step 2: Compute long-run covariance of reduced-form residuals:

$$\hat{\Omega} = \left[I - \sum_{i=1}^p \hat{A}_i \right]^{-1} \hat{\Sigma}_u \left[I - \sum_{i=1}^p \hat{A}_i \right]^{-1},$$

Step 3: Cholesky decomposition: $\hat{\Omega} = L_0 L_0'$ (lower triangular).

Step 4: Sort columns of `idmat` by number of zeros (most restricted first). Let I be the permutation.

Step 5: For each column $j = 1, \dots, K$:

- Let Z_j be the matrix whose rows correspond to zero restrictions for column j
- Define $\bar{Z}_j$ to select the zero-restricted rows
- Construct $Q_j^{\text{trans}} = [\bar{Z}_j L_0; q'_1, \dots, q'_{j-1}]'$
- QR decomposition: $Q_j^{\text{trans}'} = \tilde{Q}R$
- Set $q_j = \tilde{Q}(:, \text{end})$ (last column of $\tilde{Q}$)

Step 6: Construct $Q = [q_1, \dots, q_K]$ and $\tilde{\Theta}(1) = L_0 Q$.

Step 7: Undo the permutation:

$$\Theta(1)(:, I(j)) = \tilde{\Theta}(1)(:, j) \quad \text{for } j = 1, \dots, K$$

Step 8: Sign normalization: if $\Theta_{jj}(1) < 0$, flip the sign of column j .

Step 9: Recover structural impact matrix:

$$\hat{P} = \left[I - \sum_{i=1}^p \hat{A}_i \right] \Theta(1)$$

Verification:

$$\begin{aligned} \Theta(1) &= \left[I - \sum A_i \right]^{-1} \hat{P} \quad (\text{has desired zero pattern}) \\ \hat{P} \hat{P}' &= \left[I - \sum A_i \right] \Theta(1) \Theta(1)' \left[I - \sum A_i \right]' \\ &= \left[I - \sum A_i \right] L_0 Q Q' L_0' \left[I - \sum A_i \right]' \\ &= \left[I - \sum A_i \right] L_0 L_0' \left[I - \sum A_i \right]' \\ &= \left[I - \sum A_i \right] \hat{\Omega} \left[I - \sum A_i \right]' = \hat{\Sigma}_u \quad (\text{correct covariance}) \end{aligned}$$

Key property: The QR decomposition ensures that Q is orthonormal ($Q Q' = I$) and each column q_j is orthogonal to the space of zero-restricted rows, thereby satisfying the zero restrictions while maintaining orthogonality of structural shocks.

Impulse response functions:

$$\text{IRF}(h) = J \Phi^h P$$

where Φ is the companion matrix and $J = [I_K : 0 : \dots : 0]$.

Cumulative IRF (long-run effect):

$$\text{Cumulative IRF}(h) = \sum_{s=0}^h \text{IRF}(s) \xrightarrow{h \rightarrow \infty} \Theta(1)$$

For stationary VARs, individual IRFs decay to zero, but cumulative IRFs converge to non-zero long-run multipliers when $\Theta_{ij}(1) \neq 0$.

7.8 References

Rubio-Ramirez, J. F., D. F. Waggoner, and T. Zha. 2010. "Structural Vector Autoregressions: Theory of Identification and Algorithms for Inference." *Review of Economic Studies* 77(2): 665–696.

Lütkepohl, H. 2005. *New Introduction to Multiple Time Series Analysis*. Berlin: Springer-Verlag.

Amisano, G., and C. Giannini. 1997. *Topics in Structural VAR Econometrics*. 2nd ed. Berlin: Springer-Verlag.

8 gmmSRSVAR — GMM-estimated SVAR with short-run restrictions

8.1 Syntax

```
EQ = gmmSRSVAR(data, nlags, idmat, var_names, shock_names)
```

```
EQ = gmmSRSVAR(data, nlags, idmat, var_names, shock_names, irf_lenght, ...
monte_carlo, const, lr, dum, exog, shock, confianza)
```

8.2 Description

gmmSRSVAR estimates a structural vector autoregression (SVAR) model identified via short-run (contemporaneous) zero restrictions using Generalized Method of Moments (GMM). Unlike Cholesky decomposition which imposes exact identification with $K(K - 1)/2$ restrictions, GMM can accommodate overidentified models with more than the minimum number of zero restrictions. The function uses constrained optimization to find the structural impact matrix that minimizes the distance between the implied and observed residual covariance matrices, subject to the imposed zero restrictions.

This approach is particularly useful when economic theory suggests contemporaneous restrictions that do not fit a recursive (triangular) pattern, or when the researcher wishes to impose more restrictions than necessary for identification to test economic hypotheses.

8.3 Options

Option	Description
<code>data</code>	$N \times K$ matrix of endogenous variables (N observations, K variables). Required.
<code>nlags</code>	Number of lags to include in the VAR. Required.
<code>idmat</code>	$K \times K$ identification matrix of zeros and ones. Element $(i, j) = 0$ means structural shock j does <i>not</i> affect variable i contemporaneously; $= 1$ means it does. Must have at least $K(K - 1)/2$ zeros for identification. Can have more for overidentification. Required.
<code>var_names</code>	$1 \times K$ cell vector with variable names. Default is <code>{'var1', 'var2', ...}</code> .
<code>shock_names</code>	$1 \times K$ cell vector with structural shock names. Default is <code>{'var1', 'var2', ...}</code> . Allows economic interpretation separate from variable names (e.g., <code>{'Supply', 'Demand', 'MonetaryPolicy'}</code>).
<code>irf_lenght</code>	Length of impulse response functions. Default is 40.
<code>monte_carlo</code>	Number of Monte Carlo repetitions for computing IRF confidence intervals. Default is 1000.
<code>const</code>	Include constant term if <code>true</code> ; exclude if <code>false</code> . Default is <code>false</code> .
<code>lr</code>	$K \times K$ matrix of zeros and ones specifying restrictions on the reduced-form VAR. Element $(i, j) = 0$ restricts variable j from affecting variable i ; $= 1$ allows the effect. Default is no restrictions.
<code>dum</code>	Include seasonal dummies. Specify <code>'month'</code> for 11 monthly dummies or <code>'quarter'</code> for 3 quarterly dummies. Default is no dummies.
<code>exog</code>	$N \times E$ matrix of exogenous variables entering without lags (e.g., event dummies). Treated as deterministic in confidence band construction. Default is no exogenous variables.
<code>shock</code>	$K \times 1$ vector specifying shock sizes for each variable. Default is one-standard-deviation shocks (diagonal of estimated impact matrix).

<code>confianza</code>	Lower confidence band percentage. For example, <code>confianza = 5</code> produces 90% confidence intervals (5th and 95th percentiles). Default is 5.
------------------------	---

8.4 Stored results

`gmmSRSVAR` stores the following in structure `EQ`:

Matrices

<code>EQ.vari_shockj</code>	$K \times K$ collection of $3 \times \text{irf_length}$ matrices. For each variable i and shock j : first row = 95% confidence band, second row = IRF, third row = 5% confidence band. Field names use <code>var_names</code> for variables and <code>shock_names</code> for shocks.
<code>EQ.P</code>	$K \times K$ GMM-estimated structural impact matrix satisfying the zero restrictions specified in <code>idmat</code> . Maps structural shocks to reduced-form residuals: $u_t = P\varepsilon_t$.
<code>EQ.vardecshock_shockj</code>	K matrices of size $K \times \text{irf_length}$. Each row shows the percentage of variance of that variable explained by shock j at each horizon. Field names use <code>shock_names</code> .
<code>EQ.stdirf</code>	$K \times \text{irf_length} \times K$ array of standard deviations of IRF coefficients across Monte Carlo simulations.
<code>EQ.struc</code>	$K \times (N - \text{nlags})$ matrix of estimated structural shocks.

8.5 Remarks

GMM estimation: The function uses a two-step efficient GMM estimator. In the first step, the weighting matrix is constructed from the fourth moments of the residuals. In the second step, the structural impact matrix P is estimated by minimizing the weighted distance between the sample residual covariance $\text{vech}(\hat{\Sigma}_u)$ and the model-implied covariance $\text{vech}(PP')$, subject to zero restrictions.

Overidentification: A model is exactly identified when the number of zero restrictions equals $K(K - 1)/2$ (the number of free parameters in a lower triangular matrix). GMM allows overidentification when more than $K(K - 1)/2$ zeros are imposed. The GMM objective function value provides a test statistic (Hansen's J-statistic) for testing the validity of the overidentifying restrictions. Under the null hypothesis that the restrictions are valid, the J-statistic follows a χ^2 distribution with degrees of freedom equal to the number of overidentifying restrictions.

Constrained optimization: The function uses `fminsearchcon` (D'Errico, 2006) to perform constrained optimization, enforcing zero restrictions via bound constraints. The algorithm transforms the constrained problem into an unconstrained one using variable transformations, ensuring that zero restrictions are satisfied exactly at each iteration.

GMM vs Cholesky: For recursive (lower triangular) identification with exactly $K(K - 1)/2$ zeros, GMM estimation should produce results very similar to Cholesky decomposition. However, GMM is more flexible as it handles non-recursive patterns and overidentification, while Cholesky is restricted to recursive orderings.

Identification requirements: The `idmat` matrix must have:

- At least $K(K - 1)/2$ zeros (minimum for identification)
- Can have more than $K(K - 1)/2$ zeros (overidentification)
- Cannot have fewer than $K(K - 1)/2$ zeros (underidentified, will fail)

Confidence intervals: Generated via Monte Carlo simulation. For each draw, the GMM estimation is re-applied to obtain structural IRFs consistent with the zero restrictions. The empirical distribution across draws provides confidence intervals.

Shock naming: Separate `shock_names` from `var_names` allows clear economic interpretation. Results accessed as `EQ.Output_Supply`, `EQ.Inflation.MonetaryPolicy`, etc.

Plots: The function automatically generates K figures, one for each structural shock, showing the responses of all K variables with confidence bands.

8.6 Examples

Example 1: Basic GMM SVAR (recursive identification)

Estimate with recursive restrictions (exactly identified):

```
var_names = {'Output', 'Inflation', 'InterestRate'};
shock_names = {'Supply', 'Demand', 'MonetaryPolicy'};

% Recursive identification (3 zeros for K=3)
idmat = [1  0  0;
         1  1  0;
         1  1  1];

EQ = gmmSRSVAR(data, 2, idmat, var_names, shock_names, 40, 1000, true);

% Access impact matrix
disp(EQ.P);

% Check number of restrictions
fprintf('Zero restrictions: %d\n', sum(idmat(:)==0));
```

Example 2: Overidentified GMM SVAR

Impose one extra restriction (monetary policy shock affects only interest rate):

```
% Overidentified: 4 zeros (one more than minimum 3)
idmat_over = [1  0  0;
              1  1  0;
              0  0  1];

EQ2 = gmmSRSVAR(data, 2, idmat_over, var_names, shock_names, 40, ...
                1000, true);

fprintf('Zero restrictions: %d\n', sum(idmat_over(:)==0));
fprintf('Overidentifying restrictions: %d\n', ...
        sum(idmat_over(:)==0) - 3);
```

Example 3: Non-recursive identification pattern

Use a non-recursive pattern that cannot be achieved with Cholesky:

```
% Non-recursive: Supply affects output and inflation,
% Demand affects all, Policy affects inflation and rate
idmat_nonrec = [1  1  0;
                1  1  1;
                0  1  1];

EQ3 = gmmSRSVAR(data, 2, idmat_nonrec, var_names, shock_names, 40, ...
                1000, true);
```

Example 4: Custom shock sizes

Use 2-standard-deviation shocks:

```
shock_size = 2 * diag(EQ.P);
EQ4 = gmmSRSVAR(data, 2, idmat, var_names, shock_names, 40, 1000, ...
                true, [], [], [], shock_size);
```


Example 5: With restricted reduced-form VAR

Combine GMM structural identification with reduced-form restrictions:

```
% Interest rate does not affect output/inflation in lags
lr = [1 1 0;
      1 1 0;
      1 1 1];

EQ5 = gmmSRSVAR(data, 2, idmat, var_names, shock_names, 40, 1000, ...
               true, lr);
```

Example 6: Overidentification test

Test validity of overidentifying restrictions using J-statistic:

```
% The GMM objective function value at optimum provides J-statistic
% (not directly returned, but can be computed from GMM formulas)

% Compare just-identified vs overidentified
idmat_just = [1 0 0; 1 1 0; 1 1 1]; % 3 zeros
idmat_over = [1 0 0; 1 1 0; 0 0 1]; % 4 zeros

EQ_just = gmmSRSVAR(data, 2, idmat_just, var_names, shock_names);
EQ_over = gmmSRSVAR(data, 2, idmat_over, var_names, shock_names);

% If overidentifying restrictions are valid, impact matrices
% should be similar despite the extra restriction
```

For complete, executable examples with overidentification analysis and comparisons, see `example_gmmSRSVAR.m` in the package repository.

8.7 Methods and formulas

The structural VAR model is:

$$y_t = \nu + A_1 y_{t-1} + \cdots + A_p y_{t-p} + \Upsilon d_t + \Psi x_t + P \varepsilon_t$$

where P is the structural impact matrix satisfying zero restrictions, and $\varepsilon_t \sim N(0, I_K)$ are orthogonal structural shocks.

GMM estimation procedure:

Step 1: Estimate reduced-form VAR by OLS, obtain residuals $\hat{u}_t$ and covariance matrix $\hat{\Sigma}_u = T^{-1} \sum_{t=1}^T \hat{u}_t \hat{u}_t'$.

Step 2: Construct weighting matrix for efficient GMM:

$$\hat{\Omega} = T^{-1} \sum_{t=1}^T \left[\text{vech}(\hat{u}_t \hat{u}_t') - \text{vech}(\hat{\Sigma}_u) \right] \left[\text{vech}(\hat{u}_t \hat{u}_t') - \text{vech}(\hat{\Sigma}_u) \right]'$$

Step 3: Define GMM objective function:

$$J(P) = T \cdot [\text{vech}(\hat{\Sigma}_u) - \text{vech}(PP')]'\hat{\Omega}^{-1}[\text{vech}(\hat{\Sigma}_u) - \text{vech}(PP')]$$

Step 4: Minimize $J(P)$ subject to zero restrictions specified in `idmat`:

$$\hat{P} = \arg \min_P J(P) \quad \text{subject to } P_{ij} = 0 \text{ for all } (i, j) \text{ where } \text{idmat}(i, j) = 0$$

Step 5: At the optimum $\hat{P}$, the value $J(\hat{P})$ is Hansen's J-statistic for overidentification testing.

Zero restrictions: Let R be the set of pairs (i, j) such that shock j does not affect variable i contemporaneously:

$$P_{ij} = 0 \quad \text{for all } (i, j) \in R$$

The number of restrictions $|R|$ must satisfy:

- $|R| \geq K(K-1)/2$ for identification
- $|R| = K(K-1)/2$ gives exact identification
- $|R| > K(K-1)/2$ gives overidentification with $|R| - K(K-1)/2$ degrees of freedom

Overidentification test: Under the null hypothesis that all zero restrictions are valid:

$$J(\hat{P}) \xrightarrow{d} \chi^2_{|R| - K(K-1)/2}$$

where $|R| - K(K-1)/2$ is the number of overidentifying restrictions. Reject the null if $J(\hat{P})$ exceeds the critical value from the χ^2 distribution.

Impulse response functions: Computed as:

$$\text{IRF}(h) = J\Phi^h P$$

where Φ is the companion matrix and $J = [I_K : 0 : \dots : 0]$ is the selection matrix.

Forecast error variance decomposition: The contribution of shock j to the forecast error variance of variable i at horizon h is:

$$\text{FEVD}_{i,j}(h) = \frac{\sum_{s=0}^{h-1} (e'_i J \Phi^s P e_j)^2}{\sum_{k=1}^K \sum_{s=0}^{h-1} (e'_i J \Phi^s P e_k)^2} \times 100$$

Constrained optimization: The function uses bound constraints to enforce zeros:

- If $\text{idmat}(i, j) = 1$: set lower bound = $-\infty$, upper bound = $+\infty$ (free)
- If $\text{idmat}(i, j) = 0$: set lower bound = 0, upper bound = 0 (fixed at zero)

8.8 References

- Lütkepohl, H. 2005. *New Introduction to Multiple Time Series Analysis*. Berlin: Springer-Verlag.
- Hansen, L. P. 1982. "Large Sample Properties of Generalized Method of Moments Estimators." *Econometrica* 50(4): 1029–1054.
- D'Errico, J. 2006. *fminsearchcon: Constrained optimization using fminsearch*. MATLAB Central File Exchange. <https://www.mathworks.com/matlabcentral/fileexchange/8277>

9 gmmLRSVAR — GMM-estimated SVAR with long-run restrictions

9.1 Syntax

```
EQ = gmmLRSVAR(data, nlags, idmat, var_names, shock_names)
```

```
EQ = gmmLRSVAR(data, nlags, idmat, var_names, shock_names, irf_lenght, ...
monte_carlo, const, lr, dum, exog, shock, confianza)
```

9.2 Description

gmmLRSVAR estimates a structural vector autoregression (SVAR) model identified via long-run zero restrictions à la Blanchard and Quah (1989), using Generalized Method of Moments (GMM). The identification imposes that the long-run multiplier matrix $\Theta(1) = [I - A_1 - \dots - A_p]^{-1}P$ is lower triangular, constraining certain structural shocks to have no long-run effects on certain variables. Unlike exact identification via Cholesky decomposition, GMM can accommodate overidentified models with more than the minimum number of zero restrictions.

This approach is particularly useful when economic theory suggests that some shocks (e.g., demand shocks) should have only temporary effects on certain variables (e.g., output), while other shocks (e.g., supply shocks) can have permanent effects. GMM allows testing these restrictions via overidentification tests.

9.3 Options

Option	Description
<code>data</code>	$N \times K$ matrix of endogenous variables (N observations, K variables). Required.
<code>nlags</code>	Number of lags to include in the VAR. Required.
<code>idmat</code>	$K \times K$ identification matrix of zeros and ones. Element $(i, j) = 0$ means structural shock j does <i>not</i> have a long-run effect on variable i ; $= 1$ means it does. Must have at least $K(K-1)/2$ zeros for identification. Can have more for overidentification. Required.
<code>var_names</code>	$1 \times K$ cell vector with variable names. Default is <code>{'var1', 'var2', ...}</code> .
<code>shock_names</code>	$1 \times K$ cell vector with structural shock names. Default is <code>{'var1', 'var2', ...}</code> . Allows economic interpretation separate from variable names (e.g., <code>{'Supply', 'Demand', 'Labor'}</code>).
<code>irf_lenght</code>	Length of impulse response functions. Default is 40.
<code>monte_carlo</code>	Number of Monte Carlo repetitions for computing IRF confidence intervals. Default is 1000.
<code>const</code>	Include constant term if <code>true</code> ; exclude if <code>false</code> . Default is <code>false</code> .
<code>lr</code>	$K \times K$ matrix of zeros and ones specifying restrictions on the reduced-form VAR. Element $(i, j) = 0$ restricts variable j from affecting variable i ; $= 1$ allows the effect. Default is no restrictions.
<code>dum</code>	Include seasonal dummies. Specify <code>'month'</code> for 11 monthly dummies or <code>'quarter'</code> for 3 quarterly dummies. Default is no dummies.
<code>exog</code>	$N \times E$ matrix of exogenous variables entering without lags (e.g., event dummies). Treated as deterministic in confidence band construction. Default is no exogenous variables.
<code>shock</code>	$K \times 1$ vector specifying shock sizes for each variable. Default is one-standard-deviation shocks (diagonal of estimated impact matrix).

<code>confianza</code>	Lower confidence band percentage. For example, <code>confianza = 5</code> produces 90% confidence intervals (5th and 95th percentiles). Default is 5.
------------------------	---

9.4 Stored results

`gmmLRSVAR` stores the following in structure `EQ`:

Matrices

<code>EQ.vari_shockj</code>	$K \times K$ collection of $3 \times \text{irf_length}$ matrices. For each variable i and shock j : first row = 95% confidence band, second row = IRF, third row = 5% confidence band. Field names use <code>var_names</code> for variables and <code>shock_names</code> for shocks.
<code>EQ.P</code>	$K \times K$ GMM-estimated structural impact matrix. Satisfies $PP' = \Sigma_u$ and ensures the long-run multiplier matrix $\Theta(1) = [I - \sum A_i]^{-1}P$ is lower triangular.
<code>EQ.vardecshock_shockj</code>	K matrices of size $K \times \text{irf_length}$. Each row shows the percentage of variance of that variable explained by shock j at each horizon. Field names use <code>shock_names</code> .
<code>EQ.stdirf</code>	$K \times \text{irf_length} \times K$ array of standard deviations of IRF coefficients across Monte Carlo simulations.
<code>EQ.struc</code>	$K \times (N - \text{nlags})$ matrix of estimated structural shocks.

9.5 Remarks

Long-run restrictions: The Blanchard-Quah identification imposes that the long-run multiplier matrix $\Theta(1)$ is lower triangular. This means:

- Shock 2 has no long-run effect on variable 1
- Shock 3 has no long-run effect on variables 1 and 2
- And so on for systems with $K > 3$

These restrictions are motivated by economic theory, such as the idea that demand shocks have only temporary effects on output while supply shocks can have permanent effects.

GMM estimation: The function uses a two-step efficient GMM estimator. In the first step, the weighting matrix is constructed from the fourth moments of the residuals. In the second step, the parameter matrix B is estimated by minimizing the weighted distance between the sample long-run covariance $\text{vech}(\hat{\Omega})$ and the model-implied covariance $\text{vech}(BB')$, subject to zero restrictions. Here $\Omega = [I - \sum A_i]^{-1} \Sigma_u [I - \sum A_i]^{-1'}$ is the long-run covariance of reduced-form residuals.

Recovery of impact matrix: After estimating B , the structural impact matrix is recovered as:

$$P = [I - \sum_{i=1}^p A_i] \cdot B$$

This ensures that $\Theta(1) = [I - \sum A_i]^{-1}P = B$ is lower triangular, satisfying the long-run restrictions.

Overidentification: A model is exactly identified when the number of zero restrictions equals $K(K-1)/2$. GMM allows overidentification when more than $K(K-1)/2$ zeros are imposed. The GMM objective function value provides a test statistic (Hansen's J-statistic) for testing the validity of the overidentifying restrictions. Under the null hypothesis that the restrictions are valid, the J-statistic follows a χ^2 distribution with degrees of freedom equal to the number of overidentifying restrictions.

Constrained optimization: The function uses `fminsearchcon` (D'Errico, 2006) to perform constrained optimization, enforcing zero restrictions via bound constraints on the matrix B , not directly on P .

GMM vs Blanchard-Quah: For lower triangular identification with exactly $K(K-1)/2$ zeros, GMM estimation should produce results very similar to the standard Blanchard-Quah approach implemented in `1rSVAR`. However, GMM is more flexible as it handles overidentification and non-triangular patterns.

Identification requirements: The `idmat` matrix must have:

- At least $K(K-1)/2$ zeros (minimum for identification)
- Can have more than $K(K-1)/2$ zeros (overidentification)
- Cannot have fewer than $K(K-1)/2$ zeros (underidentified, will fail)

Stationarity requirement: Long-run restrictions require that the VAR is stationary in levels or that variables are cointegrated. If the data generating process is $I(1)$, consider estimating in first differences or using cointegration methods.

Verifying restrictions: Cumulative IRFs (sum over all horizons) approximate long-run effects. If shock j has no long-run effect on variable i , then $\sum_{h=0}^{\infty} \text{IRF}_{i,j}(h) \approx 0$.

Confidence intervals: Generated via Monte Carlo simulation. For each draw, the GMM estimation is re-applied to obtain structural IRFs consistent with the long-run zero restrictions. The empirical distribution across draws provides confidence intervals.

Shock naming: Separate `shock_names` from `var_names` is essential for long-run identification where shocks have economic interpretations distinct from variables.

Plots: The function automatically generates K figures, one for each structural shock, showing the responses of all K variables with confidence bands.

9.6 Examples

Example 1: Basic GMM long-run SVAR (Blanchard-Quah)

Estimate with standard Blanchard-Quah restrictions (exactly identified):

```
var_names = {'Output', 'Inflation', 'Unemployment'};
shock_names = {'Supply', 'Demand', 'Labor'};

% Lower triangular long-run restrictions (3 zeros for K=3)
idmat = [1  0  0;
         1  1  0;
         1  1  1];

EQ = gmmLRSVAR(data, 2, idmat, var_names, shock_names, 40, 1000, true);

% Access impact matrix
disp(EQ.P);

% Verify long-run restrictions via cumulative IRFs
cum_demand_output = sum(EQ.Output_Demand(2,:));
fprintf('Cumulative effect (Demand on Output): %.4f\n', cum_demand_output);
% Should be near zero
```

Example 2: Overidentified GMM long-run SVAR

Impose one extra restriction (supply shock has no long-run effect on unemployment):

```
% Overidentified: 4 zeros (one more than minimum 3)
idmat_over = [1  0  0;
              1  1  0;
              0  1  1];

EQ2 = gmmLRSVAR(data, 2, idmat_over, var_names, shock_names, 40, ...
                1000, true);

fprintf('Zero restrictions: %d\n', sum(idmat_over(:)==0));
fprintf('Overidentifying restrictions: %d\n', ...
```

```

sum(idmat_over(:)==0) - 3);

% Verify extra restriction
cum_supply_unemp = sum(EQ2.Unemployment_Supply(2,:));
fprintf('Supply -> Unemployment (LR): %.4f (should be ~0)\n', ...
        cum_supply_unemp);

```

Example 3: Analyzing long-run effects

Examine which shocks have permanent vs temporary effects:

```

% Cumulative IRFs approximate long-run multipliers
cum_supply_output = sum(EQ.Output_Supply(2,:));
cum_demand_output = sum(EQ.Output_Demand(2,:));

fprintf('Long-run effect on Output:\n');
fprintf(' Supply shock: %.4f (permanent)\n', cum_supply_output);
fprintf(' Demand shock: %.4f (temporary, ~0)\n', cum_demand_output);

```

Example 4: Variance decomposition at long horizons

At long horizons, only shocks with permanent effects contribute to variance:

```

% Extract variance decomposition at horizon 40
vd_supply_h40 = EQ.vardecshock_Supply(1, 40);
vd_demand_h40 = EQ.vardecshock_Demand(1, 40);

fprintf('Output variance at h=40:\n');
fprintf(' Supply: %.2f%%\n', vd_supply_h40);
fprintf(' Demand: %.2f%% (should be near 0%%)\n', vd_demand_h40);

```

Example 5: Comparison with Blanchard-Quah

Compare GMM with standard Blanchard-Quah:

```

EQ_gmm = gmmLRSVAR(data, 2, idmat, var_names, shock_names);
EQ_bq = lrSVAR(data, 2, var_names, shock_names);

% Impact matrices should be similar
disp('GMM P:');
disp(EQ_gmm.P);
disp('Blanchard-Quah P:');
disp(EQ_bq.P);

max_diff = max(abs(EQ_gmm.P(:) - EQ_bq.P(:)));
fprintf('Max difference: %.6f\n', max_diff);

```

Example 6: Overidentification test

Test validity of overidentifying restrictions:

```

% Just-identified model
idmat_just = [1 0 0; 1 1 0; 1 1 1];

% Overidentified model (one extra restriction)
idmat_over = [1 0 0; 1 1 0; 0 1 1];

EQ_just = gmmLRSVAR(data, 2, idmat_just, var_names, shock_names);
EQ_over = gmmLRSVAR(data, 2, idmat_over, var_names, shock_names);

```

% If extra restriction is valid, estimates should be similar
 % J-statistic (not directly returned) tests this formally

For complete, executable examples with overidentification analysis and long-run effect visualizations, see `example.gmmLRSVAR.m` in the package repository.

9.7 Methods and formulas

The structural VAR model is:

$$y_t = \nu + A_1 y_{t-1} + \dots + A_p y_{t-p} + \Upsilon d_t + \Psi x_t + P \varepsilon_t$$

where P is the structural impact matrix and $\varepsilon_t \sim N(0, I_K)$ are orthogonal structural shocks.

Long-run multiplier: The long-run effect of a one-unit structural shock ε on y is:

$$\Theta(1) = [I - A_1 - \dots - A_p]^{-1} P$$

GMM estimation procedure:

Step 1: Estimate reduced-form VAR by OLS, obtain residuals $\hat{u}_t$, covariance matrix $\hat{\Sigma}_u$, and coefficient matrices $\hat{A}_1, \dots, \hat{A}_p$.

Step 2: Compute the sum of lag coefficients:

$$\text{somma} = \sum_{i=1}^p \hat{A}_i$$

Step 3: Compute the long-run covariance of reduced-form residuals:

$$\hat{\Omega} = [I - \text{somma}]^{-1} \hat{\Sigma}_u [I - \text{somma}]^{-1'}$$

Step 4: Construct weighting matrix for efficient GMM (same as short-run GMM):

$$\hat{\Psi} = T^{-1} \sum_{t=1}^T \left[\text{vech}(\hat{u}_t \hat{u}_t') - \text{vech}(\hat{\Sigma}_u) \right] \left[\text{vech}(\hat{u}_t \hat{u}_t') - \text{vech}(\hat{\Sigma}_u) \right]'$$

Step 5: Define GMM objective function:

$$J(B) = T \cdot [\text{vech}(\hat{\Omega}) - \text{vech}(BB')]'\hat{\Psi}^{-1}[\text{vech}(\hat{\Omega}) - \text{vech}(BB')]$$

Step 6: Minimize $J(B)$ subject to zero restrictions specified in `idmat`:

$$\hat{B} = \arg \min_B J(B) \quad \text{subject to } B_{ij} = 0 \text{ for all } (i, j) \text{ where } \text{idmat}(i, j) = 0$$

Step 7: Recover the structural impact matrix:

$$\hat{P} = [I - \text{somma}] \cdot \hat{B}$$

This ensures $\Theta(1) = [I - \text{somma}]^{-1} \hat{P} = \hat{B}$ is lower triangular.

Verification:

$$\Theta(1) = [I - \text{somma}]^{-1} \hat{P} = [I - \text{somma}]^{-1} ([I - \text{somma}] \hat{B}) = \hat{B} \quad (\text{lower triangular})$$

$$\hat{P} \hat{P}' = [I - \text{somma}] \hat{B} \hat{B}' [I - \text{somma}]' = [I - \text{somma}] \hat{\Omega} [I - \text{somma}]' = \hat{\Sigma}_u \quad (\text{correct covariance})$$

Zero restrictions: Let R be the set of pairs (i, j) such that shock j has no long-run effect on variable i :

$$\Theta_{ij}(1) = 0 \quad \text{for all } (i, j) \in R$$

Equivalently, $B_{ij} = 0$ for all $(i, j) \in R$.

The number of restrictions $|R|$ must satisfy:

- $|R| \geq K(K-1)/2$ for identification
- $|R| = K(K-1)/2$ gives exact identification
- $|R| > K(K-1)/2$ gives overidentification with $|R| - K(K-1)/2$ degrees of freedom

Overidentification test: Under the null hypothesis that all zero restrictions are valid:

$$J(\hat{B}) \xrightarrow{d} \chi^2_{|R| - K(K-1)/2}$$

where $|R| - K(K-1)/2$ is the number of overidentifying restrictions.

Impulse response functions: Computed as:

$$\text{IRF}(h) = J\Phi^h P$$

where Φ is the companion matrix and $J = [I_K : 0 : \dots : 0]$ is the selection matrix.

Long-run effects: The long-run effect of shock j on variable i is:

$$\Theta_{ij}(1) = e_i' [I - \sum A_k]^{-1} P e_j = B_{ij}$$

where e_i is the i -th unit vector. By construction, $B_{ij} = 0$ for $i < j$ (lower triangular).

9.8 References

- Blanchard, O. J., and D. Quah. 1989. "The Dynamic Effects of Aggregate Demand and Supply Disturbances." *American Economic Review* 79(4): 655–673.
- Lütkepohl, H. 2005. *New Introduction to Multiple Time Series Analysis*. Berlin: Springer-Verlag.
- Hansen, L. P. 1982. "Large Sample Properties of Generalized Method of Moments Estimators." *Econometrica* 50(4): 1029–1054.
- D'Errico, J. 2006. *fminsearchcon: Constrained optimization using fminsearch*. MATLAB Central File Exchange. <https://www.mathworks.com/matlabcentral/fileexchange/8277>

10 maxshareSVAR — Max share identification

10.1 Syntax

```
EQ = maxshareSVAR(data, H, nlags, accum, var_names, shock_names)
```

```
EQ = maxshareSVAR(data, H, nlags, accum, var_names, shock_names, ...  
                  irf_lenght, monte_carlo, const, lr, dum, exog, ...  
                  shock, confianza)
```

10.2 Description

maxshareSVAR estimates a structural vector autoregression (SVAR) model using max share identification (Uhlig, 2003; Barsky and Sims, 2011; Francis et al., 2014). This method identifies the first structural shock as the shock that maximizes its contribution to the forecast error variance of the first variable at a specified horizon H .

This approach is particularly useful when economic theory suggests that a particular shock (e.g., a technology shock) should be the dominant driver of fluctuations in a particular variable (e.g., output) at a certain frequency or horizon. Unlike full identification methods, max share achieves only partial identification: it uniquely identifies the first shock, while the remaining $K - 1$ shocks are subject to rotational indeterminacy.

CRITICAL: Variable ordering matters. The **first variable** in the data matrix is the one whose variance is maximized, and the **first shock** is the one identified by the procedure. Place the variable of interest as the first column of the data matrix.

10.3 Options

Option	Description
<code>data</code>	$N \times K$ matrix of endogenous variables (N observations, K variables). IMPORTANT: The first variable is the one whose variance decomposition is maximized. Place the variable of interest in the first column. Required.
<code>H</code>	Scalar. Horizon at which to maximize the variance decomposition. Typical values: $H = 1$ for impact shocks, $H = 12$ – 20 for business cycle shocks, $H = 40$ + for long-run shocks. Required.
<code>nlags</code>	Number of lags to include in the VAR. Required.
<code>accum</code>	Scalar, either 1 or 0. If 1 (default), maximize the cumulative variance decomposition across all horizons from 0 to H . If 0, maximize the variance decomposition only at exactly horizon H . Note: with <code>accum=0</code> , the first shock explains 100% of the non-cumulative variance at horizon H , but the reported variance decomposition (which is always cumulative) will be less than 100%. Default is 1.
<code>var_names</code>	$1 \times K$ cell vector with variable names. Default is <code>{'var1', 'var2', ...}</code> .
<code>shock_names</code>	$1 \times K$ cell vector with structural shock names. Default is <code>{'var1', 'var2', ...}</code> . The first shock is typically given an economic interpretation (e.g., 'TFP'). The remaining shocks are not uniquely identified.
<code>irf_lenght</code>	Length of impulse response functions. Default is 40.
<code>monte_carlo</code>	Number of Monte Carlo repetitions for computing IRF confidence intervals. Default is 1000.
<code>const</code>	Include constant term if <code>true</code> ; exclude if <code>false</code> . Default is <code>false</code> .
<code>lr</code>	$K \times K$ matrix of zeros and ones specifying restrictions on the reduced-form VAR. Element $(i, j) = 0$ restricts variable j from affecting variable i ; $= 1$ allows the effect. Default is no restrictions.

<code>dum</code>	Include seasonal dummies. Specify <code>'month'</code> for 11 monthly dummies or <code>'quarter'</code> for 3 quarterly dummies. Default is no dummies.
<code>exog</code>	$N \times E$ matrix of exogenous variables entering without lags (e.g., event dummies). Treated as deterministic in confidence band construction. Default is no exogenous variables.
<code>shock</code>	$K \times 1$ vector specifying shock sizes for each variable. Default is one-standard-deviation shocks (diagonal of estimated impact matrix).
<code>confianza</code>	Lower confidence band percentage. For example, <code>confianza = 5</code> produces 90% confidence intervals (5th and 95th percentiles). Default is 5.

10.4 Stored results

`maxshareSVAR` stores the following in structure `EQ`:

Matrices

<code>EQ.vari_shockj</code>	$K \times K$ collection of $3 \times \text{irf_length}$ matrices. For each variable i and shock j : first row = 95% confidence band, second row = IRF, third row = 5% confidence band. Note: Typically only the first shock is analyzed, as remaining shocks are not uniquely identified. Field names use <code>var_names</code> for variables and <code>shock_names</code> for shocks.
<code>EQ.B</code>	$K \times K$ estimated structural impact matrix. Satisfies $BB' = \Sigma_u$ and the first column maximizes variance contribution to the first variable at horizon H .
<code>EQ.vardecshock_shockj</code>	K matrices of size $K \times \text{irf_length}$. Each row shows the percentage of variance of that variable explained by shock j at each horizon. Important: These are always cumulative variance decompositions, regardless of the <code>accum</code> parameter. Field names use <code>shock_names</code> .
<code>EQ.stdirf</code>	$K \times \text{irf_length} \times K$ array of standard deviations of IRF coefficients across Monte Carlo simulations.
<code>EQ.struc</code>	$K \times (N - \text{nlags})$ matrix of estimated structural shocks.

10.5 Remarks

Partial identification: Max share identification achieves only **partial identification**. It uniquely identifies the first structural shock as the shock that maximizes its contribution to the variance of the first variable at horizon H . The remaining $K - 1$ shocks are orthogonal to the first shock and to each other, but their individual economic interpretation is not unique (rotational indeterminacy). In practice, researchers typically analyze only the first shock and ignore the others.

Variable ordering matters - CRITICAL: Unlike Cholesky or long-run identification where variable ordering affects all shocks symmetrically, in max share identification the **first variable is special**. The first shock is identified as maximizing variance of the first variable. If you reorder variables, you identify a different shock. This is a feature, not a bug: it allows you to target specific shocks by appropriate variable ordering.

Choosing horizon H : The horizon H determines which shock is identified:

- $H = 1$: Identifies the shock with maximum impact effect (impact maximizer)
- $H = 4-8$: Identifies short-run shocks
- $H = 12-20$: Identifies medium-run/business cycle shocks (common choice for TFP shocks)
- $H = 40+$: Identifies long-run/permanent shocks

Different values of H will generally identify different shocks with different economic interpretations.

Cumulative vs point-in-time variance (`accum` parameter):

- `accum = 1` (default): Maximizes cumulative variance contribution from horizon 0 to H : $\sum_{s=0}^H \text{Var}_s$
- `accum = 0`: Maximizes variance contribution at exactly horizon H : Var_H

With `accum = 0`, the first shock explains 100% of the non-cumulative variance at horizon H by construction. However, the reported variance decomposition (`EQ.vardecshock_*`) is always cumulative over horizons 0 to h , so it will be less than 100% even at H .

Eigenvalue decomposition method: The identification uses eigenvalue decomposition of the variance contribution matrix. Given the reduced-form VAR residual covariance Σ_u with Cholesky decomposition $PP' = \Sigma_u$, the algorithm constructs a matrix measuring each shock's contribution to the first variable's variance and finds the rotation of P that maximizes this contribution via the largest eigenvector.

Sign normalization: The algorithm normalizes the sign so that the first shock has a positive impact effect on the first variable.

Numerical stability: The variance contribution matrix should be symmetric but may have small numerical asymmetries. The function forces symmetry before eigenvalue decomposition to avoid complex eigenvalues.

Applications: Common applications include:

- Identifying TFP shocks by maximizing variance of productivity or output at business cycle frequencies
- Identifying news shocks by maximizing variance at longer horizons
- Identifying demand shocks by maximizing variance of consumption
- Any setting where one shock should dominate variance of one variable at a particular horizon

Confidence intervals: Generated via Monte Carlo simulation, applying the full max share algorithm to each bootstrap sample.

Plots: The function automatically generates K figures, one for each structural shock, showing the responses of all K variables with confidence bands. Typically, only the first figure (first shock) is analyzed.

10.6 Examples

Example 1: Basic max share SVAR

Identify TFP shock as the shock maximizing output variance at business cycle frequencies:

```
% IMPORTANT: Put output first!
var_names = {'Output', 'Hours', 'Inflation'};
shock_names = {'TFP', 'LaborSupply', 'MonetaryPolicy'};

H = 20;          % Business cycle horizon
accum = 1;       % Cumulative variance

EQ = maxshareSVAR(data, H, nlags, accum, var_names, shock_names, ...
                  40, 1000, true);

% Check variance contribution
fprintf('TFP explains %.2f%% of Output variance at H=%d\n', ...
        EQ.vardecshock_TFP(1,H), H);

% Analyze only the first shock (TFP)
figure;
plot(1:40, EQ.Output_TFP(2,:), 'k-', 'LineWidth', 2);
title('Output response to TFP shock');
```

Example 2: Variable ordering matters

Different orderings identify different shocks:

```

% Order 1: Output first → identifies TFP shock
data1 = data;
var_names1 = {'Output', 'Hours', 'Inflation'};
shock_names1 = {'TFP', 'Shock2', 'Shock3'};
EQ1 = maxshareSVAR(data1, 20, nlags, 1, var_names1, shock_names1);

% Order 2: Hours first → identifies labor supply shock
data2 = data(:, [2, 1, 3]); % Reorder columns!
var_names2 = {'Hours', 'Output', 'Inflation'};
shock_names2 = {'LaborSupply', 'Shock2', 'Shock3'};
EQ2 = maxshareSVAR(data2, 20, nlags, 1, var_names2, shock_names2);

% These identify DIFFERENT shocks!

```

Example 3: Choosing horizon H

Compare identification at different horizons:

```

H_values = [1, 4, 12, 20, 40];

for i = 1:length(H_values)
    H = H_values(i);
    EQ_H{i} = maxshareSVAR(data, H, nlags, 1, var_names, ...
                           shock_names);

    fprintf('H=%d: TFP explains %.2f%% at horizon %d\n', ...
            H, EQ_H{i}.vardecshock_TFP(1,H), H);
end

% Different H identifies different shocks!

```

Example 4: Cumulative vs point-in-time

Compare the two modes:

```

H = 20;

% Cumulative variance (0 to H)
EQ_cumul = maxshareSVAR(data, H, nlags, 1, var_names, shock_names);

% Point-in-time variance (exactly at H)
EQ_point = maxshareSVAR(data, H, nlags, 0, var_names, shock_names);

fprintf('Cumulative: %.2f%% at H=%d\n', ...
        EQ_cumul.vardecshock_TFP(1,H), H);
fprintf('Point-in-time: %.2f%% at H=%d\n', ...
        EQ_point.vardecshock_TFP(1,H), H);

% Note: Both report cumulative variance decomposition
% With accum=0, the shock explains 100% of non-cumulative
% variance at H=20, but less of cumulative variance

```

Example 5: Only analyze first shock

Typical usage focuses on the identified shock only:

```

EQ = maxshareSVAR(data, 20, nlags, 1, var_names, shock_names);

```

```

% Analyze only the first shock (TFP)
figure;
subplot(3,1,1);
plot(1:40, EQ.Output_TFP(2,:), 'k-', 'LineWidth', 2);
title('Output response to TFP');

subplot(3,1,2);
plot(1:40, EQ.Hours_TFP(2,:), 'k-', 'LineWidth', 2);
title('Hours response to TFP');

subplot(3,1,3);
plot(1:40, EQ.Inflation_TFP(2,:), 'k-', 'LineWidth', 2);
title('Inflation response to TFP');

% Do NOT analyze EQ.Output_LaborSupply, etc.
% Those shocks are not uniquely identified!

```

For complete, executable examples with horizon comparisons and variance decomposition analysis, see `example_maxshareSVAR.m` in the package repository.

10.7 Methods and formulas

The structural VAR model is:

$$y_t = \nu + A_1 y_{t-1} + \dots + A_p y_{t-p} + \Upsilon d_t + \Psi x_t + B \varepsilon_t$$

where B is the structural impact matrix and $\varepsilon_t \sim N(0, I_K)$ are orthogonal structural shocks.

Max share algorithm:

Step 1: Estimate reduced-form VAR by OLS, obtain residual covariance $\hat{\Sigma}_u$.

Step 2: Cholesky decomposition: $\hat{\Sigma}_u = PP'$ (lower triangular).

Step 3: Construct the variance contribution matrix. Let $e_1 = [1, 0, \dots, 0]'$ be the selection vector for the first variable. Define:

$$\Omega(h) = \begin{cases} \sum_{s=0}^{h-1} (A^s C)' e_1 e_1' A^s C & \text{if accum} = 1 \\ (A^{h-1} C)' e_1 e_1' A^{h-1} C & \text{if accum} = 0 \end{cases}$$

where $C = [P; 0_{K(p-1) \times K}]$ and A is the companion matrix.

Step 4: Eigenvalue decomposition of $\Omega(H)$:

$$\Omega(H) = Q \Lambda Q'$$

where $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_K)$ with $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_K$, and Q is orthogonal.

Step 5: The eigenvector q_1 corresponding to the largest eigenvalue λ_1 defines the rotation that maximizes variance contribution.

Step 6: Construct the structural impact matrix:

$$\hat{B} = PQ$$

Step 7: Sign normalization: if $\hat{B}_{11} < 0$, flip the signs of the first column: $\hat{B}(:, 1) \leftarrow -\hat{B}(:, 1)$.

Interpretation: The first column of $\hat{B}$ (first shock) maximizes:

- If `accum = 1`: $\sum_{s=0}^{H-1} (\text{IRF}_{1,1}(s))^2$, the cumulative contribution to variance of variable 1
- If `accum = 0`: $(\text{IRF}_{1,1}(H-1))^2$, the contribution at exactly horizon $H-1$

Variance decomposition: The percentage of variance of variable i explained by shock j at horizon h is:

$$\text{VD}_{i,j}(h) = \frac{\sum_{s=0}^{h-1} (e_i' \Phi^s B e_j)^2}{\sum_{k=1}^K \sum_{s=0}^{h-1} (e_i' \Phi^s B e_k)^2} \times 100$$

where Φ is the VAR coefficient matrix in companion form. Note: This is always cumulative from 0 to $h - 1$, regardless of the `accum` parameter.

Impulse response functions:

$$\text{IRF}(h) = J\Phi^h B$$

where $J = [I_K : 0 : \dots : 0]$ is the selection matrix.

Partial identification: The first shock is uniquely identified as the eigenvector corresponding to λ_1 . The remaining $K - 1$ shocks correspond to eigenvectors for $\lambda_2, \dots, \lambda_K$ and are subject to rotational indeterminacy within the subspace orthogonal to the first shock.

Max share property at horizon H : At the chosen horizon H , the first shock achieves:

$$\max_{q: \|q\|=1} e_1' \sum_{s=0}^{H-1} \Phi^s P q q' P' (\Phi^s)' e_1$$

subject to orthogonality: $q'q = 1$.

10.8 References

- Uhlig, H. 2003. "What Moves Real GNP?" Unpublished manuscript, Humboldt University.
- Barsky, R. B., and E. R. Sims. 2011. "News Shocks and Business Cycles." *Journal of Monetary Economics* 58(3): 273–289.
- Francis, N., M. T. Owyang, J. E. Roush, and R. DiCecio. 2014. "A Flexible Finite-Horizon Alternative to Long-Run Restrictions with an Application to Technology Shocks." *Review of Economics and Statistics* 96(4): 638–647.
- Kurmann, A., and C. Otrok. 2013. "News Shocks and the Slope of the Term Structure of Interest Rates." *American Economic Review* 103(6): 2612–2632.
- Lütkepohl, H. 2005. *New Introduction to Multiple Time Series Analysis*. Berlin: Springer-Verlag.

11 bayes_maxshareSVAR — Bayesian max share identification

11.1 Syntax

```
EQ = bayes_maxshareSVAR(data, H, nlags, accum, var_names, shock_names)

EQ = bayes_maxshareSVAR(data, H, nlags, accum, var_names, shock_names, ...
                        irf_lenght, gibbs, const, lr, dum, exog, shock, ...
                        confianza, parbayes, burnin, verbose)
```

11.2 Description

bayes_maxshareSVAR estimates a structural vector autoregression (SVAR) model using Bayesian inference with max share identification (Uhlig, 2003; Barsky and Sims, 2011). The method combines a Minnesota Independent Normal-Wishart prior on the VAR parameters with max share identification, where the first structural shock is identified as the shock that maximizes its contribution to the forecast error variance of the first variable at a specified horizon H .

This Bayesian approach offers several advantages over frequentist max share identification: (1) incorporation of prior information through the Minnesota prior, (2) full characterization of posterior uncertainty through the Gibbs sampler, (3) more stable estimates in small samples, and (4) natural treatment of parameter uncertainty in impulse response functions.

CRITICAL: Variable ordering matters. The **first variable** in the data matrix is the one whose variance is maximized, and the **first shock** is the one identified by the procedure. Place the variable of interest as the first column of the data matrix.

11.3 Options

Option	Description
data	$N \times K$ matrix of endogenous variables (N observations, K variables). IMPORTANT: The first variable is the one whose variance decomposition is maximized. Place the variable of interest in the first column. Required.
H	Scalar. Horizon at which to maximize the variance decomposition. Typical values: $H = 1$ for impact shocks, $H = 12$ – 20 for business cycle shocks, $H = 40$ + for long-run shocks. Required.
nlags	Number of lags to include in the VAR. Required.
accum	Scalar, either 1 or 0. If 1 (default), maximize the cumulative variance decomposition across all horizons from 0 to H . If 0, maximize the variance decomposition only at exactly horizon H . Note: with <code>accum=0</code> , each Gibbs draw explains 100% of the non-cumulative variance at horizon H , but the posterior median may differ from 100% in cumulative variance. Default is 1.
var_names	$1 \times K$ cell vector with variable names. Default is {'var1', 'var2', ...}.
shock_names	$1 \times K$ cell vector with structural shock names. Default is {'var1', 'var2', ...}. The first shock is typically given an economic interpretation (e.g., 'TFP'). The remaining shocks are not uniquely identified.
irf_lenght	Length of impulse response functions. Default is 40.
gibbs	Number of Gibbs sampler iterations for posterior inference. Default is 2000.
const	Include constant term if <code>true</code> ; exclude if <code>false</code> . Default is <code>false</code> .
lr	$K \times K$ matrix of zeros and ones specifying restrictions on the reduced-form VAR. Element $(i, j) = 0$ restricts variable j from affecting variable i ; $= 1$ allows the effect. Default is no restrictions.

<code>dum</code>	Include seasonal dummies. Specify <code>'month'</code> for 11 monthly dummies or <code>'quarter'</code> for 3 quarterly dummies. Default is no dummies.
<code>exog</code>	$N \times E$ matrix of exogenous variables entering without lags (e.g., event dummies). Default is no exogenous variables.
<code>shock</code>	$K \times 1$ vector specifying shock sizes for each variable. Default is one-standard-deviation shocks (diagonal of posterior mean impact matrix).
<code>confianza</code>	Lower credible interval percentage. For example, <code>confianza = 16</code> produces 68% credible intervals (16th and 84th percentiles). Default is 16.
<code>parbayes</code>	1×6 vector of Minnesota prior hyperparameters: (1) prior mean on first own lag coefficient (default: 1), (2) prior variance on first own lag (default: 0.2), (3) ratio of cross-lag to own-lag variance (default: 1), (4) lag decay parameter (default: 2), (5) constant prior variance (default: 10^2), (6) exogenous variable prior variance (default: 10^2).
<code>burnin</code>	Number of burn-in iterations to discard before collecting posterior draws. Default is 500.
<code>verbose</code>	If <code>true</code> , print iteration numbers during Gibbs sampling; if <code>false</code> , suppress output. Default is <code>false</code> .

11.4 Stored results

`bayes_maxshareSVAR` stores the following in structure `EQ`:

Matrices

<code>EQ.vari_shockj</code>	$K \times K$ collection of $3 \times \text{irf_length}$ matrices. For each variable i and shock j : first row = 84th percentile, second row = posterior median IRF, third row = 16th percentile. Note: Typically only the first shock is analyzed, as remaining shocks are not uniquely identified. Field names use <code>var_names</code> for variables and <code>shock_names</code> for shocks.
<code>EQ.B</code>	$K \times K$ posterior mean of the structural impact matrix. The first column maximizes variance contribution to the first variable at horizon H .
<code>EQ.vardecshock_shockj</code>	K matrices of size $K \times \text{irf_length}$. Each row shows the percentage of variance of that variable explained by shock j at each horizon (posterior mean). Important: These are always cumulative variance decompositions. Field names use <code>shock_names</code> .
<code>EQ.constant_mean</code>	$K \times 1$ posterior mean of the constant term (if <code>const=true</code>).
<code>EQ.exo_mean</code>	$K \times E$ posterior mean of coefficients on exogenous variables (if <code>exog</code> provided).
<code>EQ.coefficient_mean</code>	$K \times (K \cdot \text{nlags})$ posterior mean of VAR coefficients.
<code>EQ.coeff_sim</code>	$K \times (\text{total_coeffs}) \times \text{gibbs}$ array of all coefficient draws from the posterior. Useful for posterior analysis.
<code>EQ.irf_sim</code>	$K \times \text{irf_length} \times K \times \text{gibbs}$ array of all IRF draws from the posterior. Allows detailed uncertainty quantification.

11.5 Remarks

Bayesian inference with max share identification: This function combines Minnesota prior-based Bayesian inference with max share identification. In each Gibbs iteration, the algorithm:

1. Draws VAR parameters from their posterior conditional on the residual covariance
2. Draws the residual covariance from its posterior conditional on VAR parameters
3. Applies max share identification to find the rotation that maximizes variance contribution

The result is a full posterior distribution over impulse responses that accounts for both parameter uncertainty and identification.

Partial identification: Like the frequentist version, Bayesian max share achieves only **partial identification**. It uniquely identifies the first structural shock, while the remaining $K - 1$ shocks are subject to rotational indeterminacy. In practice, researchers typically analyze only the first shock.

Variable ordering matters - CRITICAL: The **first variable** in the data matrix is special. The first shock is identified as maximizing variance of the first variable. Different variable orderings identify different shocks.

Minnesota prior: The prior specification follows the Minnesota/Litterman approach:

- Prior mean: First own lag = `parbayes(1)` (typically 1 for unit root prior), all other coefficients = 0
- Prior variance decays with lag order: $\text{Var}(\beta_{ij,\ell}) \propto 1/\ell^{\text{decay}}$
- Own-lag coefficients have tighter prior than cross-lag coefficients
- Scaled by residual variances from univariate autoregressions

Posterior inference: The function reports:

- **Posterior median** as the point estimate for IRFs (second row of `EQ.vari_shockj`)
- **Credible intervals** based on posterior percentiles (default: 16th and 84th percentiles for 68% intervals)
- **Posterior mean** for variance decompositions and structural parameters

Choosing horizon H: The horizon H determines which shock is identified. Different values identify different shocks:

- $H = 1$: Impact shocks
- $H = 4-8$: Short-run shocks
- $H = 12-20$: Medium-run/business cycle shocks (common for TFP)
- $H = 40+$: Long-run/permanent shocks

Cumulative vs point-in-time variance (accum parameter):

- `accum = 1` (default): Maximizes cumulative variance from horizon 0 to H
- `accum = 0`: Maximizes variance at exactly horizon H

With `accum = 0`, each Gibbs draw produces a shock explaining 100% of non-cumulative variance at H . However, since the posterior median is taken across draws, and reported variance decompositions are cumulative, the final numbers may differ from 100%.

Gibbs sampler: The algorithm uses a Gibbs sampler to draw from the joint posterior distribution of VAR parameters and residual covariance:

1. Draw $\beta|\Sigma, Y$ from Normal distribution
2. Draw $\Sigma|\beta, Y$ from Inverse-Wishart distribution
3. Apply max share identification to each draw
4. Compute IRFs for each draw

Burn-in iterations are discarded; remaining draws form the posterior sample.

Numerical stability: The variance contribution matrix should be symmetric but may have small numerical asymmetries. The function forces symmetry before eigenvalue decomposition to avoid complex eigenvalues.

Comparison with frequentist: Bayesian max share typically produces:

- More stable estimates in small samples (prior regularization)
- Narrower credible intervals than bootstrap confidence intervals (especially with informative priors)
- Estimates closer to prior beliefs when data are weak

Hyperparameter selection: The default Minnesota prior is relatively uninformative. For stronger prior beliefs:

- Reduce `parbayes(2)` for tighter prior on own lags
- Increase `parbayes(4)` for faster decay with lag order
- Adjust `parbayes(1)` if first own lag should differ from 1

Plots: The function automatically generates K figures, one for each structural shock, showing the responses of all K variables with credible bands. Typically, only the first figure (first shock) is analyzed.

11.6 Examples

Example 1: Basic Bayesian max share SVAR

Identify TFP shock as the shock maximizing output variance:

```
% IMPORTANT: Put output first!
var_names = {'Output', 'Hours', 'Inflation'};
shock_names = {'TFP', 'LaborSupply', 'MonetaryPolicy'};

H = 20;      % Business cycle horizon
accum = 1;   % Cumulative variance

EQ = bayes_maxshareSVAR(data, H, nlags, accum, var_names, ...
                        shock_names, 40, 2000, true);

% Check variance contribution (posterior mean)
fprintf('TFP explains %.2f%% of Output variance at H=%d\n', ...
        EQ.vardecshock_TFP(1,H), H);

% Analyze only the first shock (TFP)
figure;
plot(1:40, EQ.Output_TFP(2,:), 'k-', 'LineWidth', 2);
title('Output response to TFP shock (posterior median)');
```

Example 2: Custom Minnesota prior

Use a tighter prior on own lags:

```
% Default: parbayes = [1, 0.2, 1, 2, 100, 100]
% Tighter: reduce variance on own lags
parbayes_tight = [1, 0.1, 1, 2, 100, 100];

EQ_tight = bayes_maxshareSVAR(data, H, nlags, accum, var_names, ...
                              shock_names, 40, 2000, true, [], ...
                              [], [], 16, parbayes_tight);

% Compare with default prior
EQ_default = bayes_maxshareSVAR(data, H, nlags, accum, var_names, ...
                                shock_names, 40, 2000, true);

% Tighter prior shrinks estimates toward prior mean
```

Example 3: Posterior distribution analysis

Examine full posterior using EQ.irf_sim:

```
EQ = bayes_maxshareSVAR(data, H, nlags, 1, var_names, shock_names);

% Extract all posterior draws for TFP -> Output at horizon 10
% irf_sim dimensions: [variable, horizon, shock, draw]
irf_draws = squeeze(EQ.irf_sim(1, 10, 1, :));

% Compute posterior statistics
fprintf('Posterior mean:    %.4f\n', mean(irf_draws));
fprintf('Posterior median: %.4f\n', median(irf_draws));
fprintf('Posterior std:     %.4f\n', std(irf_draws));

% Plot posterior distribution
histogram(irf_draws, 50, 'Normalization', 'pdf');
xline(median(irf_draws), 'r-', 'LineWidth', 2);
title('Posterior: TFP shock effect on Output at h=10');
```

Example 4: Comparison with frequentist

Compare Bayesian and frequentist max share:

```
% Frequentist
EQ_freq = maxshareSVAR(data, H, nlags, 1, var_names, shock_names);

% Bayesian
EQ_bayes = bayes_maxshareSVAR(data, H, nlags, 1, var_names, ...
                               shock_names);

% Compare point estimates
fprintf('Frequentist IRF at h=10: %.4f\n', ...
        EQ_freq.Output_TFP(2,10));
fprintf('Bayesian IRF at h=10:    %.4f\n', ...
        EQ_bayes.Output_TFP(2,10));

% Bayesian credible intervals often narrower due to prior
```

Example 5: Burn-in and convergence

Use longer burn-in for better convergence:

```
% Long burn-in with verbose output
EQ = bayes_maxshareSVAR(data, H, nlags, 1, var_names, shock_names, ...
                        40, 2000, true, [], [], [], [], 16, [], ...
                        1000, true);

% 1000 burn-in iterations discarded
% 2000 iterations kept for inference
% verbose=true prints iteration numbers
```

For complete, executable examples with posterior analysis and Bayesian-frequentist comparisons, see `example_bayes_maxshare` in the package repository.

11.7 Methods and formulas

The structural VAR model is:

$$y_t = \nu + A_1 y_{t-1} + \dots + A_p y_{t-p} + \Upsilon d_t + \Psi x_t + B \varepsilon_t$$

where B is the structural impact matrix and $\varepsilon_t \sim N(0, I_K)$ are orthogonal structural shocks.

Minnesota prior on VAR parameters:

Stacking all coefficients into $\beta = \text{vec}([\nu, \Psi, A_1, \dots, A_p])$, the prior is:

$$\beta | \Sigma \sim N(\bar{\beta}, \bar{V})$$

where:

- $\bar{\beta}_{ij,1} = \text{parbayes}(1) \cdot \mathbb{I}(i = j)$ for first own lag, 0 otherwise
- $\bar{V}_{ij,\ell} = \begin{cases} \text{parbayes}(2)/\ell^{\text{decay}} & \text{if } i = j \\ \text{parbayes}(3) \cdot \text{parbayes}(2) \cdot \sigma_i^2 / (\ell^{\text{decay}} \sigma_j^2) & \text{if } i \neq j \end{cases}$
- $\bar{V}_{\text{const}} = \text{parbayes}(5) \cdot \sigma_i^2$
- $\bar{V}_{\text{exog}} = \text{parbayes}(6) \cdot \sigma_i^2$

where σ_i^2 are residual variances from univariate AR(p) regressions.

Prior on residual covariance:

The prior on Σ is uninformative:

$$\Sigma^{-1} \sim W(0, 0_{K \times K})$$

(Wishart distribution with 0 degrees of freedom)

Gibbs sampler:

Iteration $g = 1, \dots, G + B$ where B is burn-in:

Step 1: Draw $\beta^{(g)} | \Sigma^{(g-1)}, Y$ from:

$$\beta^{(g)} \sim N(\tilde{\beta}, \tilde{V})$$

where:

$$\begin{aligned} \tilde{V} &= \left(\bar{V}^{-1} + (ZZ')^{-1} \otimes \Sigma^{(g-1)-1} \right)^{-1} \\ \tilde{\beta} &= \tilde{V} \left(\bar{V}^{-1} \bar{\beta} + (ZZ')^{-1} \otimes \Sigma^{(g-1)-1} \cdot \text{vec}(\hat{\beta}_{\text{OLS}}) \right) \end{aligned}$$

Step 2: Draw $\Sigma^{(g)} | \beta^{(g)}, Y$ from:

$$\Sigma^{(g)-1} \sim W \left(T, \left[(Y - \beta^{(g)} Z)(Y - \beta^{(g)} Z)' \right]^{-1} \right)$$

Step 3: Apply max share identification to $\Sigma^{(g)}$ to obtain $B^{(g)}$.

Let $P = \text{chol}(\Sigma^{(g)})'$. Construct variance contribution matrix:

$$\Omega(h) = \begin{cases} \sum_{s=0}^{h-1} (A^s C)' e_1 e_1' A^s C & \text{if } \text{accum} = 1 \\ (A^{h-1} C)' e_1 e_1' A^{h-1} C & \text{if } \text{accum} = 0 \end{cases}$$

Force symmetry: $\Omega(H) \leftarrow (\Omega(H) + \Omega(H)')/2$

Eigenvalue decomposition: $\Omega(H) = Q \Lambda Q'$

Set $B^{(g)} = PQ$ with sign normalization.

Step 4: Compute IRFs using $B^{(g)}$ and $\beta^{(g)}$.

Posterior inference:

After discarding B burn-in iterations, the remaining G draws form a sample from the joint posterior $p(\beta, \Sigma, B | Y)$.

Posterior summaries:

- Posterior mean: $\mathbb{E}[\theta | Y] \approx \frac{1}{G} \sum_{g=1}^G \theta^{(g)}$
- Posterior median: $\text{median}(\{\theta^{(1)}, \dots, \theta^{(G)}\})$
- Credible interval: $[\theta^{(\lfloor G \cdot \alpha / 100 \rfloor)}, \theta^{(\lfloor G \cdot (100 - \alpha) / 100 \rfloor)}]$

Max share property: In each draw, the first column of $B^{(g)}$ solves:

$$\max_{q: \|q\|=1} e_1' \sum_{s=0}^{H-1} \Phi^{(g)s} P^{(g)} q q' P^{(g)'} (\Phi^{(g)s})' e_1$$

The posterior distribution over B reflects both sampling uncertainty and the max share criterion.

11.8 References

- Uhlig, H. 2003. "What Moves Real GNP?" Unpublished manuscript, Humboldt University.
- Barsky, R. B., and E. R. Sims. 2011. "News Shocks and Business Cycles." *Journal of Monetary Economics* 58(3): 273–289.
- Koop, G., and D. Korobilis. 2010. "Bayesian Multivariate Time Series Methods for Empirical Macroeconomics." *Foundations and Trends in Econometrics* 3(4): 267–358.
- Lütkepohl, H. 2005. *New Introduction to Multiple Time Series Analysis*. Berlin: Springer-Verlag.
- Litterman, R. B. 1986. "Forecasting with Bayesian Vector Autoregressions—Five Years of Experience." *Journal of Business & Economic Statistics* 4(1): 25–38.

12 signrestSVAR — Sign restrictions identification

12.1 Syntax

```
EQ = signrestSVAR(data, nlags, sign, periods_sign, var_names, shock_names)
```

```
EQ = signrestSVAR(data, nlags, sign, periods_sign, var_names, shock_names, ...  
                  irf_lenght, repetitions, const, lr, dum, exog, shock, ...  
                  confianza, vardec, verbose)
```

12.2 Description

signrestSVAR estimates a structural vector autoregression (SVAR) model using sign restrictions to achieve identification (Uhlig, 2005; Rubio-Ramirez, Waggoner, and Zha, 2010). The method imposes restrictions on the signs of impulse responses rather than requiring exact zero restrictions, making it particularly useful when economic theory provides qualitative predictions about shock effects but not precise quantitative restrictions.

Unlike recursive or long-run identification schemes that yield a unique structural model, sign restrictions identify a **set of models** that satisfy the imposed sign constraints. The function reports the **median** impulse response across all accepted models, capturing identification uncertainty rather than parameter uncertainty.

The approach uses uninformative Bayesian priors (inverse Wishart on Σ , Normal on coefficients centered at OLS) combined with random rotations of the Cholesky decomposition to generate candidate structural matrices. Only models satisfying the sign restrictions are retained, and inference is based on the distribution of accepted models.

12.3 Options

Option	Description
<code>data</code>	$N \times K$ matrix of endogenous variables (N observations, K variables). Required.
<code>nlags</code>	Number of lags to include in the VAR. Required.
<code>sign</code>	$K \times K$ matrix specifying sign restrictions. Element (i, j) indicates the required sign of the response of variable i to shock j : +1 for positive, -1 for negative, 0 for no restriction. Required.
<code>periods_sign</code>	Scalar. Number of periods over which sign restrictions must hold. Default is 1 (impact restrictions only). Larger values impose restrictions at multiple horizons, producing tighter identification but requiring stronger economic assumptions.
<code>var_names</code>	$1 \times K$ cell vector with variable names. Default is {'var1', 'var2', ...}.
<code>shock_names</code>	$1 \times K$ cell vector with structural shock names. Default is {'var1', 'var2', ...}.
<code>irf_lenght</code>	Length of impulse response functions. Default is 40.
<code>repetitions</code>	Number of accepted models to collect. The algorithm generates random rotations until this many models satisfying the sign restrictions are found. Default is 1000. If fewer than 1000 models are found after 1000 attempts per repetition, the function terminates with a warning.
<code>const</code>	Include constant term if <code>true</code> ; exclude if <code>false</code> . Default is <code>false</code> .
<code>lr</code>	$K \times K$ matrix of zeros and ones specifying restrictions on the reduced-form VAR. Element $(i, j) = 0$ restricts variable j from affecting variable i ; $= 1$ allows the effect. Default is no restrictions.
<code>dum</code>	Include seasonal dummies. Specify 'month' for 11 monthly dummies or 'quarter' for 3 quarterly dummies. Default is no dummies.

<code>exog</code>	$N \times E$ matrix of exogenous variables entering without lags (non-dynamic regressors, e.g., event dummies). Default is no exogenous variables.
<code>shock</code>	$K \times 1$ vector specifying shock sizes for each variable. Default is one-standard-deviation shocks (diagonal of median impact matrix across accepted models).
<code>confianza</code>	Lower confidence interval percentage. For example, <code>confianza = 5</code> produces 90% confidence intervals (5th and 95th percentiles). Default is 5.
<code>vardec</code>	If 1, compute variance decomposition; if 0, skip. Default is 0. Variance decomposition adds computational time.
<code>verbose</code>	If true , print iteration numbers during model search; if false , suppress output. Default is false .

12.4 Stored results

`signrestSVAR` stores the following in structure `EQ`:

Matrices

<code>EQ.vari_shockj</code>	$K \times K$ collection of $3 \times \text{irf_length}$ matrices. For each variable i and shock j : first row = 95th percentile (or $100 - \text{confianza}$), second row = median IRF across accepted models, third row = 5th percentile (or <code>confianza</code>). Field names use <code>var_names</code> for variables and <code>shock_names</code> for shocks.
<code>EQ.vardecshock_shockj</code>	K matrices of size $K \times \text{irf_length}$ (if <code>vardec=1</code>). Each row shows the percentage of variance of that variable explained by shock j at each horizon (median across accepted models). Field names use <code>shock_names</code> .
<code>EQ.irfsim</code>	$K \times \text{irf_length} \times \text{repetitions} \times K$ array of all IRF draws from accepted models. Useful for analyzing the full distribution of identification uncertainty.

12.5 Remarks

Set identification: Sign restrictions achieve only **set identification**, not point identification. The method identifies a set of structural models, all of which satisfy the imposed sign constraints. The reported IRFs are medians across this set, reflecting identification uncertainty. This differs fundamentally from Cholesky or long-run restrictions, which identify a unique structural model (point identification).

Identification vs parameter uncertainty: Sign restrictions focus on **identification uncertainty** (which structural models are consistent with the restrictions) rather than **parameter uncertainty** (sampling variation in VAR parameters). The function uses uninformative Bayesian priors—not the Minnesota prior used in other Bayesian SVAR functions—to generate draws of VAR parameters, but the primary source of uncertainty in the reported confidence bands is the multiplicity of structural models satisfying the sign restrictions.

Uninformative priors: Unlike `bayes_srSVAR` and other Bayesian functions in this package, `signrestSVAR` uses **uninformative priors**:

- Prior on Σ^{-1} : Wishart with degrees of freedom T (relatively uninformative)
- Prior on coefficients: Normal centered at OLS estimates with variance proportional to the residual covariance
- No Minnesota prior hyperparameters

This approach is standard in the sign restrictions literature and keeps the focus on identification uncertainty.

Sign restriction matrix: The `sign` matrix has dimensions $K \times K$ where:

- Rows correspond to variables (ordered as in `data`)

- Columns correspond to structural shocks
- Element (i, j) specifies the required sign of the response of variable i to shock j :
 - +1: positive response required
 - -1: negative response required
 - 0: no restriction

Example for a monetary policy shock (third column) in a three-variable VAR:

$$\text{sign} = \begin{bmatrix} 0 & 0 & -1 \\ 0 & 0 & -1 \\ 0 & 0 & +1 \end{bmatrix}$$

This requires: output ↓, inflation ↓, interest rate ↑ in response to a contractionary monetary policy shock.

Multi-period restrictions: Setting `periods_sign > 1` requires sign restrictions to hold over multiple horizons. For example, `periods_sign = 4` requires the signs to be satisfied at horizons 0, 1, 2, and 3. This:

- Produces tighter identification (smaller set of accepted models)
- Requires stronger economic assumptions
- Can be used to rule out "wrong sign" responses at short horizons

Algorithm - QR decomposition with sign checks: The identification algorithm follows Rubio-Ramirez, Waggoner, and Zha (2010):

1. Draw Σ from inverse Wishart posterior
2. Draw VAR coefficients from conditional Normal posterior
3. Compute Cholesky decomposition: $P = \text{chol}(\Sigma)'$
4. Generate random orthonormal matrix Q via QR decomposition of $W \sim N(0, I)$
5. Candidate impact matrix: $B = PQ$
6. Permute columns of B to best match sign pattern (preserving orthogonality)
7. Check if B satisfies sign restrictions
8. If yes, compute IRFs and (optionally) variance decomposition; if no, return to step 4
9. Repeat until `repetitions` accepted models are found

Column permutation: The algorithm includes a column permutation step (lines 353-369 in the code) that reorders the columns of B to best match the sign pattern. This is useful when identifying multiple shocks, as it finds the best assignment of random orthogonal shocks to economic interpretations while preserving orthogonality.

When to use sign restrictions: Sign restrictions are particularly useful when:

- Economic theory provides clear qualitative predictions but not quantitative restrictions
- Recursive (Cholesky) ordering is difficult to justify
- Long-run restrictions are inappropriate (e.g., shocks with temporary effects)
- Identifying multiple shocks with distinct qualitative patterns (e.g., supply vs demand)

Median vs mean: The function reports the **median** IRF across accepted models as the point estimate. The median is more robust to outliers than the mean and is standard practice in the sign restrictions literature.

Confidence bands: The reported confidence bands (default: 5th and 95th percentiles) reflect the distribution of IRFs across the set of accepted models. These bands capture identification uncertainty, not sampling uncertainty in VAR parameters.

Computational considerations: The algorithm searches for models satisfying the sign restrictions by random sampling. If restrictions are very tight, many candidate models may be rejected, increasing computation time. The function terminates with a warning if fewer than 1000 qualifying models are found after extensive search. Consider:

- Using fewer restrictions (more zeros in `sign` matrix)
- Reducing `periods_sign`
- Reducing `repetitions`

Plots: The function automatically generates K figures, one for each structural shock, showing the responses of all K variables with confidence bands.

12.6 Examples

Example 1: Basic sign restrictions - monetary policy shock

Identify a contractionary monetary policy shock:

```
var_names = {'Output', 'Inflation', 'InterestRate'};
shock_names = {'Supply', 'Demand', 'MonetaryPolicy'};

% Monetary policy shock: Output ↓, Inflation ↓, Rate ↑
sign_matrix = [0  0 -1;    % Output
               0  0 -1;    % Inflation
               0  0  1];   % Interest rate

EQ = signrestSVAR(data, nlags, sign_matrix, 1, var_names, shock_names);

% Median IRF across accepted models
figure;
plot(1:40, EQ.Output_MonetaryPolicy(2,:), 'k-', 'LineWidth', 2);
title('Output response to Monetary Policy shock (median)');
```

Example 2: Multiple shocks with sign restrictions

Identify supply, demand, and monetary shocks simultaneously:

```
% Supply:   Output ↑, Inflation ↓
% Demand:   Output ↑, Inflation ↑
% Monetary: Output ↓, Inflation ↓, Rate ↑
sign_full = [ 1  1 -1;    % Output
             -1  1 -1;    % Inflation
              0  0  1];   % Interest rate

EQ = signrestSVAR(data, nlags, sign_full, 1, var_names, shock_names);

% All three shocks now identified via distinct sign patterns
```

Example 3: Multi-period sign restrictions

Require restrictions to hold for 4 periods:

```

periods_sign = 4; % Restrictions at horizons 0, 1, 2, 3

EQ = signrestSVAR(data, nlags, sign_matrix, periods_sign, ...
    var_names, shock_names);

% Tighter identification: smaller set of accepted models
% Confidence bands will be narrower

```

Example 4: Variance decomposition and identification uncertainty

Analyze identification uncertainty using EQ.irfsim:

```

EQ = signrestSVAR(data, nlags, sign_full, 1, var_names, shock_names, ...
    40, 1000, true, [], [], [], [], 5, 1); % vardec=1

% Variance decomposition (median across models)
fprintf('Supply explains %.1f%% of Output variance at h=20\n', ...
    EQ.vardecshock_Supply(1,20));

% Examine full distribution of accepted models
% irfsim dimensions: [variable, horizon, repetition, shock]
irf_draws = squeeze(EQ.irfsim(1, 10, :, 3)); % Output, h=10, MP shock

% Summary statistics
fprintf('Median: %.4f\n', median(irf_draws));
fprintf('Std: %.4f\n', std(irf_draws));

% Plot distribution
histogram(irf_draws, 30);
title('Distribution of MP shock effect on Output (h=10)');

```

Example 5: Controlling for exogenous shocks

Include oil price shock as exogenous variable:

```

% Create oil shock dummy
exog = zeros(T, 1);
exog(80:85) = 1; % Oil shock period

EQ = signrestSVAR(data, nlags, sign_matrix, 1, var_names, ...
    shock_names, 40, 1000, true, [], [], exog);

% Sign restrictions applied to oil-adjusted structural shocks

```

For complete, executable examples including impact vs multi-period comparisons and identification uncertainty analysis, see `example_signrestSVAR.m` in the package repository.

12.7 Methods and formulas

The structural VAR model is:

$$y_t = \nu + A_1 y_{t-1} + \cdots + A_p y_{t-p} + \Upsilon d_t + \Psi x_t + B \varepsilon_t$$

where B is the structural impact matrix and $\varepsilon_t \sim N(0, I_K)$ are orthogonal structural shocks.

Reduced form:

The reduced-form VAR is:

$$y_t = \nu + A_1 y_{t-1} + \cdots + A_p y_{t-p} + \Upsilon d_t + \Psi x_t + u_t$$

with $u_t = B\varepsilon_t$ and $\mathbb{E}[u_t u_t'] = \Sigma = BB'$.

Identification problem:

Given Σ , any decomposition $B = PQ$ where $P = \text{chol}(\Sigma)'$ and Q is orthonormal ($QQ' = I$) satisfies $BB' = \Sigma$. Without additional restrictions, B is not uniquely identified.

Sign restrictions approach:

Let $\mathcal{S}$ denote a set of sign restrictions on impulse responses. A candidate impact matrix B is **admissible** if the impulse responses it generates satisfy all restrictions in $\mathcal{S}$.

The identified set is:

$$\mathcal{B} = \{B : BB' = \Sigma \text{ and } B \text{ satisfies } \mathcal{S}\}$$

Sign restrictions identify a **set** of structural models, not a unique model.

Algorithm (Rubio-Ramirez, Waggoner, Zha, 2010):

For $m = 1, \dots, M$ (where $M = \text{repetitions}$):

1. Draw $\Sigma^{(m)}$ from inverse Wishart posterior:

$$\Sigma^{(m)-1} \sim W(T, [(Y - \hat{\beta}Z)(Y - \hat{\beta}Z)']^{-1})$$

2. Draw $\beta^{(m)}$ from conditional Normal posterior:

$$\beta^{(m)} \sim N(\hat{\beta}, (ZZ')^{-1} \otimes \Sigma^{(m)})$$

3. Compute Cholesky decomposition: $P^{(m)} = \text{chol}(\Sigma^{(m)})'$

4. Generate random orthonormal matrix Q :

- Draw $W \sim N(0, I_K)$
- QR decomposition: $W = QR$
- Normalize so diagonal of R is positive
- Set $Q = WR^{-1}$

5. Candidate impact matrix: $B^{(m)} = P^{(m)}Q$

6. Column permutation: Find permutation π of columns of $B^{(m)}$ that best matches sign pattern

7. Compute impulse responses:

$$\Phi_h = JA^h C \quad \text{where } C = \begin{bmatrix} B^{(m)} \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

8. Check sign restrictions:

- Impact: $\text{sign}(\Phi_0) = \text{sign matrix}$
- Multi-period: $\text{sign}(\Phi_h) = \text{sign matrix}$ for $h = 0, 1, \dots, \text{periods_sign} - 1$

9. If restrictions satisfied:

- Store $B^{(m)}, \Phi_h^{(m)}$
- If `vardec=1`, compute variance decomposition
- Accept model

Otherwise, return to step 4 (up to 1000 attempts per repetition)

Reporting:

Point estimates (median across accepted models):

$$\hat{\Phi}_h = \text{median}\{\Phi_h^{(1)}, \dots, \Phi_h^{(M)}\}$$

Confidence bands (percentiles across accepted models):

$$\text{CI}_\alpha(\Phi_h) = [\text{percentile}_{\alpha/2}\{\Phi_h^{(m)}\}, \text{percentile}_{1-\alpha/2}\{\Phi_h^{(m)}\}]$$

Variance decomposition:

The contribution of shock j to the variance of variable i at horizon h is:

$$\omega_{ij}(h) = \frac{\sum_{s=0}^{h-1} [\Phi_s e_j]_i^2}{\sum_{s=0}^{h-1} \Phi_s \Phi_s'_{ii}} \times 100\%$$

where e_j is the j -th column of I_K and $[\cdot]_i$ denotes the i -th element.

Reported variance decomposition is the median across accepted models:

$$\hat{\omega}_{ij}(h) = \text{median}\{\omega_{ij}^{(1)}(h), \dots, \omega_{ij}^{(M)}(h)\}$$

Column permutation algorithm:

Given candidate B and sign matrix S , find permutation π such that $\text{sign}(B_{\pi(j)}) \approx S_j$ for all shocks j :
For each shock column j in S :

1. Find first available column k of B such that $\text{sign}(B_k) \cdot S_j \geq 0$ (all elements)
2. If found: assign $B_{\pi(j)} = \pm B_k$ (choosing sign to maximize agreement)
3. Remove column k from available columns
4. If not found: assign first remaining column

This preserves orthogonality while maximizing consistency with sign patterns.

12.8 References

- Uhlig, H. 2005. "What Are the Effects of Monetary Policy on Output? Results from an Agnostic Identification Procedure." *Journal of Monetary Economics* 52(2): 381–419.
- Rubio-Ramirez, J. F., D. F. Waggoner, and T. Zha. 2010. "Structural Vector Autoregressions: Theory of Identification and Algorithms for Inference." *Review of Economic Studies* 77(2): 665–696.
- Fry, R., and A. Pagan. 2011. "Sign Restrictions in Structural Vector Autoregressions: A Critical Review." *Journal of Economic Literature* 49(4): 938–960.
- Arias, J. E., J. F. Rubio-Ramirez, and D. F. Waggoner. 2018. "Inference Based on Structural Vector Autoregressions Identified with Sign and Zero Restrictions: Theory and Applications." *Econometrica* 86(2): 685–720.
- Lütkepohl, H. 2005. *New Introduction to Multiple Time Series Analysis*. Berlin: Springer-Verlag.

13 bloque_exo — Block-exogenous SVAR with different sample sizes

13.1 Syntax

```
EQ = bloque_exo(data_bloqueexo, data_bloqueendo, nlags, var_names, shock_names)

EQ = bloque_exo(data_bloqueexo, data_bloqueendo, nlags, var_names, shock_names, ...
                irf_lenght, monte_carlo, const, lrexo, lrendo, dum, exog, ...
                shock, confianza, verbose)
```

13.2 Description

bloque_exo estimates a structural vector autoregression (SVAR) model with a **block-exogenous structure** where the exogenous block has a possibly larger sample than the endogenous block. The method uses Cholesky (short-run) restrictions with a block-triangular structure, where exogenous variables affect endogenous variables but not vice versa.

This approach is particularly useful when:

- Variables can be partitioned into exogenous and endogenous blocks based on economic theory
- Data availability differs across blocks (e.g., international data available for longer periods than domestic data)
- Maximum efficiency is desired for identifying exogenous block shocks (using full sample)

Common applications include small open economy models (world $\rightarrow$ domestic), sectoral analysis (aggregate $\rightarrow$ sector-specific), and regional analysis (national $\rightarrow$ regional).

The identification achieves a **block-triangular impact matrix**:

$$P = \begin{bmatrix} P_{\text{exo}} & 0 \\ P_{\text{link}} & P_{\text{endo}} \end{bmatrix}$$

where P_{exo} identifies exogenous block shocks (using full sample T_1), P_{endo} identifies endogenous block shocks (using overlapping sample $T_2 < T_1$), and P_{link} captures contemporaneous effects of exogenous shocks on endogenous variables.

13.3 Options

Option	Description
<code>data_bloqueexo</code>	$N_{\text{exo}} \times K_1$ matrix of exogenous block variables (N_{exo} observations, K_1 variables). These variables are ordered first and affect the endogenous block but are not affected by it. Required.
<code>data_bloqueendo</code>	$N_{\text{endo}} \times K_2$ matrix of endogenous block variables (N_{endo} observations, K_2 variables). These variables are ordered after the exogenous block. It is assumed that $N_{\text{endo}} < N_{\text{exo}}$ and the endogenous block corresponds to the last N_{endo} observations of the exogenous block. Required.
<code>nlags</code>	Number of lags to include in the VAR. Required.
<code>var_names</code>	$1 \times (K_1 + K_2)$ cell vector with variable names. Exogenous block variables first, then endogenous block variables. Default is <code>{'var1', 'var2', ...}</code> .
<code>shock_names</code>	$1 \times (K_1 + K_2)$ cell vector with structural shock names. Default is <code>{'var1', 'var2', ...}</code> .
<code>irf_lenght</code>	Length of impulse response functions. Default is 40.
<code>monte_carlo</code>	Number of Monte Carlo repetitions for computing confidence intervals. Default is 1000.

<code>const</code>	Include constant term if <code>true</code> ; exclude if <code>false</code> . Default is <code>false</code> .
<code>lrexo</code>	$K_1 \times K_1$ matrix of zeros and ones specifying restrictions on the reduced-form VAR for the exogenous block. Element $(i, j) = 0$ restricts variable j from affecting variable i ; $= 1$ allows the effect. Default is no restrictions.
<code>lrendo</code>	$K_2 \times K_2$ matrix of zeros and ones specifying restrictions on the reduced-form VAR for the endogenous block. Element $(i, j) = 0$ restricts variable j from affecting variable i ; $= 1$ allows the effect. Default is no restrictions.
<code>dum</code>	Include seasonal dummies. Specify <code>'month'</code> for 11 monthly dummies or <code>'quarter'</code> for 3 quarterly dummies. Default is no dummies.
<code>exog</code>	$N_{\text{exo}} \times E$ matrix of exogenous variables entering without lags (non-dynamic regressors, e.g., event dummies). Must be provided for the full exogenous block sample size. Default is no exogenous variables.
<code>shock</code>	$(K_1 + K_2) \times 1$ vector specifying shock sizes for each variable. Default is one-standard-deviation shocks (diagonal of impact matrix P).
<code>confianza</code>	Lower confidence interval percentage. For example, <code>confianza = 5</code> produces 90% confidence intervals (5th and 95th percentiles). Default is 5.
<code>verbose</code>	If <code>true</code> , print Monte Carlo iteration numbers; if <code>false</code> , suppress output. Default is <code>false</code> .

13.4 Stored results

`bloque_exo` stores the following in structure `EQ`:

Matrices

<code>EQ.vari_shockj</code>	$(K_1 + K_2) \times (K_1 + K_2)$ collection of $3 \times \text{irf_length}$ matrices. For each variable i and shock j : first row = 95th percentile (or $100 - \text{confianza}$), second row = point estimate IRF, third row = 5th percentile (or <code>confianza</code>). Field names use <code>var_names</code> for variables and <code>shock_names</code> for shocks.
<code>EQ.P</code>	$(K_1 + K_2) \times (K_1 + K_2)$ block-triangular Cholesky matrix of the variance-covariance matrix. Upper-left $K_1 \times K_1$ block is the exogenous block identification (from full sample). Lower-right $K_2 \times K_2$ block is the endogenous block identification conditional on exogenous (from overlapping sample). Upper-right block is zeros. Lower-left block captures contemporaneous effects of exogenous shocks on endogenous variables.
<code>EQ.vardecshock_shockj</code>	$(K_1 + K_2)$ matrices of size $(K_1 + K_2) \times \text{irf_length}$. Each row shows the percentage of variance of that variable explained by shock j at each horizon. Field names use <code>shock_names</code> .
<code>EQ.stdirf</code>	$(K_1 + K_2) \times \text{irf_length} \times (K_1 + K_2)$ array containing standard deviations of IRFs from Monte Carlo simulation.
<code>EQ.struc</code>	$(K_1 + K_2) \times T_{\text{effective}}$ matrix of structural shocks for the overlapping sample period, where $T_{\text{effective}} = N_{\text{endo}} - \text{nlags}$.
<code>EQ.coefficients</code>	$(K_1 + K_2) \times ((K_1 + K_2) \times \text{nlags})$ matrix of VAR coefficients.
<code>EQ.constants</code>	$(K_1 + K_2) \times 1$ vector of constants (if <code>const=true</code>).
<code>EQ.exo</code>	$(K_1 + K_2) \times E$ matrix of coefficients on exogenous variables (if <code>exog</code> provided).
<code>EQ.dummies</code>	$(K_1 + K_2) \times \text{numdum}$ matrix of seasonal dummy coefficients (if <code>dum</code> provided).

13.5 Remarks

Block-exogenous structure: The model partitions variables into two blocks:

- **Exogenous block** (K_1 variables): Affect endogenous variables but are not affected by them. Identified using the full sample (N_{exo} observations).
- **Endogenous block** (K_2 variables): Affected by exogenous variables. Identified using the overlapping sample (N_{endo} observations, where $N_{\text{endo}} < N_{\text{exo}}$).

This structure is economically motivated and imposes that exogenous variables are predetermined with respect to endogenous variables.

Block-triangular identification: The impact matrix P has a block-triangular structure:

$$P = \begin{bmatrix} P_{\text{exo}} & 0 \\ P_{\text{link}} & P_{\text{endo}} \end{bmatrix}$$

where:

- P_{exo} ($K_1 \times K_1$): Upper-left block identifying exogenous shocks via Cholesky decomposition of the exogenous block's residual covariance (using full sample)
- P_{endo} ($K_2 \times K_2$): Lower-right block identifying endogenous shocks via Cholesky decomposition of residuals after removing exogenous shock effects
- P_{link} ($K_2 \times K_1$): Lower-left block capturing contemporaneous effects of exogenous shocks on endogenous variables
- Upper-right block is zero: Endogenous shocks do not affect exogenous variables

Sample alignment: The endogenous block data must correspond to the **last** N_{endo} observations of the exogenous block. For example:

- Exogenous block: 1990Q1–2024Q4 (140 quarters)
- Endogenous block: 2000Q1–2024Q4 (100 quarters)
- The overlapping period is 2000Q1–2024Q4

Efficiency gain: By using the full sample for the exogenous block, the method achieves more efficient identification of exogenous shocks compared to restricting to the overlapping sample. This is particularly valuable when:

- Exogenous variables have longer historical data (e.g., international financial data)
- Endogenous variables are only available for recent periods (e.g., newly collected domestic indicators)
- Maximum precision is desired for identifying exogenous shocks

Estimation algorithm:

1. Estimate reduced-form VAR for exogenous block using full sample (N_{exo} observations)
2. Compute Cholesky decomposition of exogenous block residual covariance: $P_{\text{exo}} = \text{chol}(\Sigma_{\text{exo}})'$
3. Recover structural shocks for exogenous block: $\varepsilon_{\text{exo}} = P_{\text{exo}}^{-1}u_{\text{exo}}$
4. Estimate reduced-form VAR for full system using overlapping sample (N_{endo} observations)
5. Regress endogenous block residuals on exogenous block structural shocks to obtain P_{link}
6. Compute residuals after removing exogenous shock effects
7. Compute Cholesky decomposition of these residuals: $P_{\text{endo}} = \text{chol}(\Sigma_{\text{endo}|\text{exo}})'$
8. Assemble block-triangular P matrix

Monte Carlo confidence intervals: The function uses Monte Carlo simulation to construct confidence intervals:

1. Generate artificial data from estimated VAR with block structure
2. For each Monte Carlo draw, re-estimate the block-exogenous SVAR
3. Compute IRFs for each draw
4. Report percentiles across Monte Carlo draws as confidence intervals

Variance decomposition: When computing forecast error variance decomposition, the block structure is preserved. Exogenous variables' variance is never attributed to endogenous shocks (by construction), but endogenous variables' variance can be decomposed into contributions from both exogenous and endogenous shocks.

Applications:

- **Small open economy models:** World variables (exogenous) $\rightarrow$ Domestic variables (endogenous). World economy affects small economy but not vice versa.
- **Sectoral analysis:** Aggregate economy (exogenous) $\rightarrow$ Sector-specific variables (endogenous). Aggregate affects sector but sector is too small to affect aggregate.
- **Regional analysis:** National variables (exogenous) $\rightarrow$ Regional variables (endogenous). National policies affect region but region is too small to affect nation.
- **Financial spillovers:** International financial markets (exogenous) $\rightarrow$ Domestic financial markets (endogenous). International markets affect domestic but not vice versa.

When to use block-exogenous SVAR: Use this approach when:

- Economic theory supports block-exogeneity (e.g., large vs small, aggregate vs specific)
- Data availability differs across variable groups
- Maximum efficiency is desired for exogenous block identification
- You want to impose economically motivated zero restrictions (endo $\nrightarrow$ exo)

Plots: The function automatically generates $(K_1 + K_2)$ figures, one for each structural shock, showing the responses of all $(K_1 + K_2)$ variables with confidence bands.

13.6 Examples

Example 1: Basic block-exogenous SVAR

Small open economy model with world variables (exogenous block, 250 obs) and domestic variables (endogenous block, 150 obs):

```
% World variables: full sample (250 observations)
% Domestic variables: shorter sample (150 observations)
var_names = {'WorldGDP', 'WorldInterest', 'DomesticGDP', 'DomesticInterest'};
shock_names = {'WorldSupply', 'WorldMonetary', ...
               'DomesticSupply', 'DomesticMonetary'};

EQ = bloque_exo(data_bloqueexo, data_bloqueendo, nlags, ...
               var_names, shock_names);

% Block-triangular impact matrix P:
% Upper-left 2x2: world shocks (identified with 250 obs)
% Lower-right 2x2: domestic shocks (identified with 150 obs)
% Upper-right: zeros (domestic shocks don't affect world)
% Lower-left: non-zeros (world shocks affect domestic)
disp(EQ.P);
```


Example 2: Examining the block structure

Extract and analyze the three components of the impact matrix:

```
% Extract blocks
P_exo = EQ.P(1:2, 1:2);          % Exogenous block identification
P_link = EQ.P(3:4, 1:2);        % Linkage from exog to endo
P_endo = EQ.P(3:4, 3:4);        % Endogenous block identification

% Exogenous block uses FULL sample (250 obs)
% More efficient than using only 150 overlapping observations
disp('Exogenous block:'); disp(P_exo);

% Linkage shows contemporaneous effects
disp('Exogenous → Endogenous linkage:'); disp(P_link);

% Endogenous block uses overlapping sample (150 obs)
disp('Endogenous block:'); disp(P_endo);
```

Example 3: Analyzing international spillovers

Examine how world shocks affect domestic economy:

```
% Spillover of world supply shock to domestic GDP
figure;
plot(1:40, EQ.DomesticGDP_WorldSupply(2,:), 'k-', 'LineWidth', 2);
hold on;
x_fill = [1:40, fliplr(1:40)];
y_fill = [EQ.DomesticGDP_WorldSupply(1,:), ...
          fliplr(EQ.DomesticGDP_WorldSupply(3,:))];
fill(x_fill, y_fill, [0.8 0.8 0.8], 'EdgeColor', 'none');
plot(1:40, zeros(1,40), 'r--');
title('Domestic GDP Response to World Supply Shock');
xlabel('Horizon'); ylabel('Response');

% Variance decomposition
fprintf('Domestic GDP variance explained by world shocks at h=20:\n');
fprintf('  World Supply: %.1f%%\n', EQ.vardecshock_WorldSupply(3,20));
fprintf('  World Monetary: %.1f%%\n', EQ.vardecshock_WorldMonetary(3,20));
```

Example 4: Structural shocks

Recover structural shocks for the overlapping period:

```
% EQ.struc contains structural shocks
% Size: (K1+K2) x (Nendo - nlags)
% For 150 observations and 2 lags: 4 x 148

figure;
for i = 1:4
    subplot(4, 1, i);
    plot(EQ.struc(i,:), 'k-');
    title([shock_names{i} ' (structural shock)']);
end

% These shocks can be used for:
% - Historical decomposition
% - Counterfactual analysis
% - Correlation with other events
```

Example 5: Custom shock sizes and confidence levels

Explore different shock sizes and confidence intervals:

```
% 2 standard deviation shocks
shock_size = 2 * diag(EQ.P);

% Use same random seed for fair comparison
rng(12345);
EQ1 = bloque_exo(data_bloqueexo, data_bloqueendo, nlags, ...
    var_names, shock_names, 40, 1000, true);

rng(12345);
EQ2 = bloque_exo(data_bloqueexo, data_bloqueendo, nlags, ...
    var_names, shock_names, 40, 1000, true, ...
    [], [], [], [], shock_size);

% Responses scale proportionally
ratio = EQ2.DomesticGDP_WorldSupply(2,10) / ...
    EQ1.DomesticGDP_WorldSupply(2,10);
fprintf('Ratio at h=10: %.2f (expected: ~2.0)\n', ratio);

% Different confidence levels
EQ_90 = bloque_exo(..., 5); % 90% CI
EQ_68 = bloque_exo(..., 16); % 68% CI
```

For complete, executable examples including variance decomposition, seasonal dummies, exogenous variables, and spillover analysis, see `example_bloque_exo.m` in the package repository.

13.7 Methods and formulas

The structural VAR model with block-exogenous structure is:

$$\begin{bmatrix} y_{1,t} \\ y_{2,t} \end{bmatrix} = \begin{bmatrix} \nu_1 \\ \nu_2 \end{bmatrix} + \begin{bmatrix} A_{11,1} & 0 \\ A_{21,1} & A_{22,1} \end{bmatrix} \begin{bmatrix} y_{1,t-1} \\ y_{2,t-1} \end{bmatrix} + \cdots + \begin{bmatrix} B_{11} & 0 \\ B_{21} & B_{22} \end{bmatrix} \begin{bmatrix} \varepsilon_{1,t} \\ \varepsilon_{2,t} \end{bmatrix}$$

where:

- $y_{1,t}$: $K_1 \times 1$ vector of exogenous block variables
- $y_{2,t}$: $K_2 \times 1$ vector of endogenous block variables
- $A_{11,p}$: Coefficients of exogenous on exogenous
- $A_{21,p}$: Coefficients of exogenous on endogenous
- $A_{22,p}$: Coefficients of endogenous on endogenous
- B_{11} : Exogenous block impact matrix ($K_1 \times K_1$)
- B_{21} : Linkage matrix ($K_2 \times K_1$)
- B_{22} : Endogenous block impact matrix ($K_2 \times K_2$)

Reduced form:

$$y_t = \nu + A_1 y_{t-1} + \cdots + A_p y_{t-p} + u_t$$

with $u_t = B\varepsilon_t$ and $\mathbb{E}[u_t u_t'] = \Sigma = BB'$.

Block-triangular impact matrix:

$$P = B = \begin{bmatrix} P_{\text{exo}} & 0 \\ P_{\text{link}} & P_{\text{endo}} \end{bmatrix}$$

Exogenous block identification (full sample):

Estimate reduced-form VAR for exogenous block using observations $t = 1, \dots, N_{\text{exo}}$:

$$y_{1,t} = \nu_1 + A_{11,1}y_{1,t-1} + \dots + A_{11,p}y_{1,t-p} + u_{1,t}$$

Compute residual covariance:

$$\Sigma_{11} = \frac{1}{N_{\text{exo}} - np - 1} \sum_{t=np+1}^{N_{\text{exo}}} u_{1,t} u'_{1,t}$$

Cholesky decomposition:

$$P_{\text{exo}} = \text{chol}(\Sigma_{11})'$$

Recover structural shocks:

$$\varepsilon_{1,t} = P_{\text{exo}}^{-1} u_{1,t}$$

Full system estimation (overlapping sample):

Estimate full system using observations $t = N_{\text{exo}} - N_{\text{endo}} + 1, \dots, N_{\text{exo}}$:

$$\begin{bmatrix} y_{1,t} \\ y_{2,t} \end{bmatrix} = \begin{bmatrix} \nu_1 \\ \nu_2 \end{bmatrix} + A_1 \begin{bmatrix} y_{1,t-1} \\ y_{2,t-1} \end{bmatrix} + \dots + \begin{bmatrix} u_{1,t} \\ u_{2,t} \end{bmatrix}$$

Endogenous block identification:

Regress endogenous block residuals on exogenous structural shocks:

$$u_{2,t} = P_{\text{link}} \varepsilon_{1,t} + \eta_t$$

where t runs over the overlapping sample.

OLS estimation:

$$P_{\text{link}} = \left(\sum_t u_{2,t} \varepsilon'_{1,t} \right) \left(\sum_t \varepsilon_{1,t} \varepsilon'_{1,t} \right)^{-1}$$

Residual covariance after removing exogenous effects:

$$\Sigma_{22|1} = \frac{1}{N_{\text{endo}} - np - 1} \sum_t \eta_t \eta'_t$$

Cholesky decomposition:

$$P_{\text{endo}} = \text{chol}(\Sigma_{22|1})'$$

Impulse responses:

Using companion form:

$$\Phi_h = J A^h J' B$$

where $J = [I_K : 0_{K \times K(p-1)}]$ and $B = P$.

Variance decomposition:

Contribution of shock j to variance of variable i at horizon h :

$$\omega_{ij}(h) = \frac{\sum_{s=0}^{h-1} [\Phi_s e_j]_i^2}{\sum_{s=0}^{h-1} \Phi_s \Phi'_{s ii}} \times 100\%$$

Due to block structure: $\omega_{i,j}(h) = 0$ for all h when $i \leq K_1$ and $j > K_1$ (endogenous shocks do not affect exogenous variables).

Monte Carlo confidence intervals:

For $m = 1, \dots, M$ (where $M = \text{monte_carlo}$):

1. Generate artificial data from estimated VAR with block structure
2. Re-estimate exogenous block VAR and compute $P_{\text{exo}}^{(m)}$
3. Re-estimate full system and compute $P_{\text{link}}^{(m)}, P_{\text{endo}}^{(m)}$
4. Compute IRFs: $\Phi_h^{(m)}$

Confidence bands are percentiles of $\{\Phi_h^{(1)}, \dots, \Phi_h^{(M)}\}$.

13.8 References

Cushman, D. O., and T. Zha. 1997. "Identifying Monetary Policy in a Small Open Economy under Flexible Exchange Rates." *Journal of Monetary Economics* 39(3): 433–448.

Kim, S., and N. Roubini. 2000. "Exchange Rate Anomalies in the Industrial Countries: A Solution with a Structural VAR Approach." *Journal of Monetary Economics* 45(3): 561–586.

Zha, T. 1999. "Block Recursion and Structural Vector Autoregressions." *Journal of Econometrics* 90(2): 291–316.

Lütkepohl, H. 2005. *New Introduction to Multiple Time Series Analysis*. Berlin: Springer-Verlag.

Dungey, M., and A. Pagan. 2000. "A Structural VAR Model of the Australian Economy." *Economic Record* 76(235): 321–342.

14 ivSVAR_blockbootstrap — Proxy-SVAR with block bootstrap

14.1 Syntax

```
EQ = ivSVAR_blockbootstrap(data_inst, other_data, iv, nlags, var_names, shock_names)
```

```
EQ = ivSVAR_blockbootstrap(data_inst, other_data, iv, nlags, var_names, shock_names, ...
                           irf_lenght, bootstrap_num, const, lr, dum, exog, shock, ...
                           confianza, rel, verbose)
```

14.2 Description

ivSVAR_blockbootstrap estimates a structural vector autoregression (SVAR) using **external instruments** (proxy variables) to identify structural shocks. This approach achieves **partial identification**: only K_1 structural shocks are identified (those with valid instruments), leaving the remaining $K - K_1$ shocks unidentified.

The method follows Mertens and Ravn (2013) and Stock and Watson (2012, 2018). Confidence intervals are constructed using **block bootstrap**, which preserves the autocorrelation structure in the data by resampling consecutive blocks of residuals and instruments together.

Key features:

- **Partial identification**: Identifies $K_1 \leq K$ structural shocks using K_1 external instruments
- **No timing assumptions**: Unlike Cholesky decomposition, no recursive ordering needed
- **Block bootstrap**: Preserves autocorrelation when constructing confidence intervals
- **Instrument diagnostics**: Reports robust F-statistics for instrument strength
- **Flexible normalization**: Absolute (unit structural shock) or relative (unit variable response) IRFs

Common applications include:

- Oil price shocks (using oil supply news as instrument)
- Monetary policy shocks (using high-frequency futures surprises)
- Government spending shocks (using military build-up events)
- Tax policy shocks (using narrative tax changes)

14.3 Options

Option	Description
<code>data_inst</code>	$N \times K_1$ matrix of instrumented variables (N observations, K_1 variables). These are the variables for which structural shocks are identified via external instruments. Required.
<code>other_data</code>	$N \times K_2$ matrix of non-instrumented variables (N observations, K_2 variables). Total system size is $K = K_1 + K_2$. Required.
<code>iv</code>	$N \times K_1$ matrix of external instruments (proxy variables). Must have same number of columns as <code>data_inst</code> . Instruments should be correlated with the structural shocks of interest but orthogonal to other shocks. Required.
<code>nlags</code>	Number of lags to include in the VAR. Required.
<code>var_names</code>	$1 \times K$ cell vector with variable names (instrumented variables first, then non-instrumented). Default is <code>{'var1', 'var2', ...}</code> .
<code>shock_names</code>	$1 \times K_1$ cell vector with names of identified structural shocks. Default is <code>{'var1', 'var2', ...}</code> .
<code>irf_lenght</code>	Length of impulse response functions. Default is 40.

<code>bootstrap_num</code>	Number of block bootstrap repetitions for confidence intervals. Default is 1000.
<code>const</code>	Include constant term if <code>true</code> ; exclude if <code>false</code> . Default is <code>false</code> .
<code>lr</code>	$K \times K$ matrix of zeros and ones specifying restrictions on the reduced-form VAR. Element $(i, j) = 0$ restricts variable j from affecting variable i ; $= 1$ allows the effect. Default is no restrictions.
<code>dum</code>	Include seasonal dummies. Specify <code>'month'</code> for 11 monthly dummies or <code>'quarter'</code> for 3 quarterly dummies. Default is no dummies.
<code>exog</code>	$N \times E$ matrix of exogenous variables entering without lags (non-dynamic regressors, e.g., event dummies). Default is no exogenous variables.
<code>shock</code>	$K_1 \times 1$ vector specifying shock sizes for each identified shock. Default is unit shocks (appropriate for proxy-SVARs).
<code>confianza</code>	Lower confidence interval percentage. For example, <code>confianza = 5</code> produces 90% confidence intervals (5th and 95th percentiles). Default is 5.
<code>rel</code>	If <code>rel = 0</code> (default): absolute IRFs (unit structural shock, Mertens & Ravn normalization). If <code>rel = 1</code> : relative IRFs (instrumented variable moves by 1 unit at impact). Use <code>rel = 1</code> when non-invertibility is a concern. Default is 0.
<code>verbose</code>	If <code>true</code> , print bootstrap iteration numbers; if <code>false</code> , suppress output. Default is <code>false</code> .

14.4 Stored results

`ivSVAR_blockbootstrap` stores the following in structure `EQ`:

Matrices

<code>EQ.vari_shockj</code>	$K \times K_1$ collection of $3 \times \text{irf_length}$ matrices. For each variable i and identified shock j : first row = 95th percentile (or $100 - \text{confianza}$), second row = IRF, third row = 5th percentile (or <code>confianza</code>). Field names use <code>var_names</code> for variables and <code>shock_names</code> for shocks.
<code>EQ.P</code>	$K \times K_1$ partially-identified impact matrix. Only the first K_1 columns (shocks with instruments) are identified.
<code>EQ.vardecshock_shockj</code>	K_1 matrices of size $K \times \text{irf_length}$. Each row shows the percentage of variance of that variable explained by identified shock j at each horizon. Field names use <code>shock_names</code> . Note: this is a partial variance decomposition (does not sum to 100%).
<code>EQ.stdirf</code>	$K \times \text{irf_length} \times K_1$ array containing standard deviations of IRFs from block bootstrap simulation.
<code>EQ.struc</code>	$K_1 \times (N - \text{nlags})$ matrix of identified structural shocks.
<code>EQ.irf_sim</code>	$K \times \text{irf_length} \times \text{bootstrap_num} \times K_1$ array of all IRF draws from block bootstrap simulation.
<code>EQ.robustF</code>	$K_1 \times 1$ vector of robust F-statistics for first-stage instrument relevance. Rule of thumb: $F > 10$ indicates strong instruments.

14.5 Remarks

Proxy-SVAR identification: Unlike recursive (Cholesky) or long-run restrictions that attempt to identify all K structural shocks, proxy-SVARs achieve **partial identification**. Only K_1 shocks (those with valid external instruments) are identified. The impact matrix is:

$$P = [\beta_1, \beta_2, \dots, \beta_{K_1}, 0, \dots, 0] \in \mathbb{R}^{K \times K}$$

where the first K_1 columns β_j are identified via instruments, and the remaining $K - K_1$ columns are zeros (unidentified shocks).

This partial identification is a feature, not a limitation. It allows researchers to:

- Focus on specific shocks of economic interest
- Avoid arbitrary restrictions on uninteresting shocks
- Use economically motivated instruments rather than statistical assumptions

External instruments (proxies): An external instrument z_t for structural shock ε_{jt} must satisfy:

1. **Relevance:** $\mathbb{E}[z_t \varepsilon_{jt}] \neq 0$ (instrument correlated with shock)
2. **Exogeneity:** $\mathbb{E}[z_t \varepsilon_{kt}] = 0$ for all $k \neq j$ (orthogonal to other shocks)

Examples of valid instruments:

- **Oil supply shocks:** Oil production disruptions due to political events (Kilian, 2008)
- **Monetary policy shocks:** High-frequency changes in futures prices around FOMC announcements (Gertler & Karadi, 2015)
- **Government spending shocks:** Military build-ups predicted by political events (Ramey & Zubairy, 2018)
- **Tax shocks:** Legislated tax changes for reasons unrelated to current economic conditions (Romer & Romer, 2010)

The key challenge is finding instruments that satisfy both relevance and exogeneity. Relevance can be tested statistically (robust F-statistic), but exogeneity relies on economic reasoning and institutional knowledge.

Instrument strength (robust F-statistic): The function reports `EQ.robustF`, the robust F-statistic from the first-stage regression:

$$u_{jt} = \pi' z_t + \eta_t$$

where u_{jt} is the reduced-form residual for instrumented variable j and z_t is the instrument.

Rule of thumb (Stock & Watson, 2018):

- $F > 10$: Strong instrument (safe to proceed)
- $F < 10$: Weak instrument (biased and imprecise estimates)

With weak instruments:

- Point estimates are biased toward zero
- Confidence intervals are too narrow (overstate precision)
- Inference is unreliable

Always report the robust F-statistic when using proxy-SVARs. If $F < 10$, consider finding a stronger instrument or acknowledging weak identification.

Block bootstrap: Confidence intervals are constructed using block bootstrap (Jentsch & Lunsford, 2019), which preserves the autocorrelation structure in residuals and instruments:

1. Compute optimal block length: $\ell = \lceil T_{\text{eff}}^{1/4} \rceil$ where $T_{\text{eff}} = N - \text{nlags}$
2. Divide residuals $\{u_t\}$ and instruments $\{z_t\}$ into overlapping blocks of length ℓ
3. Randomly draw $\lceil T_{\text{eff}}/\ell \rceil$ blocks with replacement
4. Concatenate blocks and demean to obtain bootstrap sample
5. Estimate VAR on bootstrap data, re-identify structural shocks, compute IRFs
6. Repeat for `bootstrap_num` iterations

7. Confidence bands are percentiles across bootstrap draws

Block bootstrap is appropriate when:

- Residuals exhibit autocorrelation (typical in macroeconomic data)
- Sample size is reasonably large ($N \geq 100$)
- Preserving temporal dependence is important for valid inference

The automatic block length selection $\ell = T^{1/4}$ balances bias (too short blocks) and variance (too long blocks).

Absolute vs relative IRF normalization: The `rel` parameter controls IRF normalization:

Absolute IRFs (`rel = 0`, default):

- Shock size is one unit of the structural shock ε_j
- Standard normalization in Mertens & Ravn (2013)
- Impact on instrumented variable j is β_{jj} (not necessarily 1)
- Appropriate when the structural shock has a clear economic interpretation

Relative IRFs (`rel = 1`):

- Shock size is chosen so instrumented variable j moves by 1 unit at impact
- Normalization: $\text{shock} = 1/\beta_{jj}$
- All IRFs scaled proportionally
- Useful when non-invertibility is a concern or when you prefer the instrumented variable to have a unit response

Example: If oil price is the instrumented variable:

- `rel = 0`: "Response to a one-unit oil supply shock"
- `rel = 1`: "Response when oil price increases by 1 unit"

The choice depends on economic interpretation. Both normalizations contain the same information, just scaled differently.

Partial variance decomposition: Variance decomposition with proxy-SVARs is **partial**. Only the K_1 identified shocks contribute, so the shares do not sum to 100%:

$$\sum_{j=1}^{K_1} \omega_{ij}(h) < 100\%$$

The remaining variance is attributed to the $K - K_1$ unidentified shocks. When interpreting `EQ.vardecshock_shockj`, remember:

- Values show the fraction of variance explained by identified shock j alone
- Do not expect contributions to sum to 100%
- Focus on relative importance across identified shocks
- Missing variance is due to unidentified shocks

For example, if oil shock explains 30% of output variance at a 10-quarter horizon, the other 70% comes from non-oil shocks (demand, productivity, monetary policy, etc.), which are not identified in the proxy-SVAR.

When to use proxy-SVARs: Use this approach when:

- You have valid external instruments for shocks of interest
- You want to avoid arbitrary timing or long-run restrictions
- Partial identification (identifying some shocks) is sufficient
- Economic theory suggests specific instruments

Do not use proxy-SVARs when:

- No valid instruments are available
- You need to identify all K structural shocks simultaneously
- Instruments are weak (robust $F < 10$)

Proxy-SVARs complement other methods:

- More credible than recursive timing assumptions
- Less restrictive than long-run restrictions
- More targeted than sign restrictions (identify specific shocks)

Plots: The function automatically generates K_1 figures, one for each identified structural shock, showing the responses of all K variables with confidence bands.

14.6 Examples

Example 1: Basic proxy-SVAR with oil supply shock

Identify oil supply shock using external instrument (oil production disruptions):

```
% data_inst: oil price (K1=1)
% other_data: output, inflation (K2=2)
% iv: oil supply news proxy (external instrument)

var_names = {'OilPrice', 'Output', 'Inflation'};
shock_names = {'OilShock'};

EQ = ivSVAR_blockbootstrap(data_inst, other_data, iv, nlags, ...
                           var_names, shock_names);

% Check instrument strength
fprintf('Robust F-statistic: %.2f\n', EQ.robustF);
if EQ.robustF > 10
    disp('Strong instrument (F > 10)');
end

% Partially-identified impact matrix (only first column identified)
disp(EQ.P(:,1));
```

Example 2: Absolute vs relative IRF normalization

Compare two normalizations for oil price shock:

```
% Absolute: unit structural shock (default)
EQ_abs = ivSVAR_blockbootstrap(data_inst, other_data, iv, nlags, ...
                               var_names, shock_names);

% Relative: oil price moves by 1 unit at impact
EQ_rel = ivSVAR_blockbootstrap(data_inst, other_data, iv, nlags, ...
```

```

                                var_names, shock_names, 40, 1000, ...
                                true, [], [], [], [], 5, 1);

% Oil price impact response
fprintf('Absolute (rel=0): %.4f\n', EQ_abs.OilPrice_OilShock(2,1));
fprintf('Relative (rel=1): %.4f (normalized to 1.0)\n', ...
        EQ_rel.OilPrice_OilShock(2,1));

% Output responses differ proportionally
fprintf('Output (absolute): %.4f\n', EQ_abs.Output_OilShock(2,10));
fprintf('Output (relative): %.4f\n', EQ_rel.Output_OilShock(2,10));

```

Example 3: Partial variance decomposition

Analyze variance explained by identified oil shock:

```

EQ = ivSVAR_blockbootstrap(data_inst, other_data, iv, nlags, ...
                            var_names, shock_names);

% Variance decomposition at 20-quarter horizon
fprintf('Output variance explained by:\n');
fprintf('  Oil shock:      %.1f%%\n', EQ.vardecshock_OilShock(2,20));
fprintf('  Other shocks:  %.1f%% (unidentified)\n', ...
        100 - EQ.vardecshock_OilShock(2,20));

% Plot partial variance decomposition
figure;
plot(1:40, EQ.vardecshock_OilShock(2,:), 'k-', 'LineWidth', 2);
title('Output: Variance Explained by Oil Shock');
xlabel('Horizon'); ylabel('Percentage');
ylim([0 100]);

```

Example 4: Custom shock sizes

Scale shock to represent \$10 oil price increase:

```

% Unit shock
EQ1 = ivSVAR_blockbootstrap(data_inst, other_data, iv, nlags, ...
                            var_names, shock_names);

% 10-unit shock
EQ10 = ivSVAR_blockbootstrap(data_inst, other_data, iv, nlags, ...
                             var_names, shock_names, 40, 1000, ...
                             true, [], [], [], 10);

% Responses scale proportionally
fprintf('Output response (unit shock):  %.4f\n', ...
        EQ1.Output_OilShock(2,10));
fprintf('Output response (10x shock):  %.4f\n', ...
        EQ10.Output_OilShock(2,10));
fprintf('Ratio: %.2f (expected ~10.0)\n', ...
        EQ10.Output_OilShock(2,10) / EQ1.Output_OilShock(2,10));

```

Example 5: Block bootstrap monitoring

Monitor bootstrap iterations with verbose option:

```

% With verbose=true, prints iteration numbers

```

```

EQ = ivSVAR_blockbootstrap(data_inst, other_data, iv, nlags, ...
    var_names, shock_names, 40, 1000, ...
    true, [], [], [], [], 5, 0, true);

% Optimal block length for this sample
T_eff = size(data_inst, 1) - nlags;
l_opt = ceil(T_eff^(1/4));
fprintf('Effective sample: %d observations\n', T_eff);
fprintf('Optimal block length: %d\n', l_opt);
fprintf('Blocks per bootstrap draw: ~%d\n', ceil(T_eff/l_opt));

```

For complete, executable examples including instrument diagnostics, seasonal dummies, exogenous variables, restricted VARs, and IRF visualization, see `example_ivSVAR_blockbootstrap.m` in the package repository.

14.7 Methods and formulas

The reduced-form VAR is:

$$y_t = \nu + A_1 y_{t-1} + \dots + A_p y_{t-p} + u_t$$

with $\mathbb{E}[u_t u_t'] = \Sigma$.

Partition variables: $y_t = [y_{1t}', y_{2t}']'$ where y_{1t} ($K_1 \times 1$) are instrumented variables and y_{2t} ($K_2 \times 1$) are non-instrumented.

Identification via external instruments:

For instrumented variable j with instrument z_{jt} :

First stage:

$$u_{jt} = \pi_j' z_{jt} + \eta_{jt}$$

Robust F-statistic tests $H_0 : \pi_j = 0$ using heteroskedasticity-robust standard errors.

Second stage: For non-instrumented variables, regress residuals on predicted first-stage values:

$$u_{kt} = \lambda_k' \hat{u}_{1t} + \xi_{kt}, \quad k = K_1 + 1, \dots, K$$

where $\hat{u}_{1t}$ are predicted values from first stage.

Structural impact matrix (Mertens & Ravn, 2013):

The partially-identified impact matrix has the form:

$$P = \begin{bmatrix} \beta_{11} & 0 & \cdots \\ \beta_{21} & 0 & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix} \in \mathbb{R}^{K \times K}$$

For the first column (corresponding to the identified shock):

$$\beta_1 = \begin{bmatrix} \beta_{11} \\ \beta_{21} \end{bmatrix}$$

where:

- β_{11} ($K_1 \times 1$): impact on instrumented variables
- β_{21} ($K_2 \times 1$): impact on non-instrumented variables

These are computed from variance-covariance decomposition:

$$\begin{aligned} \Sigma_{11} &= \beta_{11} \beta_{11}' + \beta_{12} \beta_{12}' \\ \Sigma_{21} &= \beta_{21} \beta_{11}' + \lambda \Sigma_{11} \\ \Sigma_{22} &= \beta_{21} \beta_{21}' + \beta_{22} \beta_{22}' + \lambda (\beta_{12} \beta_{12}') \lambda' \end{aligned}$$

where λ is from the second-stage regression.

Solving this system yields:

$$\begin{aligned}\beta_{11} &= (I - \beta_{12}\Sigma_{22}^{-1}\lambda)^{-1}S_1 \\ \beta_{21} &= \lambda(I - \beta_{12}\Sigma_{22}^{-1}\lambda)^{-1}S_1\end{aligned}$$

where $S_1S_1' = \Sigma_{11} - \beta_{12}\beta_{12}'$ is obtained via Cholesky decomposition.

Impulse response functions:

Using companion form:

$$\Phi_h = JA^hJ'P$$

where $J = [I_K : 0_{K \times K(p-1)}]$, A is the companion matrix, and P is the partially-identified impact matrix.

For shock j ($j = 1, \dots, K_1$):

- Absolute IRF (**rel** = 0): $\text{IRF}_j(h) = \Phi_h e_j$
- Relative IRF (**rel** = 1): $\text{IRF}_j(h) = \Phi_h e_j / \beta_{jj}$

where e_j is a selection vector.

Structural shocks (Lunsford, 2015):

The identified structural shocks are recovered as:

$$\varepsilon_t = (\Sigma^{-1}u_t'Z) \left((Z'u_t)\Sigma^{-1}(u_t'Z) \right)^{-1/2}$$

where $Z = [z_{1t}, \dots, z_{K_1,t}]$ are the instruments and u_t are reduced-form residuals.

These shocks satisfy:

- $\mathbb{E}[\varepsilon_t \varepsilon_t'] = I_{K_1}$ (normalized)
- $\mathbb{E}[z_{jt} \varepsilon_{jt}] > 0$ (positive correlation with instruments)
- $\mathbb{E}[z_{jt} \varepsilon_{kt}] = 0$ for $j \neq k$ (orthogonal to other instruments)

Partial variance decomposition:

Contribution of identified shock j to variance of variable i at horizon h :

$$\omega_{ij}(h) = \frac{\sum_{s=0}^{h-1} [\Phi_s e_j]_i^2}{\sum_{s=0}^{h-1} [\Phi_s \Sigma \Phi_s']_{ii}} \times 100\%$$

Note:

$$\sum_{j=1}^{K_1} \omega_{ij}(h) < 100\%$$

because only $K_1 < K$ shocks are identified.

Block bootstrap algorithm:

For $m = 1, \dots, M$ (where $M = \text{bootstrap_num}$):

1. Compute block length: $\ell = \lceil T_{\text{eff}}^{1/4} \rceil$
2. Form overlapping blocks: $\{(u_{t:t+\ell-1}, z_{t:t+\ell-1})\}_{t=1}^{T_{\text{eff}}-\ell+1}$
3. Randomly draw $\lceil T_{\text{eff}}/\ell \rceil$ blocks with replacement
4. Concatenate to form $(u^{(m)}, z^{(m)})$ of length T_{eff}
5. Demean: $\tilde{u}^{(m)} = u^{(m)} - \bar{u}^{(m)}$, $\tilde{z}^{(m)} = z^{(m)} - \bar{z}^{(m)}$
6. Generate bootstrap data: $y_t^{(m)} = \hat{\nu} + \sum_{i=1}^p \hat{A}_i y_{t-i}^{(m)} + \tilde{u}_t^{(m)}$
7. Re-estimate VAR and identify shocks using $\tilde{z}^{(m)}$ as bootstrap instruments
8. Compute IRFs: $\Phi_h^{(m)}$

Confidence bands are percentiles of $\{\Phi_h^{(1)}, \dots, \Phi_h^{(M)}\}$.

14.8 References

- Mertens, K., and M. O. Ravn. 2013. "The Dynamic Effects of Personal and Corporate Income Tax Changes in the United States." *American Economic Review* 103(4): 1212–1247.
- Stock, J. H., and M. W. Watson. 2012. "Disentangling the Channels of the 2007–2009 Recession." *Brookings Papers on Economic Activity* Spring: 81–135.
- Stock, J. H., and M. W. Watson. 2018. "Identification and Estimation of Dynamic Causal Effects in Macroeconomics Using External Instruments." *Economic Journal* 128(610): 917–948.
- Jentsch, C., and K. G. Lunsford. 2019. "The Dynamic Effects of Personal and Corporate Income Tax Changes in the United States: Comment." *American Economic Review* 109(7): 2655–2678.
- Lunsford, K. G. 2015. "Identifying Structural VARs with a Proxy Variable and a Test for a Weak Proxy." Federal Reserve Bank of Cleveland Working Paper 15-28.
- Gertler, M., and P. Karadi. 2015. "Monetary Policy Surprises, Credit Costs, and Economic Activity." *American Economic Journal: Macroeconomics* 7(1): 44–76.
- Kilian, L. 2008. "Exogenous Oil Supply Shocks: How Big Are They and How Much Do They Matter for the U.S. Economy?" *Review of Economics and Statistics* 90(2): 216–240.

15 ivSVAR_wildbootstrap — Proxy-SVAR with wild bootstrap

15.1 Syntax

```
EQ = ivSVAR_wildbootstrap(data_inst, other_data, iv, nlags, var_names, shock_names)
```

```
EQ = ivSVAR_wildbootstrap(data_inst, other_data, iv, nlags, var_names, shock_names, ...  
                           irf_lenght, bootstrap_num, const, lr, dum, exog, shock, ...  
                           confianza, rel, verbose)
```

15.2 Description

ivSVAR_wildbootstrap estimates a structural vector autoregression (SVAR) using **external instruments** (proxy variables) to identify structural shocks, with confidence intervals constructed via **wild bootstrap**. Like **ivSVAR_blockbootstrap**, this achieves partial identification of K_1 structural shocks.

The method follows Mertens and Ravn (2013) and Stock and Watson (2012, 2018) for identification. Wild bootstrap uses the **Rademacher distribution** (multiplying residuals by $+1$ or -1 with equal probability), which does not require block length selection and is simpler to implement than block bootstrap.

Key features:

- **Partial identification:** Identifies $K_1 \leq K$ structural shocks using K_1 external instruments
- **Wild bootstrap:** Resamples residual signs rather than blocks, simpler than block bootstrap
- **No parameter tuning:** Unlike block bootstrap, no block length selection required
- **Rademacher distribution:** Each residual multiplied by $+1$ or -1 independently
- **Same identification:** Results equivalent to **ivSVAR_blockbootstrap** in expectation

15.3 Options

The options are identical to **ivSVAR_blockbootstrap**. See that section for complete parameter descriptions. The only difference is the bootstrap method used to construct confidence intervals.

15.4 Stored results

Stored results are identical to **ivSVAR_blockbootstrap**. See that section for complete details.

15.5 Remarks

Wild bootstrap: Confidence intervals are constructed using wild bootstrap with the Rademacher distribution (Gonçalves & Kilian, 2004):

1. Estimate VAR and obtain residuals $\{u_t\}_{t=1}^{T_{\text{eff}}}$ and instruments $\{z_t\}_{t=1}^{T_{\text{eff}}}$
2. For each bootstrap iteration m :
 - Draw T_{eff} independent Rademacher random variables: $\omega_t^{(m)} \in \{-1, +1\}$ with $P(\omega_t = +1) = P(\omega_t = -1) = 0.5$
 - Multiply residuals: $u_t^{(m)} = \omega_t^{(m)} u_t$
 - Multiply instruments: $z_t^{(m)} = \omega_t^{(m)} z_t$
 - Generate bootstrap data: $y_t^{(m)} = \hat{\nu} + \sum_{i=1}^p \hat{A}_i y_{t-i}^{(m)} + u_t^{(m)}$
 - Re-estimate VAR and identify shocks using $z^{(m)}$ as bootstrap instruments
 - Compute IRFs: $\Phi_h^{(m)}$
3. Confidence bands are percentiles of $\{\Phi_h^{(1)}, \dots, \Phi_h^{(M)}\}$

Wild bootstrap vs block bootstrap: The choice between wild and block bootstrap depends on practical considerations:

Wild bootstrap (`ivSVAR.wildbootstrap`):

- **Simpler:** No block length selection required
- **Faster:** Less computationally intensive
- **Short samples:** Works well even with small T
- **Implementation:** Each residual resampled independently (sign flip)
- **Assumption:** Residuals have constant conditional mean but may have time-varying variance

Block bootstrap (`ivSVAR.blockbootstrap`):

- **Preserves autocorrelation:** Consecutive residuals kept together in blocks
- **More complex:** Requires optimal block length selection ($\ell = T^{1/4}$)
- **Large samples:** Performs better with $T \geq 100$
- **Implementation:** Resamples contiguous blocks of residuals
- **Assumption:** Temporal dependence structure should be preserved

Practical guidance:

- Both methods produce similar results in most applications
- Wild bootstrap is simpler and requires no tuning
- Block bootstrap may be preferred if preserving autocorrelation is critical
- Results should be robust to choice of bootstrap method
- If results differ substantially, investigate model specification

The choice is often a matter of researcher preference. Many applied papers use wild bootstrap for simplicity. Some use both as a robustness check.

Other remarks: All other remarks from `ivSVAR.blockbootstrap` apply:

- Proxy-SVAR identification (partial identification of K_1 shocks)
- External instruments (relevance and exogeneity conditions)
- Instrument strength diagnostics (robust F-statistic)
- Absolute vs relative IRF normalization (`rel` parameter)
- Partial variance decomposition (does not sum to 100%)

See `ivSVAR.blockbootstrap` documentation for details on these topics.

15.6 Examples

Example 1: Basic proxy-SVAR with wild bootstrap

Same identification as block bootstrap, different confidence intervals:

```

var_names = {'OilPrice', 'Output', 'Inflation'};
shock_names = {'OilShock'};

% Wild bootstrap
EQ = ivSVAR_wildbootstrap(data_inst, other_data, iv, nlags, ...
    var_names, shock_names);

% Point estimates identical to block bootstrap
% Confidence intervals may differ slightly
fprintf('Output response at h=10: %.4f\n', EQ.Output_OilShock(2,10));
fprintf('90%% CI: [%.4f, %.4f]\n', ...
    EQ.Output_OilShock(3,10), EQ.Output_OilShock(1,10));

```

Example 2: Comparing wild and block bootstrap

Check robustness of confidence intervals across bootstrap methods:

```

% Wild bootstrap
EQ_wild = ivSVAR_wildbootstrap(data_inst, other_data, iv, nlags, ...
    var_names, shock_names, 40, 1000, true);

% Block bootstrap
EQ_block = ivSVAR_blockbootstrap(data_inst, other_data, iv, nlags, ...
    var_names, shock_names, 40, 1000, true);

% Compare confidence intervals at h=10
fprintf('Output response to Oil Shock (h=10):\n');
fprintf(' Point estimate: %.4f (identical for both methods)\n', ...
    EQ_wild.Output_OilShock(2,10));
fprintf('\n');
fprintf('Wild bootstrap 90%% CI: [%.4f, %.4f]\n', ...
    EQ_wild.Output_OilShock(3,10), EQ_wild.Output_OilShock(1,10));
fprintf('Block bootstrap 90%% CI: [%.4f, %.4f]\n', ...
    EQ_block.Output_OilShock(3,10), EQ_block.Output_OilShock(1,10));

% Results should be similar (if not, investigate model specification)

```

Example 3: Wild bootstrap with verbose monitoring

Monitor bootstrap iterations:

```

% With verbose=true, prints iteration numbers
EQ = ivSVAR_wildbootstrap(data_inst, other_data, iv, nlags, ...
    var_names, shock_names, 40, 1000, ...
    true, [], [], [], [], 5, 0, true);

% Wild bootstrap is typically faster than block bootstrap
% No block length computation needed

```

For complete, executable examples including all features (instrument diagnostics, absolute vs relative IRFs, custom shocks, seasonal dummies, exogenous variables, partial variance decomposition), see `example_ivSVAR_wildbootstrap` in the package repository. The examples are identical to `example_ivSVAR_blockbootstrap.m` except for the bootstrap method.

15.7 Methods and formulas

Identification follows Mertens & Ravn (2013) exactly as in `ivSVAR_blockbootstrap`. See that section for complete formulas.

Wild bootstrap algorithm:

For $m = 1, \dots, M$ (where $M = \text{bootstrap_num}$):

1. Draw Rademacher random variables: $\omega_t^{(m)} \sim \text{Rademacher}(0.5)$ for $t = 1, \dots, T_{\text{eff}}$

$$\omega_t^{(m)} = \begin{cases} +1 & \text{with probability } 0.5 \\ -1 & \text{with probability } 0.5 \end{cases}$$

2. Multiply residuals and instruments:

$$u_t^{(m)} = \omega_t^{(m)} u_t, \quad z_t^{(m)} = \omega_t^{(m)} z_t$$

3. Generate bootstrap data:

$$y_t^{(m)} = \hat{\nu} + \sum_{i=1}^p \hat{A}_i y_{t-i}^{(m)} + u_t^{(m)}$$

with initial conditions $y_t^{(m)} = y_t$ for $t \leq p$

4. Re-estimate VAR on $(y^{(m)}, z^{(m)})$ and identify shocks

5. Compute IRFs: $\Phi_h^{(m)}$

Confidence bands are percentiles of $\{\Phi_h^{(1)}, \dots, \Phi_h^{(M)}\}$.

Properties of wild bootstrap:

- $\mathbb{E}[\omega_t] = 0$ (mean-preserving)
- $\mathbb{E}[\omega_t^2] = 1$ (variance-preserving)
- ω_t independent across t (no temporal dependence)
- ω_t independent of (u_s, z_s) for all s, t

The Rademacher distribution is symmetric and preserves the scale of residuals while randomizing their signs.

15.8 References

- Gonçalves, S., and L. Kilian. 2004. "Bootstrapping Autoregressions with Conditional Heteroskedasticity of Unknown Form." *Journal of Econometrics* 123(1): 89–120.
- Mertens, K., and M. O. Ravn. 2013. "The Dynamic Effects of Personal and Corporate Income Tax Changes in the United States." *American Economic Review* 103(4): 1212–1247.
- Stock, J. H., and M. W. Watson. 2012. "Disentangling the Channels of the 2007–2009 Recession." *Brookings Papers on Economic Activity* Spring: 81–135.
- Stock, J. H., and M. W. Watson. 2018. "Identification and Estimation of Dynamic Causal Effects in Macroeconomics Using External Instruments." *Economic Journal* 128(610): 917–948.

16 girfSVAR — Generalized impulse response functions

16.1 Syntax

```
EQ = girfSVAR(data, nlags, var_names, shock_names)
```

```
EQ = girfSVAR(data, nlags, var_names, shock_names, irf_lenght, monte_carlo, ...  
              const, lr, dum, exog, confianza, verbose)
```

16.2 Description

girfSVAR computes generalized impulse response functions (GIRFs) for a vector autoregression (Pesaran and Shin, 1998). Unlike traditional structural VAR methods that rely on orthogonal shocks (e.g., Cholesky decomposition), GIRFs are **order-invariant** and allow for non-orthogonal shocks.

Key features of GIRFs:

- **Order-invariant:** Results do not depend on variable ordering (unlike Cholesky)
- **Variance-normalized:** Each shock represents a one-standard-deviation innovation to that variable
- **Non-orthogonal:** Shocks can be correlated, reflecting actual correlations in the data
- **No unique identification:** GIRFs do not identify structural shocks uniquely

The GIRF of variable i with respect to a shock in variable j at horizon h is:

$$\text{GIRF}_j(h) = \frac{\Phi_h \Sigma e_j}{\sqrt{\sigma_{jj}}}$$

where Φ_h are reduced-form impulse responses, Σ is the residual covariance matrix, e_j is a selection vector, and σ_{jj} is the variance of variable j 's innovation.

GIRFs are particularly useful when:

- Variable ordering is unclear or arbitrary
- You want to allow for correlated shocks
- Economic theory does not provide clear identifying restrictions
- You need a descriptive tool rather than structural identification

16.3 Options

Option	Description
<code>data</code>	$N \times K$ matrix of endogenous variables (N observations, K variables). Required.
<code>nlags</code>	Number of lags to include in the VAR. Required.
<code>var_names</code>	$1 \times K$ cell vector with variable names. Default is <code>{'var1', 'var2', ...}</code> .
<code>shock_names</code>	$1 \times K$ cell vector with shock names. Default is <code>{'var1', 'var2', ...}</code> .
<code>irf_lenght</code>	Length of impulse response functions. Default is 40.
<code>monte_carlo</code>	Number of Monte Carlo repetitions for computing confidence intervals. Default is 1000.
<code>const</code>	Include constant term if <code>true</code> ; exclude if <code>false</code> . Default is <code>false</code> .
<code>lr</code>	$K \times K$ matrix of zeros and ones specifying restrictions on the reduced-form VAR. Element $(i, j) = 0$ restricts variable j from affecting variable i ; $= 1$ allows the effect. Default is no restrictions.

<code>dum</code>	Include seasonal dummies. Specify <code>'month'</code> for 11 monthly dummies or <code>'quarter'</code> for 3 quarterly dummies. Default is no dummies.
<code>exog</code>	$N \times E$ matrix of exogenous variables entering without lags (non-dynamic regressors, e.g., event dummies). Default is no exogenous variables.
<code>confianza</code>	Lower confidence interval percentage. For example, <code>confianza = 5</code> produces 90% confidence intervals (5th and 95th percentiles). Default is 5.
<code>verbose</code>	If <code>true</code> , print Monte Carlo iteration numbers; if <code>false</code> , suppress output. Default is <code>false</code> .

16.4 Stored results

`girfSVAR` stores the following in structure `EQ`:

Matrices

<code>EQ.vari_shockj</code>	$K \times K$ collection of $3 \times \text{irf_length}$ matrices. For each variable i and shock j : first row = 95th percentile (or <code>100-confianza</code>), second row = GIRF, third row = 5th percentile (or <code>confianza</code>). Field names use <code>var_names</code> for variables and <code>shock_names</code> for shocks.
<code>EQ.P</code>	$K \times K$ variance-covariance matrix of reduced-form residuals.
<code>EQ.vardecshock_shockj</code>	K matrices of size $K \times \text{irf_length}$. Each row shows the percentage of variance of that variable explained by shock j at each horizon. Field names use <code>shock_names</code> . Note: variance decomposition may not sum to 100% due to non-orthogonal shocks.
<code>EQ.stdirf</code>	$K \times \text{irf_length} \times K$ array containing standard deviations of GIRFs from Monte Carlo simulation.
<code>EQ.struc</code>	$K \times (N - \text{nlags})$ matrix of generalized structural shocks (non-orthogonal, variance-normalized).
<code>EQ.irf_sim</code>	$K \times \text{irf_length} \times \text{monte_carlo} \times K$ array of all GIRF draws from Monte Carlo simulation. Useful for analyzing the full distribution of uncertainty.

16.5 Remarks

Order invariance: The defining feature of GIRFs is **order invariance**. Unlike Cholesky decomposition, which imposes a recursive ordering on variables, GIRFs produce identical results regardless of how variables are ordered. This makes GIRFs particularly attractive when:

- Economic theory does not suggest a natural ordering
- Multiple orderings are plausible
- Robustness to ordering is desired

To verify order invariance, reorder variables in the data matrix and re-estimate. The GIRFs for any variable-shock pair will be identical (up to numerical precision).

Variance normalization: Each GIRF represents the response to a **one-standard-deviation shock** to that variable's innovation. Specifically, a shock to variable j has size $\sqrt{\sigma_{jj}}$, where σ_{jj} is the variance of the j -th reduced-form innovation.

This normalization makes shocks comparable across variables with different scales. For example, if inflation has higher variance than output, a "one-unit" shock to each would not be comparable. GIRFs solve this by normalizing to one standard deviation.

Non-orthogonal shocks: Unlike Cholesky or other structural methods, GIRFs allow shocks to be **correlated**. The generalized structural shocks in `EQ.struc` are non-orthogonal:

$$\mathbb{E}[\varepsilon_t \varepsilon_t'] \neq I_K$$

This reflects actual correlations in the data. While orthogonality is convenient for structural interpretation, it may be unrealistic. GIRFs acknowledge that innovations to different variables may be correlated.

Variance decomposition exceeding 100%: Because shocks are non-orthogonal, the variance decomposition may **sum to more than 100%** or less than 100%. This is not an error but a consequence of allowing correlated shocks.

For orthogonal shocks: $\text{Var}(y_t) = \sum_{j=1}^K \text{Var}(y_t|\varepsilon_j)$

For non-orthogonal shocks: $\text{Var}(y_t) \neq \sum_{j=1}^K \text{Var}(y_t|\varepsilon_j)$ due to covariance terms.

When interpreting variance decomposition with GIRFs, focus on relative magnitudes rather than expecting contributions to sum to exactly 100%.

No unique structural identification: GIRFs do **not** identify structural shocks uniquely. They describe how variables respond to their own one-standard-deviation innovations, integrating over the joint distribution of all shocks. GIRFs are a descriptive tool rather than a method for structural identification.

If the goal is to identify economically meaningful structural shocks (e.g., monetary policy, technology), use methods with identifying restrictions:

- Cholesky with justified ordering
- Long-run restrictions (**lrSVAR**)
- Sign restrictions (**signrestSVAR**)
- External instruments

GIRFs are best suited for:

- Descriptive analysis when theory is unclear
- Robustness checks (invariant to ordering)
- Forecasting and scenario analysis
- Preliminary exploration before imposing restrictions

Reduced-form VAR restrictions: The **lr** parameter allows restrictions on the reduced-form VAR (which variables affect which in lags). These restrictions are respected in computing GIRFs. Note: these are restrictions on the **reduced-form** dynamics, not on structural identification.

Monte Carlo confidence intervals: Confidence intervals are computed via Monte Carlo simulation:

1. Generate artificial data from estimated VAR
2. Re-estimate VAR and compute GIRFs
3. Repeat for `monte_carlo` draws
4. Report percentiles as confidence bands

These intervals reflect **parameter uncertainty** (sampling variation in VAR coefficients and covariance matrix), not identification uncertainty.

Generalized structural shocks: `EQ.struc` contains generalized structural shocks, computed as:

$$\varepsilon_{jt} = \frac{\Sigma_{jj}^{-1/2} \Sigma_{j\cdot} u_t}{\sigma_{jj}}$$

where $\Sigma_{jj}^{-1/2}$ is the j -th diagonal element of the inverse square root of Σ , and u_t are reduced-form residuals. These shocks are:

- Non-orthogonal (correlated)
- Variance-normalized (each has unit variance)
- Useful for historical decomposition or counterfactual analysis

Check correlation: `corrcoef(EQ.struc')` will show non-zero off-diagonal elements.

When to use GIRFs vs other methods:

Use GIRFs when:

- Variable ordering is unclear or multiple orderings are plausible
- You want robustness to ordering assumptions
- Allowing for correlated shocks is desirable
- The goal is descriptive rather than structural

Use other methods when:

- You have strong economic theory suggesting specific restrictions
- Unique structural identification is needed
- Orthogonal shocks are required
- Variance decomposition must sum to 100%

Comparison:

- **GIRFs:** Order-invariant, non-orthogonal, no unique identification
- **Cholesky:** Order-dependent, orthogonal, unique identification
- **Long-run:** Restrictions from theory, orthogonal, unique identification
- **Sign restrictions:** Set identification, potentially non-orthogonal, theoretically motivated

Plots: The function automatically generates K figures, one for each shock, showing the responses of all K variables with confidence bands.

16.6 Examples

Example 1: Basic generalized impulse response functions

Compute GIRFs for a three-variable VAR:

```
var_names = {'Output', 'Inflation', 'InterestRate'};
shock_names = {'OutputShock', 'InflationShock', 'RateShock'};

EQ = girfSVAR(data, nlags, var_names, shock_names);

% Variance-covariance matrix
disp(EQ.P);

% Each shock is 1-SD of that variable
disp('Standard deviations:');
for i = 1:3
    fprintf('%s: %.4f\n', var_names{i}, sqrt(EQ.P(i,i)));
end
```

Example 2: Demonstrating order invariance

Verify that GIRFs are order-invariant:

```

% Original ordering
EQ1 = girfSVAR(data, nlags, ...
    {'Output', 'Inflation', 'InterestRate'}, ...
    {'OutputShock', 'InflationShock', 'RateShock'});

% Different ordering: Interest Rate first
data_reordered = data(:, [3, 1, 2]);
EQ2 = girfSVAR(data_reordered, nlags, ...
    {'InterestRate', 'Output', 'Inflation'}, ...
    {'RateShock', 'OutputShock', 'InflationShock'});

% Compare Output response to Inflation shock at h=10
irf1 = EQ1.Output_InflationShock(2, 10);
irf2 = EQ2.Output_InflationShock(2, 10);

fprintf('Original ordering: %.6f\n', irf1);
fprintf('Different ordering: %.6f\n', irf2);
fprintf('Difference: %.10f (should be ~0)\n', abs(irf1 - irf2));

% GIRFs are identical regardless of ordering

```

Example 3: Non-orthogonal shocks and variance decomposition

Examine variance decomposition that may exceed 100%:

```

EQ = girfSVAR(data, nlags, var_names, shock_names);

% Variance decomposition of Output at h=20
fprintf('Output variance explained by:\n');
fprintf('  Output shock:   %.1f%%\n', EQ.vardecshock_OutputShock(1,20));
fprintf('  Inflation shock: %.1f%%\n', EQ.vardecshock_InflationShock(1,20));
fprintf('  Rate shock:      %.1f%%\n', EQ.vardecshock_RateShock(1,20));
total = EQ.vardecshock_OutputShock(1,20) + ...
    EQ.vardecshock_InflationShock(1,20) + ...
    EQ.vardecshock_RateShock(1,20);
fprintf('  Total:          %.1f%% (may 100%%)\n', total);

% Check correlation of generalized structural shocks
shock_corr = corrcoef(EQ.struc');
disp('Correlation matrix of shocks:');
disp(shock_corr);
% Non-zero off-diagonal elements show non-orthogonality

```

Example 4: Monte Carlo uncertainty analysis

Analyze the full distribution of GIRFs using EQ.irf_sim:

```

EQ = girfSVAR(data, nlags, var_names, shock_names);

% Extract all Monte Carlo draws: Inflation shock → Output at h=10
% irf_sim dimensions: [variable, horizon, repetition, shock]
irf_draws = squeeze(EQ.irf_sim(1, 10, :, 2));

% Summary statistics
fprintf('Median: %.4f\n', median(irf_draws));
fprintf('Mean:   %.4f\n', mean(irf_draws));
fprintf('Std:    %.4f\n', std(irf_draws));

```

```
% Plot distribution
figure;
histogram(irf_draws, 30, 'Normalization', 'pdf');
hold on;
xline(median(irf_draws), 'r-', 'LineWidth', 2);
xline(prctile(irf_draws, 5), 'b--');
xline(prctile(irf_draws, 95), 'b--');
title('Distribution of GIRF: Inflation → Output (h=10)');
xlabel('GIRF value'); ylabel('Density');
```

Example 5: When to use GIRFs

Guidance on appropriate use cases:

```
% Use GIRFs when:
% 1. Variable ordering is unclear
% 2. You want robustness to ordering
% 3. Allowing for correlated shocks is desirable
% 4. Descriptive analysis, not structural identification

% Do NOT use GIRFs when:
% 1. You need unique structural identification
% 2. You have strong theory suggesting restrictions
% 3. Orthogonal shocks are required
% 4. Variance decomposition must sum to 100%

% Example: Compare with Cholesky
EQ_girf = girfSVAR(data, nlags, var_names, shock_names);
EQ_chol = srSVAR(data, nlags, var_names, shock_names);

% GIRFs: order-invariant, non-orthogonal
% Cholesky: order-dependent, orthogonal
```

For complete, executable examples including variance decomposition visualization, seasonal dummies, exogenous variables, and detailed comparisons, see `example_girfSVAR.m` in the package repository.

16.7 Methods and formulas

The reduced-form VAR is:

$$y_t = \nu + A_1 y_{t-1} + \cdots + A_p y_{t-p} + u_t$$

with $\mathbb{E}[u_t u_t'] = \Sigma$.

Generalized impulse response function:

The GIRF of variable i with respect to a shock in variable j at horizon h is defined as:

$$\text{GIRF}_{i,j}(h) = \mathbb{E}[y_{t+h} | u_{jt} = \sqrt{\sigma_{jj}}, \Omega_{t-1}] - \mathbb{E}[y_{t+h} | \Omega_{t-1}]$$

where Ω_{t-1} denotes information available at time $t-1$.

Pesaran and Shin (1998) show this equals:

$$\text{GIRF}_j(h) = \frac{\Phi_h \Sigma e_j}{\sqrt{\sigma_{jj}}}$$

where:

- $\Phi_h = J A^h J'$ are reduced-form impulse responses at horizon h
- $J = [I_K : 0_{K \times K(p-1)}]$ is the selection matrix
- A is the companion matrix

- Σ is the residual covariance matrix
- e_j is a $K \times 1$ selection vector (1 in position j , zero elsewhere)
- σ_{jj} is the variance of innovation j

Order invariance:

The GIRF is invariant to variable ordering because it does not depend on any particular factorization of Σ . For any permutation π of variables:

$$\text{GIRF}_{i,j}^\pi(h) = \text{GIRF}_{i,j}(h)$$

This contrasts with Cholesky decomposition, where $P = \text{chol}(\Sigma)'$ depends on ordering.

Variance normalization:

The shock to variable j has magnitude $\sqrt{\sigma_{jj}}$, ensuring each shock is one standard deviation. This makes shocks comparable across variables with different scales.

The impact effect is:

$$\text{GIRF}_j(0) = \frac{\Sigma e_j}{\sqrt{\sigma_{jj}}} = \frac{1}{\sqrt{\sigma_{jj}}} \begin{bmatrix} \sigma_{1j} \\ \sigma_{2j} \\ \vdots \\ \sigma_{Kj} \end{bmatrix}$$

Variable j has an impact of $\sqrt{\sigma_{jj}}$ on itself, while other variables respond proportionally to their covariance with variable j .

Generalized structural shocks:

The generalized structural shock for variable j is:

$$\varepsilon_{jt} = \frac{e_j' \Sigma^{-1} u_t}{\sqrt{e_j' \Sigma^{-1} e_j}}$$

These shocks are:

- Non-orthogonal: $\mathbb{E}[\varepsilon_t \varepsilon_t'] \neq I_K$
- Variance-normalized: $\mathbb{E}[\varepsilon_{jt}^2] = 1$ for all j
- Related to reduced-form innovations: $\varepsilon_t = D^{-1} \Sigma^{-1} u_t$ where D normalizes variances

Variance decomposition:

The contribution of shock j to the variance of variable i at horizon h is:

$$\omega_{ij}(h) = \frac{\sigma_{jj}^{-1} \sum_{s=0}^{h-1} [(\Phi_s \Sigma e_j)_i]^2}{\sum_{s=0}^{h-1} [\Phi_s \Sigma \Phi_s']_{ii}} \times 100\%$$

Due to non-orthogonality:

$$\sum_{j=1}^K \omega_{ij}(h) \neq 100\%$$

The deviation from 100% reflects covariances between shocks.

Monte Carlo confidence intervals:

For $m = 1, \dots, M$ (where $M = \text{monte_carlo}$):

1. Generate $u^{(m)} \sim N(0, \hat{\Sigma})$ where $\hat{\Sigma}$ is the estimated covariance
2. Generate artificial data: $y_t^{(m)} = \hat{\nu} + \sum_{i=1}^p \hat{A}_i y_{t-i}^{(m)} + u_t^{(m)}$
3. Re-estimate VAR: $\hat{A}^{(m)}, \hat{\Sigma}^{(m)}$
4. Compute GIRFs: $\text{GIRF}_j^{(m)}(h) = \Phi_h^{(m)} \hat{\Sigma}^{(m)} e_j / \sqrt{\sigma_{jj}^{(m)}}$

Confidence bands are percentiles of $\{\text{GIRF}_j^{(1)}(h), \dots, \text{GIRF}_j^{(M)}(h)\}$.

16.8 References

Pesaran, H. H., and Y. Shin. 1998. "Generalized Impulse Response Analysis in Linear Multivariate Models." *Economics Letters* 58(1): 17–29.

Koop, G., M. H. Pesaran, and S. M. Potter. 1996. "Impulse Response Analysis in Nonlinear Multivariate Models." *Journal of Econometrics* 74(1): 119–147.

Lütkepohl, H. 2005. *New Introduction to Multiple Time Series Analysis*. Berlin: Springer-Verlag.

Pesaran, H. H., and B. Pesaran. 1997. *Working with Microfit 4.0: Interactive Econometric Analysis*. Oxford: Oxford University Press.

17 nolucas — Counterfactual historical decomposition with channel restrictions

17.1 Syntax

```
EQ = nolucas(data, constants, coefficients, idmatrix, strucshocks, shock, ...
             withvar, withconst)
```

```
EQ = nolucas(data, constants, coefficients, idmatrix, strucshocks, shock, ...
             withvar, withconst, dum, dummies, exog, coef_exog, forecast)
```

17.2 Description

nolucas performs **counterfactual historical decomposition** for structural VARs with the ability to **block specific transmission channels**. The function decomposes the contribution of a single structural shock while allowing the researcher to shut down the transmission of that shock through selected variables.

CRITICAL CAVEAT: The name "nolucas" reflects that this analysis is **NOT robust to the Lucas critique**. When transmission channels are artificially blocked, the function assumes VAR coefficients remain unchanged and agents do not respond to the structural change. This is a **diagnostic tool for measuring historical importance of channels**, not a structural model for policy evaluation.

Key features:

- **Historical decomposition:** Decomposes data into shock contributions and deterministic component
- **Channel blocking:** Shut down transmission through specific variables
- **Direct vs indirect effects:** Decompose total shock effect into direct and indirect channels
- **Counterfactual forecasting:** Project forward with restricted transmission
- **Flexible deterministic treatment:** Choose whether blocked variables affect deterministic dynamics

Appropriate uses:

- **Diagnostic tool:** "How important was the interest rate channel historically?"
- **Decomposition:** "What share of the monetary policy effect came through exchange rates?"
- **Comparison:** "Which transmission channel matters more for output vs inflation?"

Inappropriate uses (violate Lucas critique):

- **Policy recommendation:** "We should block channel X"
- **Counterfactual policy:** "If we had blocked X, Y would have happened"
- **Structural interpretation:** "Channel X is unnecessary"

17.3 Options

Option	Description
<code>data</code>	$N \times K$ matrix of observed data (N observations, K variables). Required.
<code>constants</code>	$K \times 1$ vector of constants from the reduced-form VAR estimation. Typically obtained from <code>Q.constants</code> . Required.
<code>coefficients</code>	$K \times (K \cdot \text{nlags})$ matrix of autoregressive coefficients from the reduced-form VAR. Typically obtained from <code>Q.coefficients</code> . Required.
<code>idmatrix</code>	$K \times K$ structural impact matrix (identification matrix P). Transforms structural shocks into reduced-form residuals. Typically obtained from <code>EQ.P</code> . Required.

<code>strucshocks</code>	$K \times (N - \text{nlags})$ matrix of structural shocks. Row j contains the time series of structural shock j . Typically obtained from <code>EQ.struc</code> . Required.
<code>shock</code>	Scalar index (1 to K) of the structural shock to analyze. For example, if shock 3 is monetary policy, use <code>shock = 3</code> . Required.
<code>withvar</code>	$1 \times K$ vector specifying which transmission channels to keep open. Element $j = 1$ allows transmission through variable j ; element $j = 0$ blocks transmission through variable j . Example: <code>[1 1 0 1]</code> blocks transmission through variable 3. Required.
<code>withconst</code>	Scalar controlling whether blocked variables affect the deterministic component. If <code>withconst = 1</code> : blocked variables still affect autoregressive dynamics in the deterministic part (uses full A matrix). If <code>withconst = 0</code> : blocked variables are fully shut down in both shock transmission and deterministic dynamics (uses restricted A matrix). Required.
<code>dum</code>	Seasonal dummies. Specify 'month' for 11 monthly dummies or 'quarter' for 3 quarterly dummies. Default is no dummies.
<code>dummies</code>	$K \times \text{numdum}$ matrix of seasonal dummy coefficients from VAR estimation. Typically obtained from <code>Q.dummies</code> . Required if <code>dum</code> is specified.
<code>exog</code>	$N \times E$ matrix of exogenous variables (e.g., crisis dummies, policy events). These enter the VAR without lags. Default is no exogenous variables.
<code>coef_exog</code>	$K \times E$ matrix of coefficients on exogenous variables from VAR estimation. Typically obtained from <code>Q.exo</code> . Required if <code>exog</code> is specified.
<code>forecast</code>	Number of periods ahead to forecast with restricted transmission. Default is 0 (no forecasting).

17.4 Stored results

`nolucas` stores the following in structure `EQ`:

Matrices

<code>EQ.contribution</code>	$K \times N$ matrix containing the total counterfactual path: contribution of the shock plus deterministic component, both computed with restricted transmission channels.
<code>EQ.onlyshock</code>	$K \times N$ matrix containing only the shock contribution (no deterministic part). Shows the pure effect of the structural shock under restricted transmission.
<code>EQ.deterministic</code>	$K \times N$ matrix containing only the deterministic component (constants, seasonal dummies, exogenous variables, autoregressive dynamics).
<code>EQ.forecast_contribution</code>	$K \times \text{forecast}$ matrix of total forecasted contribution (only if <code>forecast > 0</code>).
<code>EQ.forecast_onlyshock</code>	$K \times \text{forecast}$ matrix of forecasted shock contribution (only if <code>forecast > 0</code>).
<code>EQ.forecast_deterministic</code>	$K \times \text{forecast}$ matrix of forecasted deterministic component (only if <code>forecast > 0</code>).

17.5 Remarks

Identity (no channels blocked): When all transmission channels are open (`withvar = ones(1,K)` and `withconst = 1`), the function performs standard historical decomposition. In this case, the following identity holds:

$$\text{data} = \sum_{j=1}^K \text{EQ}_j.\text{onlyshock} + \text{EQ}.\text{deterministic}$$

where EQ_j is the output from running `nolucas` for shock j . This identity states that the observed data equals the sum of all shock contributions plus the deterministic component. The examples verify this identity to machine precision.

Transmission channel blocking: The `withvar` vector controls which variables can transmit the shock's effects:

- `withvar(j) = 1`: Variable j can transmit shock effects to other variables (lags of variable j affect future values of all variables)
- `withvar(j) = 0`: Variable j cannot transmit shock effects (all coefficients on lags of variable j are set to zero in the restricted dynamics)

Example: In a 4-variable VAR with [Output, Inflation, InterestRate, ExchangeRate]:

- `withvar = [1 1 0 1]`: Blocks transmission through InterestRate
- `withvar = [1 1 1 0]`: Blocks transmission through ExchangeRate
- `withvar = [1 1 0 0]`: Blocks both InterestRate and ExchangeRate (only direct effects remain)

When transmission through variable j is blocked, the shock still impacts variable j directly (through the impact matrix P), but subsequent dynamics do not propagate through lags of variable j .

The `withconst` parameter: This parameter controls whether blocked variables affect the deterministic component of the decomposition:

`withconst = 1` (partial blocking):

- Blocked variables cannot transmit the **shock** (restricted A matrix for shock dynamics)
- Blocked variables **can** affect the **deterministic** component (full A matrix for deterministic dynamics)
- Interpretation: "The shock cannot propagate through variable j , but variable j still influences the baseline path"
- Less restrictive

`withconst = 0` (full blocking):

- Blocked variables cannot transmit the shock (restricted A matrix)
- Blocked variables **cannot** affect the deterministic component (restricted A matrix for deterministic dynamics)
- Interpretation: "Variable j is completely shut down from the system dynamics"
- More restrictive

Example: If blocking the interest rate channel for a monetary shock:

- `withconst = 1`: Interest rate lags still affect output/inflation in the baseline path, but the monetary shock cannot work through interest rates
- `withconst = 0`: Interest rate lags are zeroed out completely—shock cannot work through rates, and rates don't affect baseline dynamics

In most applications, `withconst = 1` is more appropriate, as you typically want to isolate the shock transmission mechanism while keeping the structural dynamics intact.

Direct vs indirect effects: The function enables decomposition of shock effects into direct and indirect channels:

- **Total effect:** Run with `withvar = ones(1,K)` (all channels open)
- **Direct effect:** Run with selected channels blocked (e.g., `withvar = [1 1 0 0]`)
- **Indirect effect:** Difference between total and direct

Example: For a monetary policy shock in a VAR with [Output, Inflation, InterestRate, ExchangeRate]:

$$\begin{aligned}\text{Total effect} &= \text{nolucas}(\dots, [1, 1, 1, 1], \dots) \\ \text{Direct effect} &= \text{nolucas}(\dots, [1, 1, 0, 0], \dots) \\ \text{Indirect effect} &= \text{Total effect} - \text{Direct effect}\end{aligned}$$

The indirect effect measures how much of the monetary shock's impact on output/inflation operates through interest rates and exchange rates.

Lucas critique and appropriate interpretation: This function is **NOT robust to the Lucas critique**. When you block a transmission channel, the analysis assumes:

1. VAR coefficients ($A_1, A_2, \dots, A_p$) remain unchanged
2. Other transmission channels function exactly as before
3. Economic agents do not respond to the structural change
4. The policy rule does not adjust

In reality, if you actually shut down a transmission mechanism:

- Policymakers would change their behavior (e.g., central bank would use different instruments)
- Private agents would adjust expectations and decision rules
- Other channels would change (not remain constant)
- VAR coefficients would be fundamentally different

Correct interpretation: "Historically, X% of the monetary shock's effect on output operated through the interest rate channel, holding everything else constant."

Incorrect interpretation: "If we blocked the interest rate channel, output would have responded X% less to monetary shocks."

The function is a **diagnostic tool** for understanding historical transmission mechanisms, not a structural model for evaluating counterfactual policies. Use it to measure the relative importance of different channels in the data, not to predict what would happen under alternative policy regimes.

Seasonal dummies: When `dum` is specified, seasonal dummies are included in the deterministic component. The function properly handles time-varying dummy vectors throughout the sample and forecast periods. Seasonal patterns are controlled for before isolating shock contributions.

Exogenous variables: Exogenous variables (e.g., crisis dummies, commodity price shocks, policy events) are included in the deterministic component. These variables affect all variables contemporaneously without lags, and their effects are separated from the structural shock contribution being analyzed.

Forecasting: When `forecast` > 0, the function projects the shock contribution and deterministic component forward using the restricted dynamics. This answers: "If this shock's effects had been constrained to work only through certain channels, what would the forecast path look like?" Again, this is subject to the Lucas critique and should be interpreted as a mechanical extrapolation, not a structural forecast.

17.6 Examples

Example 1: Standard historical decomposition (no channels blocked)

Decompose monetary shock contribution with all transmission channels active:

```
% After estimating SVAR
EQ = srSVAR(data, nlags, var_names, 40, 1000, true);
Q = reducedformVAR(data, nlags, true);

% All channels open
withvar = ones(1, K);
withconst = 1;
```

```

shock_idx = 3; % Monetary shock

EQ1 = nolucas(data, Q.constants, Q.coefficients, EQ.P, EQ.struc, ...
              shock_idx, withvar, withconst);

% Plot decomposition
plot(1:T, EQ1.onlyshock(1,:)); % Shock contribution to output
plot(1:T, EQ1.deterministic(1,:)); % Deterministic component
plot(1:T, EQ1.contribution(1,:)); % Total = shock + deterministic

```

Example 2: Verify decomposition identity

Check that sum of all shocks plus deterministic equals data:

```

% Sum shock contributions from all K shocks
sum_shocks = zeros(K, T);
for s = 1:K
    EQ_s = nolucas(data, Q.constants, Q.coefficients, EQ.P, ...
                  EQ.struc, s, ones(1,K), 1);
    sum_shocks = sum_shocks + EQ_s.onlyshock;
end

% Add deterministic component
reconstructed = sum_shocks + EQ_s.deterministic;

% Check identity
max_error = max(max(abs(reconstructed - data')));
fprintf('Max reconstruction error: %.2e\n', max_error);
% Should be < 1e-10 (machine precision)

```

Example 3: Block interest rate transmission channel

Isolate monetary shock effects that do NOT work through interest rates:

```

% VAR: [Output, Inflation, InterestRate, ExchangeRate]
% Block transmission through InterestRate (variable 3)

withvar = [1 1 0 1]; % Zero out interest rate channel
withconst = 0; % Fully block interest rate
shock_idx = 3; % Monetary shock

EQ_blocked = nolucas(data, Q.constants, Q.coefficients, EQ.P, ...
                    EQ.struc, shock_idx, withvar, withconst);

% Compare with unrestricted
EQ_full = nolucas(data, Q.constants, Q.coefficients, EQ.P, ...
                  EQ.struc, shock_idx, ones(1,K), 1);

% Role of interest rate channel for output
role_IR = mean(abs(EQ_full.onlyshock(1,:) - EQ_blocked.onlyshock(1,:)));
fprintf('Average importance of IR channel: %.4f\n', role_IR);

```

Example 4: Decompose total effect into direct and indirect

Separate direct effects from indirect effects through interest and exchange rates:

```

% Total effect (all channels)
EQ_total = nolucas(data, Q.constants, Q.coefficients, EQ.P, ...

```

```

EQ.struc, 3, [1 1 1 1], 1);

% Direct effect only (block InterestRate and ExchangeRate)
EQ_direct = nolucas(data, Q.constants, Q.coefficients, EQ.P, ...
    EQ.struc, 3, [1 1 0 0], 0);

% Indirect effects
indirect = EQ_total.onlyshock - EQ_direct.onlyshock;

% Average contributions to output
fprintf('Total effect:    %.4f\n', mean(abs(EQ_total.onlyshock(1,50:end))));
fprintf('Direct effect:   %.4f\n', mean(abs(EQ_direct.onlyshock(1,50:end))));
fprintf('Indirect effect: %.4f\n', mean(abs(indirect(1,50:end))));

```

Example 5: Understanding withconst parameter

Compare partial blocking (withconst=1) vs full blocking (withconst=0):

```

withvar = [1 1 0 1]; % Block InterestRate
shock_idx = 3;

% Partial blocking: InterestRate still affects deterministic part
EQ_partial = nolucas(data, Q.constants, Q.coefficients, EQ.P, ...
    EQ.struc, shock_idx, withvar, 1);

% Full blocking: InterestRate fully shut down
EQ_full = nolucas(data, Q.constants, Q.coefficients, EQ.P, ...
    EQ.struc, shock_idx, withvar, 0);

% Compare deterministic components
fprintf('Deterministic (withconst=1): %.4f\n', ...
    EQ_partial.deterministic(1,100));
fprintf('Deterministic (withconst=0): %.4f\n', ...
    EQ_full.deterministic(1,100));

```

Example 6: Counterfactual forecasting with blocked channels

Project forward the shock contribution under restricted transmission:

```

forecast_horizon = 20;
withvar = [1 1 0 1]; % Block InterestRate channel
shock_idx = 3;

EQ_forecast = nolucas(data, Q.constants, Q.coefficients, EQ.P, ...
    EQ.struc, shock_idx, withvar, 0, [], [], ...
    [], [], forecast_horizon);

% Plot historical + forecast
plot(T-50:T, EQ_forecast.onlyshock(1, T-50:T), 'b-');
hold on;
plot(T+1:T+forecast_horizon, EQ_forecast.forecast_onlyshock(1,:), 'r--');
xlabel('Time'); ylabel('Output');
legend('Historical', 'Forecast');
title('Monetary Shock Forecast (IR Channel Blocked)');

```

Example 7: With seasonal dummies

Include quarterly seasonal patterns in the decomposition:

```
% Estimate VAR with seasonal dummies
EQ = srSVAR(data, nlags, var_names, 40, 1000, true, [], 'quarter');
Q = reducedformVAR(data, nlags, true, [], [], 'quarter');

% Decompose with seasonal dummies
EQ_seas = nolucas(data, Q.constants, Q.coefficients, EQ.P, ...
    EQ.struc, 2, ones(1,K), 1, 'quarter', Q.dummies);

% Seasonal patterns properly controlled in deterministic component
```

Example 8: With exogenous crisis dummy

Control for financial crisis when decomposing shock contributions:

```
% Crisis dummy
exog = zeros(T, 1);
exog(120:130) = 1; % Crisis period

% Add crisis effect to data
data_crisis = data + repmat([0.5; 1.0; 0.3; -0.2]', T, 1) .* exog;

% Estimate VAR with exogenous variable
EQ = srSVAR(data_crisis, nlags, var_names, 40, 1000, true, ...
    [], [], exog);
Q = reducedformVAR(data_crisis, nlags, true, [], [], [], exog);

% Decompose controlling for crisis
EQ_exog = nolucas(data_crisis, Q.constants, Q.coefficients, EQ.P, ...
    EQ.struc, 2, ones(1,K), 1, [], [], exog, Q.exo);

% Crisis effects isolated in deterministic component
```

For complete, executable examples including identity verification, channel decomposition plots, direct vs indirect effect analysis, and Lucas critique discussion, see `example_nolucas.m` in the package repository.

17.7 Methods and formulas

The function operates on the companion-form VAR:

$$Y_t = V + AY_{t-1} + U_t$$

where Y_t is the $(Kp) \times 1$ stacked vector of current and lagged values, V contains constants and seasonal dummies, A is the companion matrix, and U_t contains structural shocks.

Standard historical decomposition:

For a single structural shock ε_j , the contribution to observed data is:

$$y_t^{(j)} = \sum_{h=0}^{t-1} \Phi_h P e_j \varepsilon_{j,t-h}$$

where:

- $\Phi_h = JA^h J'$ is the reduced-form moving average coefficient at lag h
- P is the structural impact matrix (`idmatrix`)
- e_j is the j -th unit vector selecting shock j
- $J = [I_K : 0_{K \times K(p-1)}]$ selects the first K elements

The deterministic component is:

$$y_t^{\text{det}} = \sum_{h=0}^{t-1} \Phi_h V_t$$

The identity holds:

$$y_t = \sum_{j=1}^K y_t^{(j)} + y_t^{\text{det}}$$

Channel-restricted decomposition:

Define the restricted companion matrix A_{less} by zeroing out columns corresponding to blocked variables:

$$[A_{\text{less}}]_{:,k+K(i-1)} = \begin{cases} [A]_{:,k+K(i-1)} & \text{if } \text{withvar}(k) = 1 \\ 0 & \text{if } \text{withvar}(k) = 0 \end{cases}$$

for $k = 1, \dots, K$ and $i = 1, \dots, p$.

The restricted moving average coefficients are:

$$\Phi_h^{\text{less}} = J A_{\text{less}}^h J'$$

Shock contribution with restricted transmission:

$$y_t^{(j),\text{rest}} = \sum_{h=0}^{t-1} \Phi_h^{\text{less}} P e_j \varepsilon_{j,t-h}$$

Deterministic component:

- If `withconst = 1`: Uses full A matrix

$$y_t^{\text{det}} = \sum_{h=0}^{t-1} \Phi_h V_t$$

- If `withconst = 0`: Uses restricted A_{less} matrix

$$y_t^{\text{det,rest}} = \sum_{h=0}^{t-1} \Phi_h^{\text{less}} V_t$$

Outputs:

- `EQ.onlyshock` = $y_t^{(j),\text{rest}}$
- `EQ.deterministic` = y_t^{det} or $y_t^{\text{det,rest}}$ depending on `withconst`
- `EQ.contribution` = $y_t^{(j),\text{rest}} + y_t^{\text{det}}$ (or $y_t^{\text{det,rest}}$)

Forecasting:

For $h > 0$ periods ahead, the shock contribution forecast is:

$$\hat{y}_{T+h}^{(j),\text{rest}} = \Phi_h^{\text{less}} y_T^{(j),\text{rest}}$$

The deterministic forecast uses either Φ_h or Φ_h^{less} depending on `withconst`.

17.8 References

- Lucas, R. E., Jr. 1976. "Econometric Policy Evaluation: A Critique." *Carnegie-Rochester Conference Series on Public Policy* 1: 19–46.
- Kilian, L., and H. Lütkepohl. 2017. *Structural Vector Autoregressive Analysis*. Cambridge University Press. Chapter 13 (Historical Decomposition).
- Stock, J. H., and M. W. Watson. 2001. "Vector Autoregressions." *Journal of Economic Perspectives* 15(4): 101–115.

18 granger — Pairwise Granger causality tests

18.1 Syntax

```
Q = granger(data, nlags)
```

```
Q = granger(data, nlags, var_names, const, verbose)
```

18.2 Description

granger performs pairwise **Granger causality tests** on a vector autoregression. For each pair of variables (i, j) , the function tests the null hypothesis that variable j does NOT Granger-cause variable i by testing whether all coefficients on lags of variable j in the equation for variable i are jointly zero.

The function reports both Wald F-statistics and Wald Chi-squared statistics. These are numerically related by $\chi^2 = \text{nlags} \times F$, but have different distributional properties. The F-statistic is typically preferred in finite samples.

Key features:

- **Pairwise tests:** Tests all $K(K - 1)$ directed pairs in a K -variable VAR
- **Wald test:** Jointly tests **nlags** restrictions per pair
- **F and Chi-squared versions:** Both statistics reported with p-values
- **Flexible output:** Verbose mode displays formatted results with significance stars
- **Matrix structure:** Element (i, j) tests whether variable j Granger-causes variable i

Interpretation of Granger causality:

- "Variable j Granger-causes variable i " means: lags of j help predict i beyond what lags of i (and other variables) already predict
- **Does NOT imply:** true causation, exogeneity, or structural relationships
- **Does imply:** predictive content, temporal precedence, useful forecasting information

Granger causality is a test of **predictive causality**, not structural causality. It answers: "Does knowing the past of variable j improve forecasts of variable i ?"

18.3 Options

Option	Description
<code>data</code>	$N \times K$ matrix of observed data (N observations, K variables). Required.
<code>nlags</code>	Number of lags in the VAR. The test imposes nlags restrictions per variable pair. Required.
<code>var_names</code>	$1 \times K$ cell vector with variable names for display. Default is <code>{'var1', 'var2', ..., 'varK'}</code> .
<code>const</code>	Include constant term if true ; exclude if false . Affects degrees of freedom in F-test. Default is false .
<code>verbose</code>	Display formatted results with significance stars if true ; suppress output if false . Default is true .

18.4 Stored results

granger stores the following in structure **Q**:

Matrices

<code>Q.lambdaF</code>	$K \times K$ matrix of Wald F-statistics. Element (i, j) is the F-statistic for testing the null hypothesis that variable j does NOT Granger-cause variable i . These are the statistics reported by Stata's vargranger command.
<code>Q.Pf</code>	$K \times K$ matrix of p-values corresponding to <code>Q.lambdaF</code> . Element (i, j) is the p-value for the F-test.
<code>Q.lambdaCHI</code>	$K \times K$ matrix of Wald Chi-squared statistics. Element (i, j) is the χ^2 statistic for testing the null hypothesis that variable j does NOT Granger-cause variable i .
<code>Q.Pchi</code>	$K \times K$ matrix of p-values corresponding to <code>Q.lambdaCHI</code> .

18.5 Remarks

Null hypothesis and interpretation: For each (i, j) pair with $i \neq j$, the null hypothesis is:

$$H_0 : \beta_{ij}^{(1)} = \beta_{ij}^{(2)} = \dots = \beta_{ij}^{(p)} = 0$$

where $\beta_{ij}^{(\ell)}$ is the coefficient on lag ℓ of variable j in the equation for variable i , and $p = \text{nlags}$.

- **Reject H_0 (small p-value):** Variable j Granger-causes variable i . Past values of j contain useful information for forecasting i beyond what is already contained in past values of i and other variables.
- **Fail to reject H_0 (large p-value):** No evidence of Granger causality from j to i . Past values of j do not significantly improve forecasts of i .

Important caveats:

- Granger causality is **not structural causality**. It measures predictive relationships, not causal mechanisms.
- Presence of omitted variables can create spurious Granger causality.
- Results are sensitive to lag length selection. Too few lags miss dynamics; too many lags reduce power.
- Rejection tells you j helps predict i , but not **how** (direct effect, indirect through other variables, or both).

Matrix indexing convention: The (i, j) element tests whether column variable j Granger-causes row variable i :

$$\text{Q.lambdaF}(i, j) = \text{F-statistic for } H_0 : j \text{ does not Granger-cause } i$$

Example: In a 3-variable VAR with [Output, Inflation, InterestRate]:

- `Q.lambdaF(2,1)`: Tests whether Output Granger-causes Inflation
- `Q.lambdaF(1,2)`: Tests whether Inflation Granger-causes Output
- `Q.Pf(3,2)`: P-value for testing whether Inflation Granger-causes InterestRate

The diagonal elements (i, i) are not meaningful (a variable always "Granger-causes" itself through its own lags).

F-statistic vs Chi-squared statistic: The function reports both test statistics:

Wald F-statistic:

- Finite-sample distribution: $F(\text{nlags}, T - Kp - c)$ under H_0
- Preferred for small to moderate samples

- Stata-compatible (matches `vargranger` output)

Wald Chi-squared statistic:

- Asymptotic distribution: $\chi^2(\text{nlags})$ under H_0
- Valid in large samples
- Numerically related to F by: $\chi^2 = \text{nlags} \times F$ exactly

where T is effective sample size, K is number of variables, $p = \text{nlags}$, and $c = 1$ if `const = true`, else $c = 0$.

In practice, use the F-statistic and its p-value (`Q.lambdaF`, `Q.Pf`) for inference. The Chi-squared version is provided for completeness and comparison with large-sample approximations.

Lag length selection: Granger causality tests are sensitive to lag length:

- **Too few lags:** Misspecification if true dynamics require more lags. May miss Granger causality relationships or find spurious ones.
- **Too many lags:** Loss of power due to estimating unnecessary parameters. Tests may fail to reject even when causality exists.
- **Optimal lag selection:** Use information criteria (AIC, BIC) or likelihood ratio tests to select lag length before conducting Granger tests.

Best practice: Select lag length using information criteria on the unrestricted VAR, then conduct Granger tests at that lag length. Sensitivity analysis across lag lengths is also informative.

Bidirectional (feedback) causality: Granger causality can be bidirectional:

- Variable j Granger-causes i : `Q.Pf(i,j)` is small
- Variable i Granger-causes j : `Q.Pf(j,i)` is small
- Both can be true simultaneously (feedback relationship)

Example: In a monetary policy VAR, if both `Q.Pf(Output, InterestRate)` and `Q.Pf(InterestRate, Output)` are small, this indicates a feedback system where output and interest rates mutually predict each other.

Verbose output format: When `verbose = true` (default), the function displays:

- Test results for all $K(K - 1)$ pairs (excluding $i = j$)
- F-statistic and p-value for each test
- Significance stars: *** ($p < 0.01$), ** ($p < 0.05$), * ($p < 0.10$)
- Formatted table for easy interpretation

Set `verbose = false` for programmatic use when you want to extract specific results without console output.

Constant term: Including a constant (`const = true`) affects:

- Degrees of freedom in F-test: $T - Kp - 1$ vs $T - Kp$
- Point estimates of VAR coefficients
- Power of the test (slightly different critical values)

In most applications, include a constant unless you have strong theoretical reasons to exclude it (e.g., differenced data with zero mean).

18.6 Examples

Example 1: Basic Granger causality test

Test all pairwise Granger causality relationships in a 3-variable VAR:

```
var_names = {'Output', 'Inflation', 'InterestRate'};
Q = granger(data, 2, var_names, true);

% Displays formatted table with F-statistics and p-values
% Output includes significance stars for quick interpretation
```

Example 2: Silent mode for programmatic access

Extract specific test results without console output:

```
Q = granger(data, 2, var_names, true, false);

% Test: Does Output Granger-cause Inflation?
if Q.Pf(2, 1) < 0.05
    disp('Output Granger-causes Inflation (p < 0.05)');
else
    disp('No evidence of Granger causality from Output to Inflation');
end

% F-statistic for this test
fprintf('F-statistic = %.4f\n', Q.lambdaF(2, 1));
```

Example 3: Full p-value matrix

Display all p-values in matrix form:

```
Q = granger(data, 2, var_names, true, false);

fprintf('P-value matrix (row i = caused, col j = causing):\n');
fprintf('%-16s', '');
for j = 1:K
    fprintf('%-18s', var_names{j});
end
fprintf('\n');
for i = 1:K
    fprintf('%-16s', var_names{i});
    for j = 1:K
        if i == j
            fprintf('%-18s', '---');
        else
            if Q.Pf(i,j) < 0.01
                fprintf('%-18s', [sprintf('%.4f', Q.Pf(i,j)) '***']);
            elseif Q.Pf(i,j) < 0.05
                fprintf('%-18s', [sprintf('%.4f', Q.Pf(i,j)) '**']);
            elseif Q.Pf(i,j) < 0.10
                fprintf('%-18s', [sprintf('%.4f', Q.Pf(i,j)) '*']);
            else
                fprintf('%-18s', sprintf('%.4f', Q.Pf(i,j)));
            end
        end
    end
    fprintf('\n');
end
```

Example 4: Lag length sensitivity analysis

Test how results change with different lag lengths:

```
fprintf('Lag   Output->Inflation   Inflation->InterestRate\n');
fprintf('      F-stat   p-value     F-stat   p-value\n');
for lag = 1:4
    Q = granger(data, lag, var_names, true, false);
    fprintf('%2d   %6.3f   %.4f       %6.3f   %.4f\n', lag, ...
            Q.lambdaF(2,1), Q.Pf(2,1), ...
            Q.lambdaF(3,2), Q.Pf(3,2));
end

% Signal typically strongest when lag matches true DGP
```

Example 5: F-statistic vs Chi-squared relationship

Verify that $\chi^2 = nlags \times F$:

```
nlags = 2;
Q = granger(data, nlags, var_names, true, false);

% Test: Output -> Inflation
fprintf('Output -> Inflation:\n');
fprintf('  F-statistic:   %.4f\n', Q.lambdaF(2,1));
fprintf('  Chi-squared:    %.4f\n', Q.lambdaCHI(2,1));
fprintf('  Chi2 / F:       %.4f (should equal nlags = %d)\n', ...
        Q.lambdaCHI(2,1) / Q.lambdaF(2,1), nlags);

% Relationship holds exactly for all pairs
```

Example 6: Bidirectional causality

Test for feedback relationships:

```
Q = granger(data, 2, {'Output', 'Inflation'}, true, false);

% Check both directions
if Q.Pf(2,1) < 0.05 && Q.Pf(1,2) < 0.05
    disp('Bidirectional causality: Output <-> Inflation');
elseif Q.Pf(2,1) < 0.05
    disp('Unidirectional: Output -> Inflation');
elseif Q.Pf(1,2) < 0.05
    disp('Unidirectional: Inflation -> Output');
else
    disp('No Granger causality in either direction');
end
```

Example 7: Testing a specific causal structure

Verify a hypothesized causal chain: Output $\rightarrow$ Inflation $\rightarrow$ InterestRate:

```
Q = granger(data, 2, var_names, true, false);

% Test forward causality in the chain
forward_chain = (Q.Pf(2,1) < 0.05) && (Q.Pf(3,2) < 0.05);

% Test absence of reverse causality
no_reverse = (Q.Pf(1,2) > 0.10) && (Q.Pf(2,3) > 0.10) && (Q.Pf(1,3) > 0.10);
```

```

if forward_chain && no_reverse
    disp('Data consistent with causal chain: Output -> Inflation -> InterestRate');
else
    disp('Data not consistent with hypothesized causal structure');
end

```

Example 8: With and without constant

Compare results when including vs excluding a constant:

```

Q_nc = granger(data, 2, var_names, false, false); % No constant
Q_c  = granger(data, 2, var_names, true,  false); % With constant

fprintf('%-28s  No Const    With Const\n', 'Test');
fprintf('%-28s  F / p-val   F / p-val\n', '');
fprintf('Output -> Inflation      %.3f / %.3f   %.3f / %.3f\n', ...
        Q_nc.lambdaF(2,1), Q_nc.Pf(2,1), Q_c.lambdaF(2,1), Q_c.Pf(2,1));
fprintf('Inflation -> InterestRate  %.3f / %.3f   %.3f / %.3f\n', ...
        Q_nc.lambdaF(3,2), Q_nc.Pf(3,2), Q_c.lambdaF(3,2), Q_c.Pf(3,2));

% Results typically similar; differences due to df adjustment

```

For complete, executable examples including verification of known causal structures, independent AR processes (null hypothesis true), 4-variable systems, and lag sensitivity analysis, see `example_granger.m` in the package repository.

18.7 Methods and formulas

The Granger causality test is based on the K -variable VAR(p) model:

$$y_t = \nu + A_1 y_{t-1} + A_2 y_{t-2} + \cdots + A_p y_{t-p} + u_t$$

where y_t is $K \times 1$, A_ℓ are $K \times K$ coefficient matrices, and $u_t \sim (0, \Sigma)$.

Null hypothesis: For testing whether variable j Granger-causes variable i :

$$H_0 : [A_1]_{ij} = [A_2]_{ij} = \cdots = [A_p]_{ij} = 0$$

This imposes $r = p$ restrictions on the coefficients.

Restricted model: Under H_0 , the equation for y_{it} excludes all lags of y_{jt} :

$$y_{it} = \nu_i + \sum_{\ell=1}^p \sum_{k \neq j} [A_\ell]_{ik} y_{k,t-\ell} + u_{it}$$

Wald test statistic: Let $\hat{\beta}$ be the stacked coefficient vector from the unrestricted VAR, and let C be the $r \times m$ restriction matrix (where m is the total number of coefficients) that selects the r coefficients being tested. The Wald Chi-squared statistic is:

$$\lambda_{\chi^2} = (C\hat{\beta})' [C\widehat{\text{Var}}(\hat{\beta})C']^{-1} (C\hat{\beta})$$

The estimated covariance matrix of $\hat{\beta}$ is:

$$\widehat{\text{Var}}(\hat{\beta}) = (ZZ')^{-1} \otimes \hat{\Sigma}$$

where Z is the matrix of stacked regressors and $\otimes$ denotes the Kronecker product.

Under H_0 :

$$\lambda_{\chi^2} \xrightarrow{d} \chi^2(r)$$

F-statistic version:

$$\lambda_F = \frac{\lambda_{\chi^2}}{r}$$

Under H_0 and finite-sample normality:

$$\lambda_F \sim F(r, T - Kp - c)$$

where T is the effective sample size, K is the number of variables, p is the number of lags, and $c = 1$ if a constant is included, else $c = 0$.

The exact relationship is:

$$\lambda_{\chi^2} = r \cdot \lambda_F = p \cdot \lambda_F$$

since $r = p$ (number of restrictions equals number of lags).

P-values:

- F-test: $p = P(F_{r, T-Kp-c} > \lambda_F)$
- Chi-squared test: $p = P(\chi_r^2 > \lambda_{\chi^2})$

Decision rule: At significance level α :

- Reject H_0 if $p < \alpha$: Conclude variable j Granger-causes variable i
- Fail to reject if $p \geq \alpha$: No evidence of Granger causality from j to i

18.8 References

Granger, C. W. J. 1969. "Investigating Causal Relations by Econometric Models and Cross-spectral Methods." *Econometrica* 37(3): 424–438.

Lütkepohl, H. 2005. *New Introduction to Multiple Time Series Analysis*. Springer-Verlag. Chapter 2.3 (Granger Causality).

Hamilton, J. D. 1994. *Time Series Analysis*. Princeton University Press. Chapter 11 (Vector Autoregressions).

Sims, C. A. 1980. "Macroeconomics and Reality." *Econometrica* 48(1): 1–48.